

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGUYỄN HOÀNG ANH

PHÁT HIỆN TIN GIẢ TIẾNG VIỆT SỬ DỤNG HỌC
MÁY VÀ HỌC SÂU

ĐỒ ÁN NGÀNH
NGÀNH TRÍ TUỆ NHÂN TẠO

TP. HỒ CHÍ MINH, NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGUYỄN HOÀNG ANH

PHÁT HIỆN TIN GIẢ TIẾNG VIỆT SỬ DỤNG HỌC
MÁY VÀ HỌC SÂU

Mã số sinh viên: 2351010009

ĐỒ ÁN NGÀNH
NGÀNH TRÍ TUỆ NHÂN TẠO

Giảng viên hướng dẫn: TS. LÊ QUANG MINH

TP. HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn sâu sắc đến **TS. LÊ QUANG MINH** giảng viên hướng dẫn của đề án, người đã tận tình định hướng, chỉ bảo tỉ mỉ và liên tục động viên em xuyên suốt toàn bộ quá trình nghiên cứu cũng như hoàn thiện đề án ngành.

Em cũng xin bày tỏ lòng biết ơn chân thành đến quý thầy cô thuộc Đào tạo Đặc biệt Trường Đại học Mở TP. Hồ Chí Minh, những người đã truyền thụ cho em nền tảng kiến thức chuyên ngành **TRÍ TUỆ NHÂN TẠO** cùng những hành trang nghề nghiệp quý báu trong suốt những năm học vừa qua.

Em cũng xin cảm ơn gia đình và những người đã luôn lắng nghe và khuyến khích em trong suốt chặng đường thực hiện đề án. Do thời gian và kiến thức còn nhiều giới hạn, đề án khó tránh khỏi những thiếu sót và điểm chưa hoàn thiện. Em kính mong nhận được sự góp ý từ quý thầy cô để có thể không ngừng hoàn thiện bản thân, nâng cao kỹ năng và chất lượng các nghiên cứu trong tương lai.

Thành phố Hồ Chí Minh, tháng 9 năm 2026

Sinh viên thực hiện

NGUYỄN HOÀNG ANH

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Giảng viên hướng dẫn: TS. Lê Quang Minh

Sinh viên thực hiện: Nguyễn Hoàng Anh

Ngày sinh: 27/06/2005

Lớp: DH23AI01C

Nơi sinh: TP Hồ Chí Minh

Tên đề tài: PHÁT HIỆN TIN GIẢ TIẾNG VIỆT SỬ DỤNG HỌC MÁY VÀ HỌC SÂU

Nội dung nhận xét của giảng viên hướng dẫn:

[illegible]

Thành phố Hồ Chí Minh, ngày ... tháng ... năm 2026

Người nhận xét

Mục lục

Danh mục viết tắt	IX
Danh mục hình vẽ	XI
Danh mục bảng biểu	XII
1 Tổng quan về đề tài	1
1.1 Bối cảnh và động lực nghiên cứu	1
1.2 Mục tiêu của đề án	1
1.3 Phạm vi nghiên cứu	2
1.4 Cấu trúc báo cáo	2
2 Nền tảng lý thuyết	4
2.1 Các khái niệm cơ bản	4
2.1.1 Văn bản và biểu diễn văn bản	4
2.1.2 Đặc trưng văn bản và tiền xử lý	5
2.1.3 Phân loại nội dung văn bản	6
2.2 Đặc trưng và xử lý ngôn ngữ tiếng Việt	7
2.2.1 Những đặc trưng của tiếng Việt	7
2.2.2 Phân đoạn từ (Word Segmentation)	8
2.2.3 Biểu diễn văn bản số	8
2.2.4 Quá trình hình thành, phát triển và các nghiên cứu liên quan	8
2.3 Học máy	9
2.3.1 Định nghĩa	9
2.3.2 Các mô hình học máy cho bài toán phân loại	10
2.3.2.1 Logistic Regression (LR)	10
2.3.2.2 Support Vector Machine (SVM)	10
2.3.3 Biểu diễn đặc trưng cơ bản	12

	2.3.3.1	TF-IDF (Term Frequency – Inverse Document Frequency)	12
2.4	Học sâu		14
	2.4.1	Định nghĩa	14
	2.4.2	Mạng nơ-ron nhân tạo (Artificial Neural Network)	14
	2.4.2.1	Một số thành phần cần chú ý	16
	2.4.3	Recurrent Neural Networks (RNN)	17
	2.4.4	Long Short-Term Memory (LSTM)	17
	2.4.5	Mạng LSTM hai chiều (Bidirectional LSTM)	18
	2.4.6	Cơ chế chú ý (Attention Mechanism)	19
2.5	Biểu diễn phân tán và mã hóa con (Distributed & Subword Representations)		20
	2.5.1	Word2Vec và GloVe	21
	2.5.2	FastText và thông tin subword	22
	2.5.3	Mã hóa con (Subword Tokenization)	22
2.6	Mô hình Transformer và phương thức Self-Attention		23
	2.6.1	Cơ chế Scaled Dot-Product Attention	23
	2.6.2	Multi-Head Attention và Positional Encoding	23
	2.6.3	Encoder và Decoder	26
2.7	PhoBERT và mô hình ngôn ngữ tiền huấn luyện		26
	2.7.1	Tiền huấn luyện (Pre-Training) và tinh chỉnh (Fine-Tuning)	26
	2.7.2	PhoBERT cho tiếng Việt	27
	2.7.3	Một số công trình liên quan về phát hiện tin giả tiếng Việt	28
2.8	Đánh giá mô hình và phân tích hiệu chuẩn		28
	2.8.1	Các độ đo hiệu năng phân loại	28
	2.8.2	Kiểm định thống kê	29
	2.8.3	Hiệu chuẩn xác suất (Probability Calibration)	30
2.9	Kết luận chương		31
3	Bộ dữ liệu và tiền xử lý		32
3.1	Thu thập và mô tả bộ dữ liệu		32
	3.1.1	Nguồn dữ liệu	32
	3.1.2	Phân phối nhân	33
3.2	Tiền xử lý văn bản		34
	3.2.1	Làm sạch và lọc dữ liệu	34
	3.2.2	Phân đoạn từ bằng VnCoreNLP	35

3.3	Chia tập dữ liệu	36
3.3.1	Chia phân tầng	36
3.3.2	Kiểm tra rò rỉ dữ liệu	38
3.4	Kết luận chương	38
4	Xác định tin giả	40
4.1	Phương pháp đề xuất	40
4.2	Lý do lựa chọn các nhóm mô hình	41
4.3	Tổng quan kiến trúc hệ thống	42
4.4	Trích xuất đặc trưng	43
4.4.1	TF-IDF cho mô hình học máy truyền thống	43
4.4.1.1	Lý do lựa chọn TF-IDF	43
4.4.1.2	Cấu hình TF-IDF	43
4.4.2	Word Embedding cho BiLSTM	44
4.4.3	Tokenization PhoBERT (BPE)	45
4.5	Các mô hình phân loại	45
4.5.1	Các mô hình học máy truyền thống	45
4.5.1.1	Logistic Regression	45
4.5.1.2	Support Vector Machine	46
4.5.2	Các mô hình học sâu	46
4.5.2.1	Mạng BiLSTM	46
4.5.2.2	Tinh chỉnh PhoBERT	47
4.6	Chiến lược xử lý mất cân bằng nhãn	47
4.7	Các phân tích mở rộng được thiết kế cho chương thực nghiệm	48
4.7.1	Phân tích khả năng giải thích (Explainability)	48
4.7.2	Hiệu chỉnh xác suất sau huấn luyện (Post-Hoc Calibration)	48
4.7.3	Phân tích các trường hợp khó (Hard Cases)	49
4.7.4	Knowledge Distillation	49
4.8	Kết luận chương	50
5	Kết quả thực nghiệm	51
5.1	Môi trường thực nghiệm	51
5.2	So sánh hiệu năng các mô hình	52
5.2.1	Kết quả tổng thể	52

5.2.2	Hiệu năng theo từng lớp	53
5.2.3	Ma trận nhầm lẫn	54
5.2.4	Đường cong ROC và Precision-Recall	55
5.2.5	Phân tích hiệu chuẩn xác suất (Calibration Analysis)	55
5.3	Kiểm định thống kê	58
5.3.1	Kiểm định McNemar	58
5.3.2	Khoảng tin cậy Bootstrap	59
5.3.3	Kiểm tra chéo (Cross-Validation)	61
5.4	Nghiên cứu triệt bỏ (Ablation Study)	62
5.5	Phân tích lỗi	65
5.5.1	Thống kê lỗi tổng quan	65
5.5.1.1	So sánh FP và FN	65
5.5.1.2	Phân phối lỗi theo độ dài văn bản	66
5.5.1.3	Hàm ý cải tiến mô hình	67
5.5.2	Phân tích khả năng giải thích	68
5.5.2.1	Phân tích khả năng giải thích cấp token (Token-Level Explainability)	69
5.5.2.2	Hiệu chỉnh xác suất sau huấn luyện (Post-Hoc Calibration)	72
5.5.2.3	Phân tích chuyên sâu các trường hợp khó (Hard Cases Deep Analysis)	74
5.5.2.4	Knowledge Distillation – Mô hình sinh viên nhỏ, chất lượng tiệm cận giáo viên	77
5.6	Kết luận chương	78
Kết luận		79
Phụ lục		85

DANH MỤC VIẾT TẮT

AUC	Area Under the ROC Curve
BERT	Bidirectional Encoder Representations from Transformers
BPE	Byte Pair Encoding
CI	Confidence Interval
CV	Cross-Validation
ECE	Expected Calibration Error
F1	F1-Score
FFN	Feed-Forward Network
FN	False Negative
FP	False Positive
GPU	Graphics Processing Unit
IDF	Inverse Document Frequency
KD	Knowledge Distillation
LR	Logistic Regression
MCE	Maximum Calibration Error
NLP	Natural Language Processing
PhoBERT	Pre-trained language model for Vietnamese (VinAI)
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
TF	Term Frequency
TN	True Negative
TP	True Positive

Danh mục hình vẽ

2.1	Minh họa phương pháp biểu diễn văn bản bằng TF-IDF ¹	5
2.2	Hình ảnh minh họa bước tiền xử lý văn bản. ¹	6
2.3	Minh họa bài toán phân loại văn bản ¹	7
2.4	Minh họa thuật toán nhị phân SVM ¹	12
2.5	Ví dụ minh họa cách tính TF-IDF trên hai tài liệu ¹	13
2.6	Minh họa mạng nơ-ron nhân tạo ¹	15
2.7	Minh họa kiến trúc nhiều lớp của mạng nơ-ron nhân tạo ¹	15
2.8	Minh họa cấu trúc mạng LSTM ¹	18
2.9	Minh họa cấu trúc mạng LSTM hai chiều ¹	19
2.10	Minh họa kiến trúc Transformer encoder ¹	25
3.1	Phân phối nhãn theo từng tập dữ liệu và tổng quan toàn bộ 15.789 mẫu . . .	33
3.2	Phân phối nhãn trong các tập dữ liệu huấn luyện, kiểm định và kiểm tra . . .	37
5.1	So sánh các chỉ số hiệu năng (Accuracy, F1-Score, ROC-AUC) của bốn mô hình trên tập kiểm tra	52
5.2	Precision, Recall và F1-Score theo từng lớp (<i>thật</i> / <i>giả</i>) cho bốn mô hình . .	53
5.3	Ma trận lỗi của bốn mô hình khi thực hiện dự đoán trên tập kiểm tra (2.094 mẫu)	54
5.4	Đường cong ROC và Precision-Recall của bốn mô hình trên tập kiểm tra . . .	55
5.5	Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình trên tập kiểm tra. Đường chéo nét đứt biểu diễn hiệu chuẩn hoàn hảo; các điểm vuông biểu diễn tỷ lệ dương tính thực tế trong mỗi bin xác suất.	57
5.6	So sánh số lượng False Positive (FP) và False Negative (FN) trong quá trình phân loại	65
5.7	Phân phối lỗi phân loại theo độ dài văn bản	66
5.8	Top 15 đặc trưng TF-IDF có hệ số LR cao nhất (dương = chỉ điểm <i>giả</i> , âm = chỉ điểm <i>thật</i>)	68

5.9	Phân loại lỗi theo độ dài văn bản và mức độ tin cậy dự đoán	69
5.10	Token attribution bằng SHAP cho Logistic Regression và SVM trên cùng một mẫu kiểm tra.	71
5.11	Jaccard overlap của top-10 token quan trọng giữa các phương pháp attribution, trung bình trên 20 mẫu kiểm tra.	72
5.12	Biểu đồ hiệu chuẩn của bốn mô hình trước và sau khi áp dụng post-hoc calibration.	74
5.13	Phân bố của 85 mẫu khó theo chủ đề, độ dài, lý do thất bại và độ tự tin của từng mô hình.	76

Danh mục bảng biểu

3.1	Thống kê tập dữ liệu	37
5.1	Siêu tham số các mô hình	51
5.2	Độ phức tạp và thời gian huấn luyện	52
5.3	So sánh hiệu suất các mô hình	52
5.4	Chỉ số hiệu suất theo lớp	53
5.5	Chỉ số hiệu chuẩn xác suất của bốn mô hình trên tập kiểm tra	57
5.6	Kết quả kiểm định McNemar (Sau hiệu chỉnh Holm-Bonferroni)	59
5.7	Khoảng tin cậy 95% Bootstrap [1] cho F1-Score (10.000 lần lấy mẫu lại)	60
5.8	Kết quả kiểm tra chéo 5-fold (3 seeds)	61
5.9	Kết quả Ablation Study trên tập kiểm tra	63
5.10	Phân tích lỗi trên tập kiểm tra	67
5.11	Faithfulness of token-level explanations	72
5.12	So sánh hiệu chuẩn sau post-hoc calibration	74
5.13	Các mẫu khó được phân tích chi tiết	76
5.14	So sánh teacher và student sau Knowledge Distillation	78
5.15	Số mẫu theo lớp của từng tập	85
5.16	Cấu hình TF-IDF, LR, SVM	87
5.17	Cấu hình BiLSTM và PhoBERT	87
5.18	Metric tổng hợp trên tập test	88
5.19	Precision/Recall/F1 theo lớp	88
5.20	Confusion matrix trên tập test	89
5.21	Khoảng tin cậy Bootstrap 95% cho F1-score	89
5.22	So sánh giáo viên – sinh viên	89
5.23	Faithfulness của sáu phương pháp attribution	90
5.24	Cross-validation 5-fold	90
5.25	Phân bố hard cases theo lý do	90

Chương 1

TỔNG QUAN VỀ ĐỀ TÀI

1.1 Bối cảnh và động lực nghiên cứu

Sự phát triển của môi trường thông tin trực tuyến đã làm thay đổi đáng kể cách người dùng Việt Nam thu nhận và truyền tải thông tin. Các nền tảng như Facebook, TikTok, Zalo cùng với những trang tin điện tử cho phép nội dung được phổ biến đến số lượng lớn người dùng trong thời gian ngắn. Tuy nhiên, tốc độ lan truyền cao cũng khiến những nội dung chưa được kiểm chứng dễ dàng tiếp cận cộng đồng, từ đó làm gia tăng nguy cơ thông tin sai lệch được tiếp nhận và chia sẻ rộng rãi.

Khối lượng nội dung ngày càng lớn khiến việc xác minh thủ công từng bài viết trở nên khó duy trì ở quy mô thực tế. Nếu quá trình kiểm tra phụ thuộc hoàn toàn vào con người, chi phí về thời gian và nhân lực sẽ tăng đáng kể khi số lượng bài viết được tạo ra mỗi ngày càng nhiều. Đây là động lực để nghiên cứu các phương pháp tự động hóa quá trình phát hiện thông tin sai lệch, đặc biệt thông qua việc kết hợp các kỹ thuật học máy và học sâu trong xử lý văn bản tiếng Việt.

1.2 Mục tiêu của đề án

Từ nhận định trên, đề án đặt ra bốn mục tiêu cụ thể:

- Xây dựng và đánh giá song song bốn mô hình phân lớp cho văn bản tiếng Việt, gồm: Hồi quy Logistic (LR), Máy vectơ hỗ trợ (SVM), mạng LSTM hai chiều (BiLSTM) và mô hình ngôn ngữ tiền huấn luyện PhoBERT.

- Khảo sát tác động của bước phân đoạn từ — một đặc thù riêng của tiếng Việt — đến chất lượng dự đoán của mỗi mô hình.
- Đánh giá đa chiều: kiểm định thống kê, kiểm tra chéo và phân tích khả năng giải thích của mô hình.
- Từ các kết quả thu được, chỉ ra phương pháp phù hợp nhất cho bài toán phát hiện thông tin sai lệch trên tiếng Việt.

1.3 Phạm vi nghiên cứu

Nghiên cứu tập trung vào việc nhận diện tin giả trong văn bản tiếng Việt dưới dạng bài toán phân loại hai lớp, tương ứng với *thật* và *giả*. Dữ liệu đầu vào được hình thành từ hai thành phần của bài viết là tiêu đề và nội dung, qua đó cung cấp thông tin cho quá trình phân loại. Tập dữ liệu được thu thập từ các bài đăng công khai trên một số nền tảng mạng xã hội phổ biến tại Việt Nam và từ các bài viết xuất hiện trên các trang báo điện tử.

Các dạng dữ liệu và nhiệm vụ nằm ngoài phạm vi nghiên cứu gồm nội dung hình ảnh, video và các tín hiệu lan truyền trên môi trường mạng. Đề án cũng không xem xét bài toán phát hiện tin giả trong nhiều ngôn ngữ, đồng thời không triển khai các hình thức phân loại đa lớp hoặc đa nhãn.

1.4 Cấu trúc báo cáo

Nội dung của đề án được tổ chức theo trình tự từ việc xác định bài toán đến xây dựng phương pháp, thực nghiệm và phân tích kết quả. Báo cáo gồm năm chương và phần kết luận, với nội dung chính của từng phần được trình bày như sau:

- **Chương 1:** Đặt nền tảng cho đề tài thông qua việc trình bày vấn đề nghiên cứu, mục tiêu hướng đến và phạm vi triển khai. Nội dung chương cũng cung cấp cái nhìn tổng quan về bố cục của các phần tiếp theo trong báo cáo.
- **Chương 2:** Cung cấp những kiến thức nền tảng cho nghiên cứu, bao gồm xử lý ngôn ngữ tự nhiên tiếng Việt, những phương pháp học máy và kiến trúc học sâu được sử dụng. Các khái niệm liên quan đến biểu diễn văn bản và mô hình ngôn ngữ cũng được nhắc tới trong chương.

- **Chương 3:** Trình bày quá trình chuẩn bị dữ liệu cho thực nghiệm, từ mô tả nguồn dữ liệu, các bước tiền xử lý đến phương pháp phân chia dữ liệu thành các tập Train, Validation và Test.
- **Chương 4:** Trình bày quá trình triển khai bốn mô hình phát hiện tin giả, từ việc xây dựng đầu vào và thiết lập quá trình huấn luyện đến các phương pháp xử lý sự chênh lệch giữa các lớp dữ liệu.
- **Chương 5:** Phân tích các kết quả thu được qua thực nghiệm và đánh giá hiệu năng của từng mô hình. Chương cũng xem xét các trường hợp dự đoán sai, thực hiện kiểm định thống kê và tiến hành các thí nghiệm ablation để đánh giá tác động của từng thành phần trong hệ thống.
- **Phần kết luận:** khái quát những kết quả chính của nghiên cứu, đồng thời thảo luận các hạn chế còn tồn tại và đề xuất những hướng phát triển có thể thực hiện trong tương lai.

Chương 2

NỀN TẢNG LÝ THUYẾT

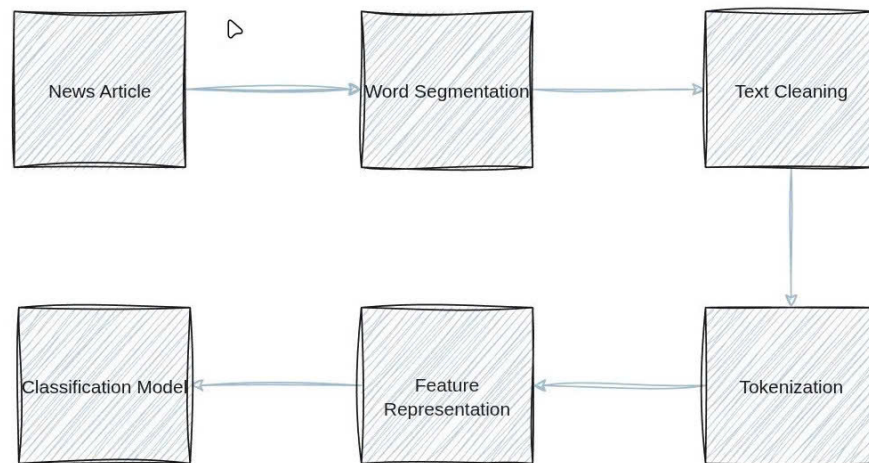
Chương này tập hợp các kiến thức nền mà đề án khai thác để huấn luyện và đánh giá bốn mô hình nhận dạng tin giả. Hành trình nội dung được bố trí tăng dần: bắt đầu từ những khái niệm nền — văn bản, đặc trưng, và phép ánh xạ sang không gian số; tiếp đó giới thiệu các thuật toán học máy kinh điển gồm Hồi quy Logistic và Máy vectơ hỗ trợ; sau đó mở rộng sang các kiến trúc học sâu — mạng hồi tiếp, LSTM hai chiều, cơ chế chú ý, Transformer; và khép lại ở PhoBERT. Phần cuối chương hệ thống lại các độ đo hiệu năng, kiểm định thống kê và phép phân tích hiệu chuẩn xác suất — bộ công cụ đi kèm xuyên suốt các chương thực nghiệm ở phía sau.

2.1 Các khái niệm cơ bản

2.1.1 Văn bản và biểu diễn văn bản

Trong các hệ thống xử lý ngôn ngữ tự nhiên, dạng dữ liệu được sử dụng rộng rãi nhất chính là văn bản — phương tiện giao tiếp phổ biến giữa người với người. Xét về cấu trúc, một văn bản là chuỗi nhiều câu, mỗi câu lại gồm nhiều từ và ký tự. Vấn đề là máy tính không trực tiếp xử lý được chuỗi ký tự thô; vì vậy văn bản cần phải đổi thành một dạng biểu diễn số để các thuật toán học máy có thể thao tác. Hai kỹ thuật kinh điển thường được sử dụng cho mục đích này là bag-of-words và TF-IDF, từ được chuyển thành các vector trong một không gian nhiều chiều, với mỗi chiều biểu thị một mục từ riêng biệt trong từ vựng.. Minh họa trực quan cho cách biểu diễn bằng TF-IDF có thể xem tại Hình 2.1.

Test preprocessing pipeline

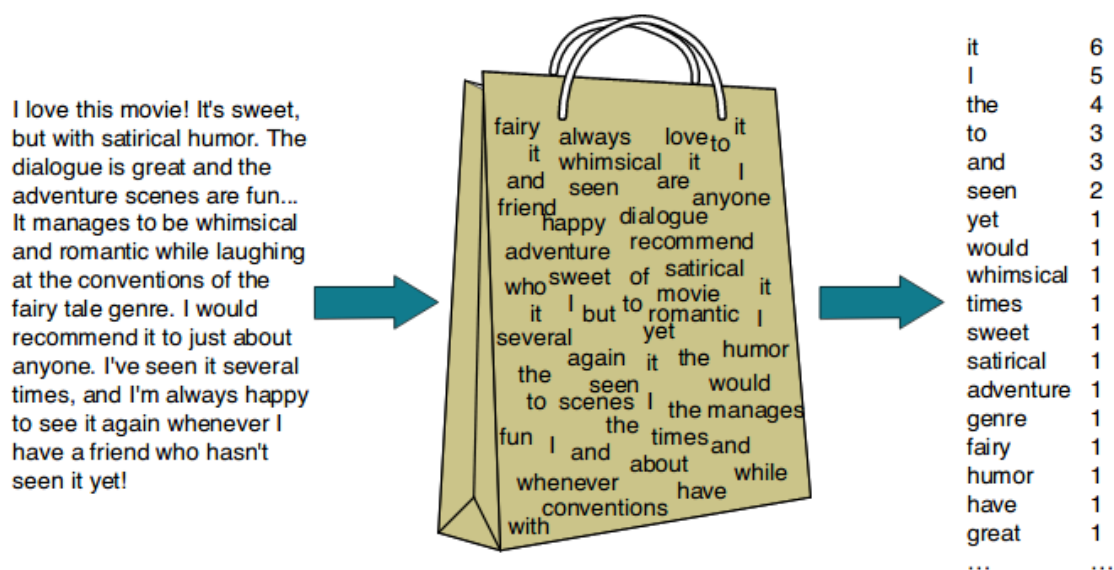


Hình 2.1: Minh họa phương pháp biểu diễn văn bản bằng TF-IDF¹

2.1.2 Đặc trưng văn bản và tiền xử lý

Đặc trưng văn bản là vector số rút ra từ văn bản gốc, đóng vai trò đầu vào cho mô hình ở cả giai đoạn huấn luyện lẫn suy diễn. Tuy nhiên, trước khi trích xuất, dữ liệu thô cần được đưa về một dạng thống nhất để loại bỏ nhiễu và san phẳng sự chênh lệch giữa các tài liệu. Các bước chuẩn bị dữ liệu tập trung vào việc đưa văn bản về dạng thống nhất và loại bỏ những ký tự đặc biệt không cần thiết, tách từ và lọc stopwords. Riêng với tiếng Việt, khâu tách từ đặc biệt quan trọng vì ranh giới từ và âm tiết không trùng nhau, cùng với sự phong phú của từ ghép, từ láy.

¹*nguồn ảnh:* <https://www.linkedin.com/pulse/creating-tf-idf-model-from-scratch-python-noor-saeed>

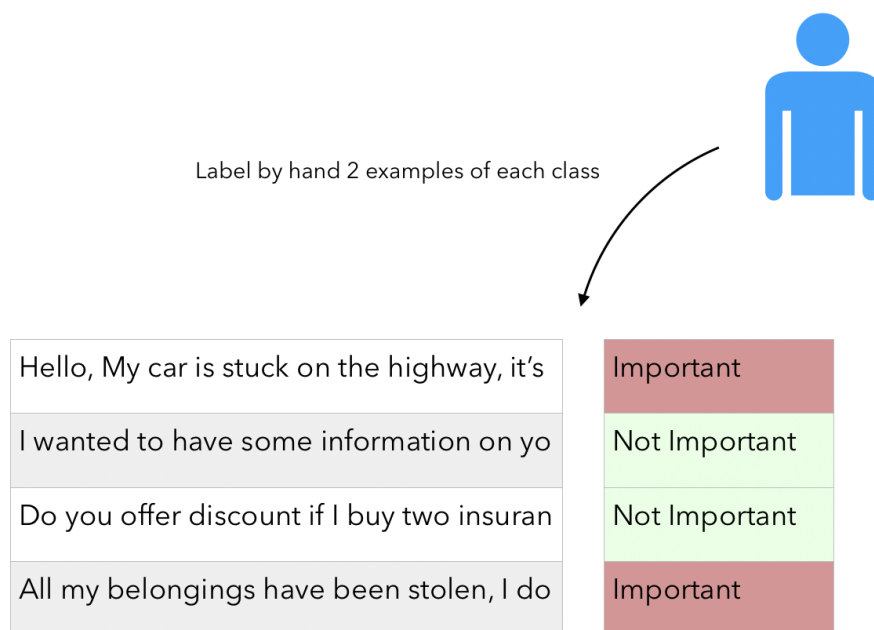


Hình 2.2: Hình ảnh minh họa bước tiền xử lý văn bản.²

2.1.3 Phân loại nội dung văn bản

Về bản chất, phát hiện tin giả là bài toán phân loại nhị phân: ứng với mỗi văn bản đầu vào, mô hình cần gán một trong hai nhãn — *thật* hoặc *giả*. Xét dưới góc độ tính toán, bài toán được xem như việc học một ánh xạ từ không gian đặc trưng sang không gian nhãn, sao cho mô hình có thể phân biệt chính xác các mẫu thuộc từng lớp trong tập huấn luyện.

²nguồn ảnh: <https://www.couchbase.com/blog/data-preprocessing-in-machine-learning/>



Hình 2.3: Minh họa bài toán phân loại văn bản³

2.2 Đặc trưng và xử lý ngôn ngữ tiếng Việt

2.2.1 Những đặc trưng của tiếng Việt

Khi đặt cạnh tiếng Anh hay phần lớn ngôn ngữ châu Âu, tiếng Việt sở hữu hàng loạt đặc thù cấu trúc mà người xây dựng hệ thống NLP cần nắm rõ. Tiếng Việt là ngôn ngữ đơn lập (isolating) và không biến hình (non-inflectional); các đặc trưng nổi bật gồm:

- **Không có dấu phân cách âm tiết:** ranh giới giữa các từ và âm tiết không trùng khớp nhau. Ví dụ, “thành phố” là một từ duy nhất nhưng được viết thành hai âm tiết tách rời.
- **Từ ghép và từ láy phong phú:** nhiều khái niệm được diễn đạt bằng cụm nhiều âm tiết, chẳng hạn các từ ghép “học_sinh”, “kinh_tế” hay từ láy “lung_linh”.
- **Hệ thanh điệu:** tiếng Việt có đến sáu thanh; mỗi thanh mang một giá trị ngữ nghĩa riêng mà không làm thay đổi cấu trúc hình thái của từ.

³nguồn ảnh: https://maelfabien.github.io/machinelearning/NLP_5/

- **Ngôn ngữ cô lập về mặt hình thái:** các quan hệ ngữ pháp chủ yếu được thể hiện thông qua trật tự từ và hư từ, thay vì qua biến đổi hình thái như ở tiếng Anh.

Chính những đặc điểm này đòi hỏi phải có một bước tiền xử lý đặc thù cho tiếng Việt — nổi bật nhất là *phân đoạn từ* — trước khi áp dụng phần lớn những mô hình hiện đại.

2.2.2 Phân đoạn từ (Word Segmentation)

Phân đoạn từ là bước xác định ranh giới giữa các từ trong một câu tiếng Việt. Hầu hết mô hình NLP — đặc biệt là PhoBERT — đều được huấn luyện trên dữ liệu đã qua phân đoạn từ. Vì vậy, cách tách từ phải nhất quán giữa hai giai đoạn huấn luyện và suy diễn; nếu cách tách lệch với từ vựng mà mô hình ngôn ngữ đã học, chất lượng biểu diễn embedding sẽ giảm sút đáng kể.

Ví dụ, một câu gốc có thể được phân đoạn thành:

Trước: “món ăn này rất ngon đối với em.” *Sau:* “món_ăn này rất ngon đối_với em.”

2.2.3 Biểu diễn văn bản số

Để đưa văn bản thô vào mô hình học máy, cần chuyển sang dạng số. Trong nghiên cứu này, ba phương pháp biểu diễn được sử dụng, mỗi phương pháp phù hợp với một nhóm mô hình khác nhau:

1. **TF-IDF:** vector đặc trưng thừa dựa trên tần suất từ — dùng cho LR và SVM.
2. **Word Embedding:** vector dày đặc học được [2], khởi tạo từ FastText tiếng Việt [3] — dùng cho BiLSTM.
3. **Subword Tokenization (BPE):** tách từ thành các đơn vị con, đặc biệt phù hợp với PhoBERT.

2.2.4 Quá trình hình thành, phát triển và các nghiên cứu liên quan

Nghiên cứu xử lý ngôn ngữ tự nhiên (NLP) cho tiếng Việt phải giải quyết nhiều đặc trưng riêng của ngôn ngữ này. Một vấn đề quan trọng là xác định đơn vị từ, bởi khoảng trắng trong văn bản không phải luôn biểu thị ranh giới giữa các từ. Một từ có thể gồm nhiều âm tiết được

viết cách nhau, do đó việc phân đoạn từ thường được thực hiện như một bước chuẩn bị trước khi áp dụng các phương pháp NLP. Cùng với đó, nhu cầu xử lý tiếng Việt cũng thúc đẩy sự phát triển của các tài nguyên và công cụ chuyên dụng.

Ở giai đoạn đầu của quá trình nghiên cứu, đặc biệt trước năm 2010, phân đoạn từ là một trong những hướng được quan tâm nhiều và CRF (Conditional Random Fields) được sử dụng phổ biến cho nhiệm vụ này. Từ khoảng năm 2015, các phương pháp học sâu dần được áp dụng vào nhiều bài toán NLP tiếng Việt, chẳng hạn phân đoạn từ, gán nhãn từ loại và phân tích cú pháp. RDRSegmenter [4] là một công cụ tiêu biểu cho hướng tiếp cận này, kết hợp mạng nơ-ron hồi quy với cơ chế suy luận ngược (Backward Inference) và đạt độ chính xác trên 98% trên tập VLSP 2013.

Những nghiên cứu trước cho thấy việc xác định nội dung sai lệch không chỉ phụ thuộc vào đặc điểm ngữ nghĩa của văn bản mà còn liên quan đến phong cách ngôn ngữ, ngữ cảnh lan truyền và độ tin cậy của nguồn thông tin [5]. Từ những yếu tố này, có thể nhận thấy ba nhóm phương pháp chính: kết hợp các mô hình học máy với đặc trưng do con người xây dựng; khai thác khả năng biểu diễn của các kiến trúc học sâu như BiLSTM và CNN; hoặc sử dụng các mô hình ngôn ngữ tiền huấn luyện thuộc họ BERT như BERT, RoBERTa và PhoBERT [6]. Bên cạnh nhiệm vụ phát hiện tin giả, các nghiên cứu trong CheckThat! Lab [7] cũng quan tâm đến bài toán xác định mức độ đáng kiểm chứng của thông tin (Check-Worthiness), có mối liên hệ với việc đánh giá nội dung cần được xác minh.

Trên cơ sở các hướng tiếp cận trên, nghiên cứu lựa chọn bốn mô hình gồm LR, SVM, BiLSTM và PhoBERT để tiến hành thực nghiệm. Bốn phương pháp này tạo thành các nhóm có mức độ biểu diễn và độ phức tạp khác nhau.

2.3 Học máy

2.3.1 Định nghĩa

Học máy (Machine Learning) có thể được xem là một hướng tiếp cận trong trí tuệ nhân tạo, trong đó mô hình được xây dựng để rút ra thông tin và quy luật từ dữ liệu thay vì phải xác định trước toàn bộ các bước xử lý. Quá trình học cho phép mô hình sử dụng những mẫu dữ liệu đã để hình thành tiền đề cho việc dự đoán trên các dữ liệu khác.

Đối với bài toán phát hiện tin giả của đề án, dữ liệu được gán vào một trong hai lớp *thật* và *giả*, do đó bài toán được triển khai theo phương pháp *học có giám sát* (Supervised Learning). Mô hình sử dụng các mẫu được dán nhãn trong tập huấn luyện để tìm ra mối liên hệ giữa các

đặc trưng đầu vào và nhãn dự đoán. Sau khi học xong, mô hình sẽ xác định nhãn cho những văn bản không có trong tập dữ liệu huấn luyện.

Học máy còn bao gồm nhiều cách tiếp cận khác, chẳng hạn học không giám sát (Unsupervised Learning), học bán giám sát (Semi-Supervised Learning) và học tăng cường (Reinforcement Learning). Do không nằm trong phạm vi thực nghiệm của đề tài, các phương pháp này chỉ được giới thiệu ở mức tổng quan.

2.3.2 Các mô hình học máy cho bài toán phân loại

2.3.2.1 Logistic Regression (LR)

Thuộc nhóm phương pháp học có giám sát, Logistic Regression (LR) [8] được áp dụng rộng rãi trong các bài toán phân loại, trong đó trường hợp hai lớp là một ứng dụng phổ biến. Đầu ra của Logistic Regression được biểu diễn dưới dạng một giá trị trong khoảng $(0, 1)$, phản ánh mức độ thuộc về lớp đang xét. Giá trị này được xác định bằng hàm sigmoid với đầu vào là tổ hợp tuyến tính của các đặc trưng và hệ số chệch b . Với $\mathbf{x} \in \mathbb{R}^d$ biểu diễn vector đặc trưng và $\mathbf{w} \in \mathbb{R}^d$ là vector trọng số, xác suất để mẫu được xếp vào lớp $y = 1$ được xác định bởi:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))}$$

Binary Cross-Entropy (Log-Loss) được sử dụng để đánh giá sai lệch giữa đầu ra xác suất của mô hình và nhãn thực tế trong quá trình huấn luyện. Việc tối ưu các tham số được thực hiện theo hướng giảm giá trị của hàm mất mát này trên toàn bộ các mẫu dữ liệu:

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Với mẫu thứ i , giá trị $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$ biểu thị xác suất mà mô hình gán cho lớp dương. Do hàm sigmoid đưa kết quả về khoảng $(0, 1)$, đầu ra của Logistic Regression có thể được diễn giải trực tiếp dưới dạng xác suất. Bên cạnh ưu điểm về khả năng giải thích thông qua các hệ số trọng số, mô hình có số lượng phép tính tương đối đơn giản, nhờ đó thích hợp làm mô hình đối chứng với chi phí huấn luyện và suy diễn thấp.

2.3.2.2 Support Vector Machine (SVM)

Trong nhóm các thuật toán phân loại có giám sát, Support Vector Machine (SVM) [9] nổi bật với cách tiếp cận dựa trên việc xác định ranh giới phân tách giữa các lớp dữ liệu. Với

trường hợp phân loại nhị phân, mô hình xác định một siêu phẳng (Hyperplane) làm ranh giới giữa hai lớp. Thay vì chỉ tìm một ranh giới có khả năng phân chia dữ liệu, SVM xác định siêu phẳng phân loại dựa trên việc tối đa hóa khoảng cách đến các điểm dữ liệu gần ranh giới. Khoảng cách này được gọi là biên (margin), và việc tối đa hóa margin giúp xác định một ranh giới phân loại phù hợp hơn.

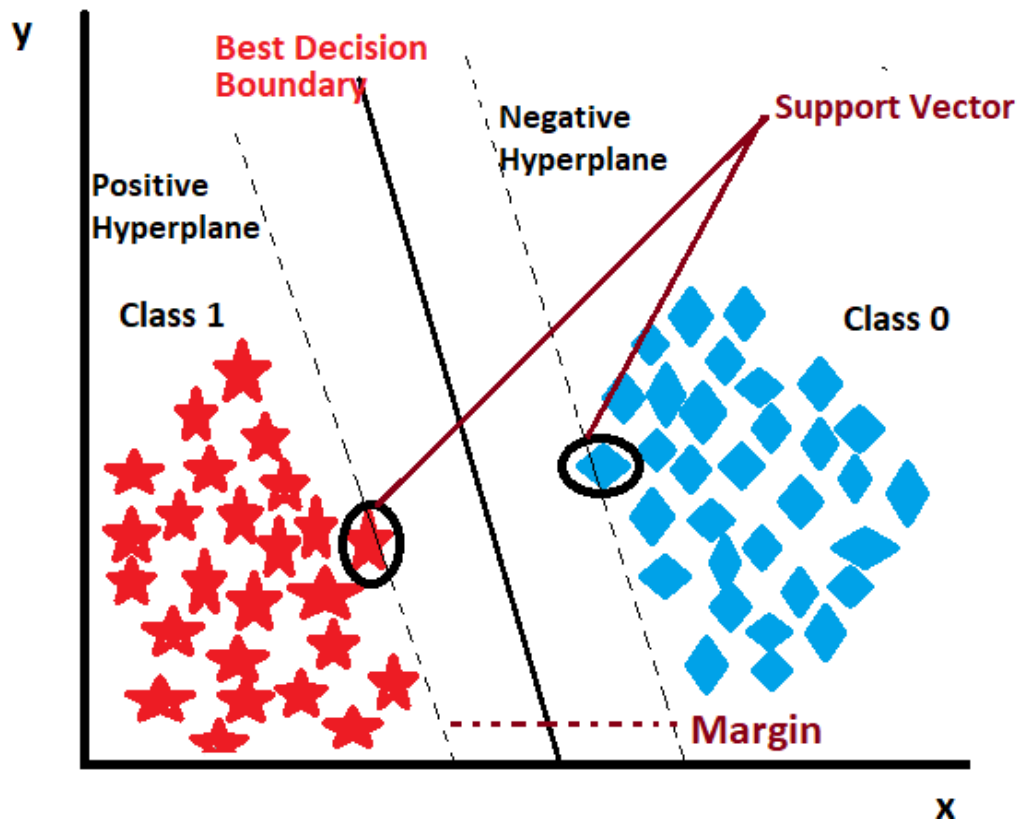
Ranh giới quyết định của mô hình có dạng $\mathbf{w}^\top \mathbf{x} + b = 0$, trong đó $\mathbf{w}$ biểu diễn vector trọng số còn b là hệ số chệch. Với tập dữ liệu huấn luyện $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ và nhãn $y_i \in \{-1, +1\}$, quá trình xác định siêu phẳng được đưa về bài toán tối ưu sau:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i$$

Đây là trường hợp *hard-margin SVM* — chỉ khả thi khi dữ liệu hoàn toàn phân tách tuyến tính. Trên thực tế, phần lớn các tập dữ liệu đều chứa nhiễu và có hiện tượng chồng lấn giữa hai lớp; do đó SVM thường được mở rộng sang dạng *soft-margin* với biến slack $\xi_i \geq 0$ cho phép một số điểm vi phạm biên:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i$$

Hằng số $C > 0$ giữ vai trò cân bằng giữa việc cực đại hóa margin và mức phạt cho các điểm vi phạm. Những mẫu huấn luyện có $\alpha_i > 0$ trong nghiệm bài toán đối ngẫu được gọi là *support vectors* — chúng là những mẫu quyết định vị trí của siêu phẳng tối ưu. Nhờ cơ chế kernel, SVM có thể ánh xạ dữ liệu sang không gian đặc trưng số chiều cao mà không cần biểu diễn tường minh, qua đó xử lý hiệu quả các bài toán có ranh giới quyết định phi tuyến [9]. Kernel RBF (Radial Basis Function) là lựa chọn phổ biến nhất nhờ khả năng mô hình hóa ranh giới phi tuyến trong không gian đặc trưng nhiều chiều.



Hình 2.4: Minh họa thuật toán nhị phân SVM⁴

2.3.3 Biểu diễn đặc trưng cơ bản

2.3.3.1 TF-IDF (Term Frequency – Inverse Document Frequency)

Trong các bài toán phân loại văn bản, TF-IDF [10] cung cấp một cách mã hóa nội dung thành các vector đặc trưng dựa trên thông tin từ vựng, từ đó có thể được sử dụng làm dữ liệu đầu vào cho các mô hình phân loại. Giá trị TF-IDF được xác định từ cả thông tin cục bộ của từ trong từng tài liệu và mức độ phân bố của từ trên toàn bộ tập dữ liệu. Theo cách này, các từ xuất hiện với tần suất đáng kể trong một tài liệu nhưng ít gặp ở những tài liệu khác sẽ được xem là những đặc trưng có tính phân biệt cao hơn so với các từ phổ biến trên nhiều tài liệu.

Cụ thể, một từ xuất hiện với tần suất cao trong tài liệu đang xét nhưng ít xuất hiện ở các tài liệu khác sẽ nhận được giá trị TF-IDF lớn hơn. Ngược lại, các từ xuất hiện phổ biến trên nhiều tài liệu sẽ có trọng số thấp hơn do khả năng phân biệt giữa các tài liệu không cao. Đặc

⁴nguồn ảnh: https://statusneo.com/wp-content/uploads/2023/08/SVM_3-1.png

điểm này đặc biệt hữu ích khi xử lý các từ thường xuất hiện với tần suất lớn nhưng mang ít thông tin phân loại, chẳng hạn như các stopwords.

Term Frequency (TF) đo số lần xuất hiện của từ t trong tài liệu d . Để tránh tình trạng tài liệu dài có TF lớn một cách giả tạo, TF thường được chuẩn hóa theo tổng số từ trong tài liệu:

$$TF(T, D) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

trong đó $f_{t,d}$ là số lần xuất hiện của từ t trong tài liệu d .

Inverse Document Frequency (IDF) được sử dụng để xác định mức độ phổ biến của một từ xét trên toàn bộ tập văn bản. Những từ xuất hiện ở ít tài liệu sẽ có giá trị IDF cao hơn, trong khi các từ xuất hiện ở nhiều tài liệu sẽ nhận giá trị thấp hơn. Giá trị IDF được tính theo công thức:

$$IDF(T) = \log \left(\frac{N}{df_t} \right)$$

với N là số lượng tài liệu trong tập dữ liệu và df_t là số tài liệu có chứa từ t .

Trọng số TF-IDF cuối cùng là tích của hai thành phần trên:

$$TF-IDF(T, D) = TF(T, D) \times IDF(T)$$

Sau khi tính trọng số cho toàn bộ từ vựng, mỗi tài liệu được biểu diễn bằng một vector có kích thước bằng số từ trong từ điển — thường là vector thưa (sparse) — và đưa vào làm đầu vào cho các mô hình học máy truyền thống như LR hay SVM. Hình 2.5 minh họa cách tính TF-IDF trên một tập tài liệu nhỏ gồm hai văn bản giả định; qua ví dụ có thể thấy những từ chỉ xuất hiện ở một tài liệu có trọng số IDF thấp hơn, nên trọng số TF-IDF nhỏ, trong khi những từ mang tính đặc trưng cho từng tài liệu lại nhận trọng số cao hơn.

TF-IDF Calculation Example							
Words	Count		Term Frequency (TF)		Inverse Document Frequency (IDF)	TF * IDF	
	Document 1	Document 2	Document 1	Document 2		Document 1	Document 2
read	1	1	0.17	0.17	0	0	0
svm	1	0	0.17	0	0.3	0.05	0
algorithm	1	1	0.17	0.17	0	0	0
article	1	1	0.17	0.17	0	0	0
dataaspirant	1	1	0.17	0.17	0	0	0
blog	1	1	0.17	0.17	0	0	0
randomforest	0	1	0	0.17	0.3	0	0.05

Hình 2.5: Ví dụ minh họa cách tính TF-IDF trên hai tài liệu⁵

⁵nguồn ảnh: <https://dataaspirant.com/tf-idf-term-frequency-inverse-document-frequency/>

2.4 Học sâu

2.4.1 Định nghĩa

Học sâu (Deep Learning) có thể được xem là một hướng phát triển của học máy, trong đó mạng nơ-ron được tổ chức thành nhiều tầng nhằm thực hiện quá trình xử lý dữ liệu theo từng cấp độ. Thông tin từ lớp đầu vào được biến đổi liên tục qua các tầng, tạo nên những biểu diễn ngày càng phù hợp với nhiệm vụ cần thực hiện. Một điểm khác biệt so với các phương pháp học máy dựa trên đặc trưng được thiết kế thủ công là mô hình học sâu có thể tự học các biểu diễn cần thiết ngay trong quá trình tối ưu tham số.

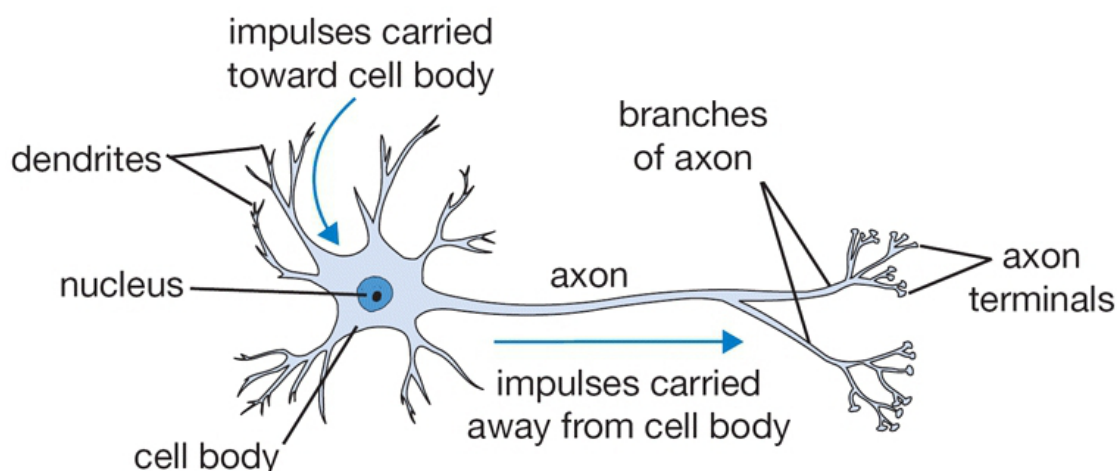
Về nguyên lý, dữ liệu được đưa vào mạng và lần lượt đi qua các tầng theo chiều thuận (Feedforward). Mỗi tầng thực hiện các phép biến đổi trên đầu ra của tầng trước để tạo thành biểu diễn mới. Dựa trên kết quả cuối cùng, hàm mất mát được sử dụng để xác định mức độ sai lệch giữa dự đoán của mô hình và nhãn thực tế. Giá trị sai lệch này sau đó được lan truyền ngược (Backpropagation) qua các tầng của mạng để xác định hướng điều chỉnh các tham số. Quá trình cập nhật được lặp lại trong quá trình huấn luyện nhằm từng bước cải thiện khả năng dự đoán của mô hình.

Việc sử dụng nhiều tầng xử lý giúp học sâu có thể xây dựng các biểu diễn phù hợp với những dạng dữ liệu phức tạp, bao gồm văn bản, hình ảnh và âm thanh. Trong đề tài này, BiLSTM và PhoBERT được lựa chọn cho nhiệm vụ xử lý văn bản tiếng Việt. BiLSTM khai thác thông tin theo cả chiều thuận và chiều ngược của chuỗi, còn PhoBERT sử dụng biểu diễn ngữ cảnh thu được từ quá trình tiền huấn luyện trên ngữ liệu tiếng Việt. Hai mô hình vì vậy cung cấp những hướng tiếp cận khác nhau cho bài toán phát hiện tin giả.

2.4.2 Mạng nơ-ron nhân tạo (Artificial Neural Network)

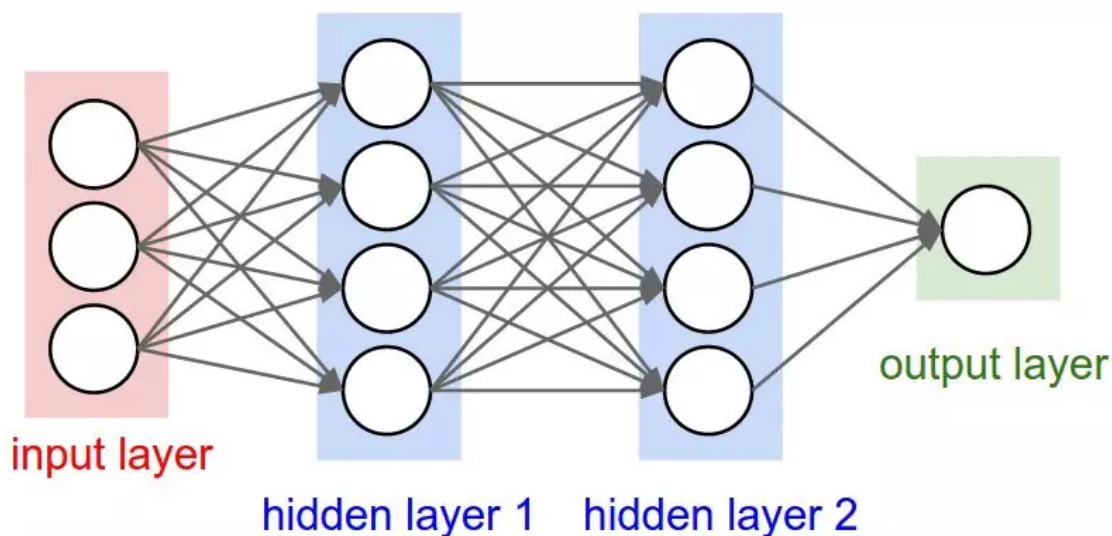
Lấy cảm hứng từ cơ chế hoạt động của hệ thần kinh sinh học, mạng nơ-ron nhân tạo (Artificial Neural Network, ANN) được xây dựng nhằm thực hiện quá trình xử lý thông tin thông qua các đơn vị tính toán liên kết với nhau. Trong hệ thần kinh, mỗi neuron là một đơn vị cơ bản với thân tế bào chứa nhân, các sợi nhánh (Dendrite) đảm nhận tiếp nhận tín hiệu từ neuron khác và sợi trục (Axon) dùng để truyền tín hiệu đi. Tương tự, Trong ANN, quá trình xử lý tại mỗi neuron được thực hiện bằng cách kết hợp các tín hiệu đầu vào với những trọng số tương ứng, sau đó áp dụng một *hàm kích hoạt* (Activation Function) lên kết quả thu được để tạo đầu ra. Việc mô hình hóa các phép xử lý này dưới dạng toán học cho phép mạng học

được những quy luật và mối quan hệ phức tạp từ dữ liệu đầu vào.



Hình 2.6: Minh họa mạng nơ-ron nhân tạo⁶

Mạng nơ-ron nhân tạo được xây dựng từ các neuron nhân tạo và sắp xếp chúng thành nhiều tầng xử lý. Theo quy ước, dữ liệu được truyền từ lớp đầu vào, qua các lớp ẩn (Hidden Layer), đến lớp đầu ra. Số lớp của mạng thường được tính bằng tổng của lớp đầu vào và các lớp ẩn, không tính lớp đầu ra. Các neuron ở hai lớp liên kế được kết nối đầy đủ để truyền thông tin, trong khi neuron trong cùng một lớp không có kết nối trực tiếp.



Hình 2.7: Minh họa kiến trúc nhiều lớp của mạng nơ-ron nhân tạo⁷

⁶ nguồn ảnh: <http://www.wpclipart.com/medical/anatomy/cells/neuron/neuron.png.html>

⁷ nguồn ảnh: [www.researchgate.net/figure/A-general-model-of-a-deep-neural...](http://www.researchgate.net/figure/A-general-model-of-a-deep-neural-network)

Quá trình huấn luyện ANN diễn ra theo vòng lặp: dữ liệu được truyền thuật qua các tầng để tạo đầu ra; sai số giữa đầu ra và nhãn thực tế được hàm mất mát lượng hóa; thuật toán lan truyền ngược tính gradient của sai số theo từng tham số; và cuối cùng các tham số được điều chỉnh theo hướng giảm sai số. Chu trình này lặp lại cho đến khi mô hình đạt trạng thái hội tụ.

Kiến trúc của ANN có thể thay đổi tùy theo yêu cầu của từng bài toán và những đặc điểm riêng của dữ liệu đầu vào. Với dữ liệu văn bản — vốn có tính tuần tự — mạng nơ-ron hồi tiếp (RNN) và các kiến trúc dựa trên Transformer là những lựa chọn phổ biến nhất.

2.4.2.1 Một số thành phần cần chú ý

Trong ANN, hàm kích hoạt giữ vai trò quyết định tính phi tuyến của mô hình. Sau khi tín hiệu đầu vào được kết hợp với trọng số và hệ số chệch, hàm kích hoạt biến đổi giá trị thu được thành đầu ra của neuron. Nếu thiếu bước phi tuyến này, toàn bộ mạng chỉ là một phép biểu diễn tuyến tính đơn thuần, không đủ khả năng mô tả các quan hệ phức tạp trong dữ liệu thực tế.

Các hàm kích hoạt thường gặp gồm Sigmoid, Tanh và ReLU. Trong bài toán phân loại nhị phân, Sigmoid được ưu tiên ở tầng cuối nhờ đưa đầu ra về khoảng $(0, 1)$ — thuận tiện cho việc diễn giải dưới dạng xác suất. Với bài toán đa lớp, hàm Softmax được dùng để chuyển vector đầu ra thành phân phối xác suất trên toàn bộ các lớp.

Hàm mất mát (Loss Function) phản ánh mức độ sai lệch của kết quả dự đoán so với nhãn thực tế. Trong phân loại văn bản, hàm mất mát phổ biến nhất là cross-entropy — loại hàm phản ánh tốt sự khác biệt giữa phân phối xác suất dự đoán và phân phối mục tiêu. Giá trị hàm mất mát càng nhỏ thể hiện dự đoán càng gần nhãn đúng.

Tối ưu mạng nơ-ron chính là quá trình điều chỉnh dần các tham số nhằm giảm giá trị hàm mất mát. Ở mỗi vòng lặp, dữ liệu được truyền thuật qua mạng để tính đầu ra, sau đó sai số được so sánh với mục tiêu thông qua hàm mất mát. Thuật toán lan truyền ngược tính gradient của hàm mất mát theo từng tham số, rồi phương pháp tối ưu (thường là các biến thể của Gradient Descent) cập nhật trọng số và hệ số chệch theo hướng làm giảm sai số.

Với mạng nhiều tầng, quá trình tối ưu có thể gặp phải hai hiện tượng đáng ngại: *gradient biến mất* (vanishing) và *gradient bùng nổ* (exploding). Cả hai đều khiến việc cập nhật tham số trở nên kém hiệu quả và ảnh hưởng tiêu cực đến khả năng hội tụ. Để hạn chế những vấn đề này, nhiều kiến trúc mạng đã được phát triển với các cơ chế xử lý khác nhau. LSTM sử dụng hệ thống các cổng để kiểm soát dòng thông tin và duy trì những phụ thuộc dài hạn, trong khi Transformer dựa trên cơ chế Attention để thiết lập quan hệ trực tiếp giữa các phần tử trong

chuỗi. Nhờ những cơ chế này, các kiến trúc trên có khả năng khai thác thông tin ngữ cảnh hiệu quả trong các bài toán xử lý ngôn ngữ tự nhiên nói chung và phát hiện tin giả nói riêng.

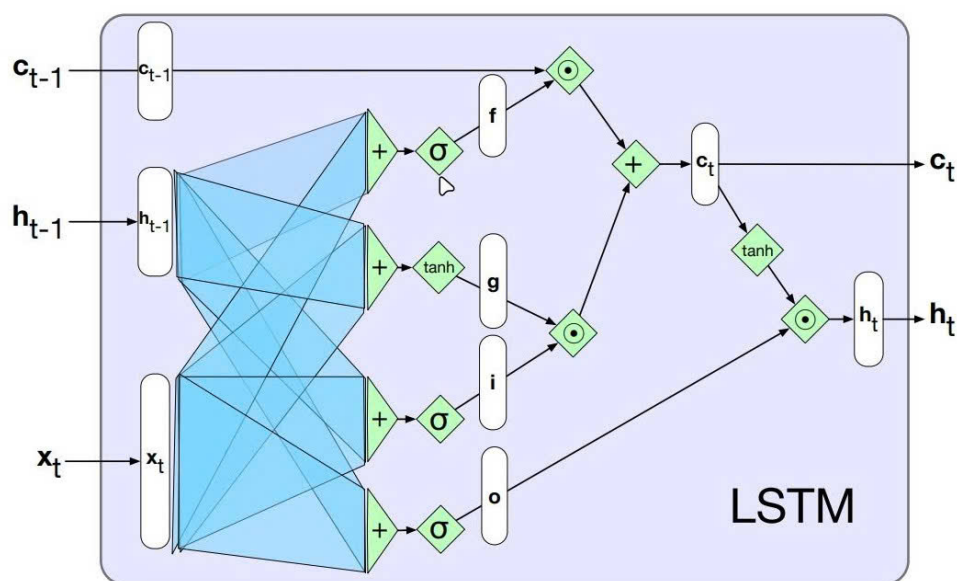
2.4.3 Recurrent Neural Networks (RNN)

RNN (Recurrent Neural Networks) là kiến trúc mạng nơ-ron phù hợp với các bài toán mà dữ liệu được tổ chức theo chuỗi. Trong quá trình xử lý, mô hình sử dụng trạng thái ẩn (hidden state) để lưu giữ thông tin từ các bước trước và truyền lại cho bước tiếp theo, qua đó mô hình hóa sự phụ thuộc giữa các phần tử trong chuỗi. Khác với mạng truyền thẳng xử lý từng đầu vào độc lập, RNN có vòng lặp bên trong giúp thông tin lan truyền từ bước trước sang bước sau.

Khi xử lý các chuỗi dài, RNN truyền thống gặp khó khăn trong việc truyền thông tin qua nhiều bước do ảnh hưởng của hiện tượng vanishing hoặc exploding gradient. Hệ quả là mô hình khó duy trì và học được những mối quan hệ giữa các phần tử nằm cách xa nhau trong chuỗi. LSTM ra đời để giải quyết triệt để hạn chế này.

2.4.4 Long Short-Term Memory (LSTM)

LSTM (Long Short-Term Memory) [11] được thiết kế với mục tiêu xử lý hiệu quả chuỗi có độ dài lớn. Điểm khác biệt cốt lõi nằm ở chỗ LSTM duy trì một trạng thái bộ nhớ riêng (memory cell) và sử dụng ba cổng chức năng — cổng quên, cổng đầu vào, cổng đầu ra — để điều tiết luồng thông tin đi vào, được lưu giữ hay đưa ra khỏi bộ nhớ. Nhờ vậy, mô hình có thể giữ lại những thông tin cần thiết qua nhiều bước trong chuỗi mà không bị mất mát do lan truyền gradient.



Hình 2.8: Minh họa cấu trúc mạng LSTM⁸

LSTM (Long Short-Term Memory) được xây dựng để cải thiện khả năng xử lý các chuỗi có sự phụ thuộc giữa những phần tử ở xa nhau. Trong quá trình đọc dữ liệu, mô hình duy trì một trạng thái bộ nhớ và liên tục cập nhật trạng thái này qua từng bước. Nhờ đặc điểm đó, LSTM được sử dụng trong nhiều bài toán xử lý dữ liệu tuần tự như xử lý ngôn ngữ tự nhiên, dịch máy, nhận dạng giọng nói và dự báo chuỗi thời gian. Khi áp dụng cho phân loại văn bản, mô hình có thể sử dụng thông tin đã được tích lũy ở các bước trước để hình thành biểu diễn ngữ cảnh cho toàn bộ chuỗi, từ đó hỗ trợ nhận diện các đặc điểm liên quan đến nội dung tin giả.

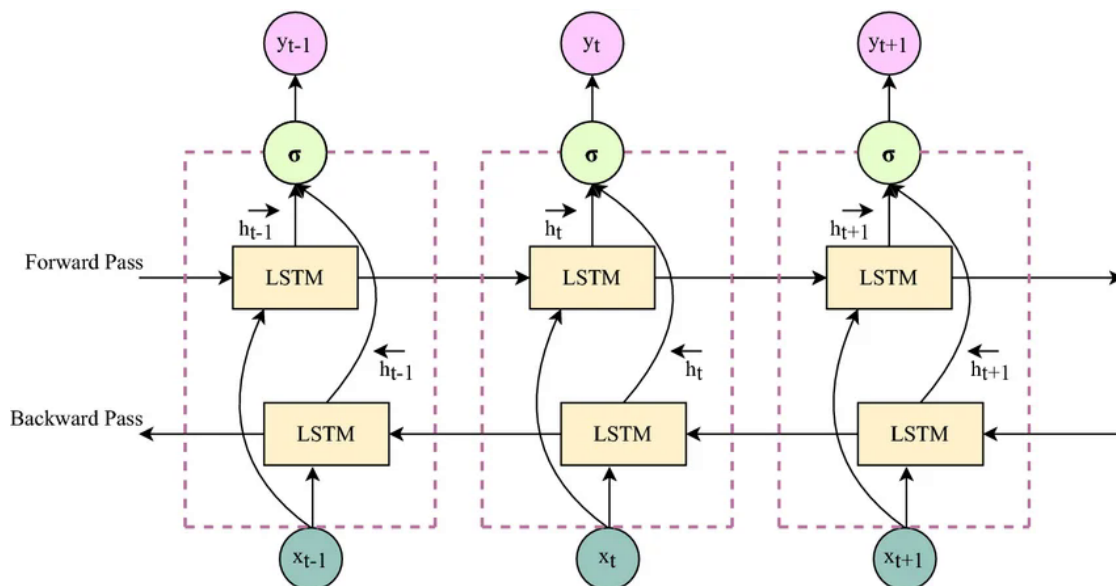
Thông tin được lưu giữ trong LSTM được điều tiết thông qua ba cổng chính. Cổng quên quyết định những thành phần của trạng thái bộ nhớ trước đó cần tiếp tục được giữ lại. Sau đó, cổng vào kiểm soát thông tin mới được bổ sung vào bộ nhớ. Cuối cùng, cổng ra xác định phần thông tin trong trạng thái bộ nhớ được sử dụng để tạo trạng thái ẩn ở bước đang xét. Sự phối hợp giữa các cổng cho phép mô hình kiểm soát quá trình lưu trữ và cập nhật thông tin theo thời gian, đồng thời hạn chế ảnh hưởng của hiện tượng suy giảm gradient khi xử lý các chuỗi có độ dài lớn so với RNN truyền thống.

2.4.5 Mạng LSTM hai chiều (Bidirectional LSTM)

LSTM tiêu chuẩn chỉ khai thác ngữ cảnh theo một chiều — từ trái sang phải. Tuy nhiên, ý nghĩa của một từ trong câu thường chịu ảnh hưởng đồng thời của cả phần ngữ cảnh phía

⁸nguồn ảnh: towardsdatascience.com/an-introduction-to-long-short-term-me...

trước lần phía sau. Để tận dụng triệt để thông tin ngữ cảnh, kiến trúc *Bidirectional LSTM* (BiLSTM) sử dụng hai nhánh xử lý độc lập: một nhánh đọc chuỗi theo chiều thuận (từ đầu đến cuối) và nhánh còn lại đọc theo chiều nghịch. Tại mỗi vị trí trong chuỗi, hai trạng thái ẩn từ các hướng xử lý được nối lại (Concatenation) để hình thành một vector biểu diễn duy nhất, chứa thông tin từ cả hai chiều. Trong đồ án, đặc tính này giúp BiLSTM khai thác đầy đủ hơn ngữ cảnh văn bản và phân biệt hai nhóm nhân.



Hình 2.9: Minh họa cấu trúc mạng LSTM hai chiều⁹

2.4.6 Cơ chế chú ý (Attention Mechanism)

Trong các mô hình xử lý chuỗi truyền thống, thông tin của toàn bộ câu thường được mã hóa vào một vector duy nhất trước khi thực hiện bước tiếp theo. Cơ chế chú ý (Attention Mechanism) [12] được đề xuất nhằm hạn chế hạn chế này bằng cách cho phép mô hình lựa chọn và khai thác những phần thông tin phù hợp trong quá trình xử lý. Ý tưởng cốt lõi của cơ chế chú ý là cho phép mô hình tự động học trọng số ưu tiên cho từng bước thời gian, từ đó tập trung tính toán vào những thành phần thực sự có giá trị cho nhiệm vụ dự đoán.

Giả sử $\{h_1, h_2, \dots, h_T\}$ là các trạng thái ẩn thu được từ mô hình LSTM hoặc BiLSTM. Tại bước t , một điểm số e_t đo mức độ liên quan của trạng thái h_t với đầu ra cuối cùng:

$$e_t = \mathbf{v}^\top \tanh(\mathbf{W}_a \overleftrightarrow{\mathbf{h}}_t) \quad (2.1)$$

⁹nguồn ảnh: <https://www.geeksforgeeks.org/nlp/bidirectional-lstm-in-nlp/>

Hàm softmax được áp dụng lên các điểm số thu được để xác định trọng số chú ý tương ứng α_t :

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} \quad (2.2)$$

Cuối cùng, vector ngữ cảnh $\mathbf{c}$ là tổng có trọng số của toàn bộ trạng thái ẩn:

$$\mathbf{c} = \sum_{t=1}^T \alpha_t \overleftrightarrow{\mathbf{h}}_t \quad (2.3)$$

Ở đầu ra của cơ chế chú ý, mỗi vị trí trong chuỗi được gán một giá trị α_t thể hiện vai trò của vị trí đó đối với biểu diễn chung. Các vị trí chứa thông tin hữu ích sẽ có ảnh hưởng lớn hơn đến kết quả tổng hợp, trong khi những vị trí ít mang tính phân biệt sẽ đóng góp ở mức thấp hơn. Cách phân bổ trọng số này giúp mô hình lựa chọn thông tin cần thiết thay vì phân bổ mức độ xử lý như nhau cho toàn bộ chuỗi.

Đối với văn bản, cơ chế chú ý có thể khai thác những dấu hiệu liên quan đến nội dung và cách thể hiện, chẳng hạn đặc điểm diễn đạt, các chi tiết bất thường hoặc những nội dung gây tranh cãi. Việc phân bổ mức độ quan tâm khác nhau cho từng thành phần giúp mô hình khai thác các tín hiệu có giá trị phân loại, từ đó hỗ trợ quá trình nhận diện văn bản thuộc hai nhóm *thật* và *giả*.

Một trường hợp đặc biệt của cơ chế chú ý là *tự chú ý* (self-attention) — thành phần trung tâm của kiến trúc Transformer, sẽ được trình bày cụ thể tại Mục 2.6. Với self-attention, biểu diễn của mỗi phần tử được tính từ mối tương quan với toàn bộ phần tử còn lại trong chuỗi, thay vì chỉ dựa vào các vị trí lân cận. Nhờ đó, biểu diễn thu được có thể phản ánh mối quan hệ giữa các thành phần trên toàn bộ chuỗi.

2.5 Biểu diễn phân tán và mã hóa con (Distributed & Subword Representations)

Các phương pháp biểu diễn truyền thống như bag-of-words hay TF-IDF (trình bày ở Mục 2.3.3.1) tuy đơn giản và hiệu quả với mô hình tuyến tính, vẫn tồn tại hai hạn chế lớn. Thứ nhất là *curse of dimensionality*: số chiều vector tăng theo kích thước từ vựng, dẫn đến vector thưa có hàng chục nghìn chiều — vừa tốn bộ nhớ vừa khó xử lý. Thứ hai là *orthogonality*: hai từ khác nhau bất kỳ luôn có vector trực giao, không phản ánh được mối quan hệ ngữ nghĩa

giữa chúng. Để khắc phục, các phương pháp biểu diễn phân tán (Distributed Representations) đã ra đời: mỗi từ được ánh xạ vào một không gian vector dày đặc có số chiều cố định (thường từ 50 đến 300), trong đó những từ có ngữ nghĩa tương tự nằm gần nhau trong không gian.

2.5.1 Word2Vec và GloVe

Word2Vec [2] học biểu diễn vector cho mỗi từ bằng cách khai thác mối quan hệ giữa từ đó với các từ lân cận trong ngữ liệu. Hai kiểu huấn luyện phổ biến gồm *continuous bag-of-words* (CBOW) và *skip-gram*. Ở chế độ CBOW, các từ ngữ cảnh được dùng để suy ra từ mục tiêu; với skip-gram, quá trình diễn ra ngược lại — dùng từ mục tiêu để dự đoán các từ xung quanh. Sau huấn luyện, ta thu được một ma trận $\mathbf{E} \in \mathbb{R}^{V \times d}$, với V là kích thước từ và d là số chiều của vector.

Một đặc tính thú vị của không gian Word2Vec là khoảng cách và hướng giữa các vector có thể phản ánh một số mối quan hệ ngữ nghĩa. Phép toán điển hình:

$$\text{vua} - \text{nam} + \text{nữ} \approx \text{hoàng hậu}$$

Một điểm hạn chế của Word2Vec nằm ở cách biểu diễn cố định: mỗi từ chỉ được gán với một vector duy nhất, bất kể nó xuất hiện trong ngữ cảnh nào. Vì vậy, khi cùng một từ mang những nghĩa khác nhau tùy theo ngữ cảnh, mô hình không thể tạo ra các biểu diễn riêng biệt cho từng nghĩa. Đây là vấn đề thường được gọi là *word-sense ambiguity*.

GloVe [13] tiếp cận bài toán biểu diễn từ theo hướng khai thác thông tin đồng xuất hiện trên toàn tập văn bản. Khác với Word2Vec chỉ học từ quan hệ ngữ cảnh cục bộ, GloVe sử dụng ma trận đồng xuất hiện (co-occurrence matrix) để đưa thống kê toàn cục vào quá trình tối ưu. Mối quan hệ giữa từ mục tiêu i và từ ngữ cảnh j được mô hình hóa thông qua:

$$\log \hat{P}(i | j) = \mathbf{u}_i^\top \mathbf{v}_j + b_i + b_j$$

trong đó $\hat{P}(i | j)$ là xác suất quan sát từ i trong ngữ cảnh của từ j . Việc đưa thông tin đồng xuất hiện ở quy mô toàn tập giúp GloVe xây dựng không gian biểu diễn dựa trên quan hệ thống kê giữa các từ, phục vụ tốt cho các tác vụ yêu cầu đo lường tương đồng ngữ nghĩa.

2.5.2 FastText và thông tin subword

Cả Word2Vec lẫn GloVe đều có chung một nhược điểm: *từ hiếm* và *từ ngoài từ vựng* (OOV — Out-Of-Vocabulary) không có embedding vì không xuất hiện trong tập huấn luyện. Đối với tiếng Việt — ngôn ngữ có tính tổng hợp và đa dạng biến thể ngữ pháp — vấn đề này càng nghiêm trọng, vì cùng một gốc từ có thể sinh ra hàng chục hình thái khác nhau.

FastText [3] khắc phục điểm yếu này bằng cách biểu diễn mỗi từ như một túi các n-gram ký tự (character n-grams). Với từ w và tập các n-gram $\mathcal{G}_w \subseteq \{3, 4, \dots, 6\}$, embedding của w được tính:

$$\mathbf{e}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

trong đó $\mathbf{z}_g$ là vector embedding của n-gram g . Nhờ vậy, ngay cả khi một từ chưa từng xuất hiện trong tập huấn luyện, mô hình vẫn tính được embedding từ các n-gram con, giúp tăng độ phủ từ vựng và giảm hiện tượng OOV cho những từ ghép ít gặp.

2.5.3 Mã hóa con (Subword Tokenization)

Trong khi FastText khai thác n-gram ký tự ở cấp độ từ, các mô hình Transformer hiện đại như BERT [14] và PhoBERT [15] lại sử dụng BPE (Byte Pair Encoding) [16] — thuật toán mã hóa con phổ biến nhất hiện nay. Ý tưởng chính của BPE gồm các bước:

1. Khởi tạo từ vựng bằng tất cả các ký tự xuất hiện trong kho ngữ liệu.
2. Thực hiện K vòng lặp, trong mỗi vòng xác định cặp ký hiệu liên kề có tần suất xuất hiện cao nhất, sau đó gộp chúng thành một ký hiệu mới và cập nhật vào từ vựng.
3. Sau K lần hợp nhất, từ vựng gồm $K + |\Sigma|$ token, trong đó $|\Sigma|$ là số ký tự gốc.

Kết quả thu được là một từ vựng cân bằng giữa kích thước và tính bao phủ: những từ phổ biến được giữ nguyên vẹn (1 token), còn các từ hiếm được tách thành vài đơn vị con (subword) có nghĩa. Ví dụ, từ tiếng Việt **khó** có thể được biểu diễn bằng **kh** + **ó**. Cách tiếp cận này giải quyết triệt để bài toán OOV và giảm kích thước từ vựng từ hàng trăm nghìn xuống chỉ vài chục nghìn subword.

PhoBERT sử dụng BPE được huấn luyện trên kho ngữ liệu tiếng Việt khoảng 20GB [15], cho từ vựng khoảng 64,000 subword. Kết hợp với RDRSegmenter (Mục 2.2.2), PhoBERT có thể biểu diễn hiệu quả cả những từ ghép và từ Hán-Việt dài mà không cần mở rộng từ vựng.

2.6 Mô hình Transformer và phương thức Self-Attention

Transformer được giới thiệu dựa trên cơ chế tự chú ý (Self-Attention) [17], cho phép mô hình xác định mức độ liên quan giữa các thành phần trong cùng một chuỗi. Khác với RNN và LSTM, vốn phải duyệt dữ liệu theo thứ tự từng bước, Transformer có thể xử lý nhiều vị trí trong chuỗi đồng thời. Đặc điểm này cho phép kiến trúc khai thác tốt khả năng tính toán song song của GPU và hỗ trợ rút ngắn thời gian huấn luyện [18]. Đồng thời, việc xây dựng quan hệ trực tiếp giữa các vị trí trong chuỗi giúp mô hình khai thác được những phụ thuộc giữa các thành phần nằm cách xa nhau.

2.6.1 Cơ chế Scaled Dot-Product Attention

Scaled Dot-Product Attention là phép tính attention cơ bản được sử dụng trong Transformer. Đối với chuỗi đầu vào, thông tin biểu diễn được biến đổi thành ba thành phần $\mathbf{Q}$ (Query), $\mathbf{K}$ (Key) và $\mathbf{V}$ (Value) thông qua các phép chiếu tuyến tính. Từ ba ma trận này, đầu ra của cơ chế attention được xác định bởi:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (2.4)$$

Ở đây, d_k biểu thị số chiều của vector Key. Thành phần $\sqrt{d_k}$ được đưa vào mẫu số nhằm kiểm soát độ lớn của giá trị tích vô hướng trước khi áp dụng softmax. Khi số chiều của vector tăng, phép chia này giúp hạn chế việc các giá trị đầu vào softmax trở nên quá lớn và làm quá trình tối ưu gặp vấn đề [17]. Trong trường hợp self-attention, $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ đều được xây dựng từ cùng một chuỗi biểu diễn. Vì vậy, mỗi vị trí có thể sử dụng thông tin từ toàn bộ các vị trí khác trong chuỗi khi tạo biểu diễn đầu ra.

2.6.2 Multi-Head Attention và Positional Encoding

Một phép attention đơn lẻ có thể chưa đủ để mô hình hóa nhiều dạng quan hệ khác nhau giữa các thành phần của văn bản. Transformer giải quyết vấn đề này bằng Multi-Head Attention, trong đó biểu diễn đầu vào được đưa qua nhiều phép chiếu riêng biệt để hình thành h không gian con. Attention được tính độc lập trên từng không gian, sau đó các kết quả được kết hợp thành biểu diễn cuối cùng:

$$\text{multihead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (2.5)$$

với

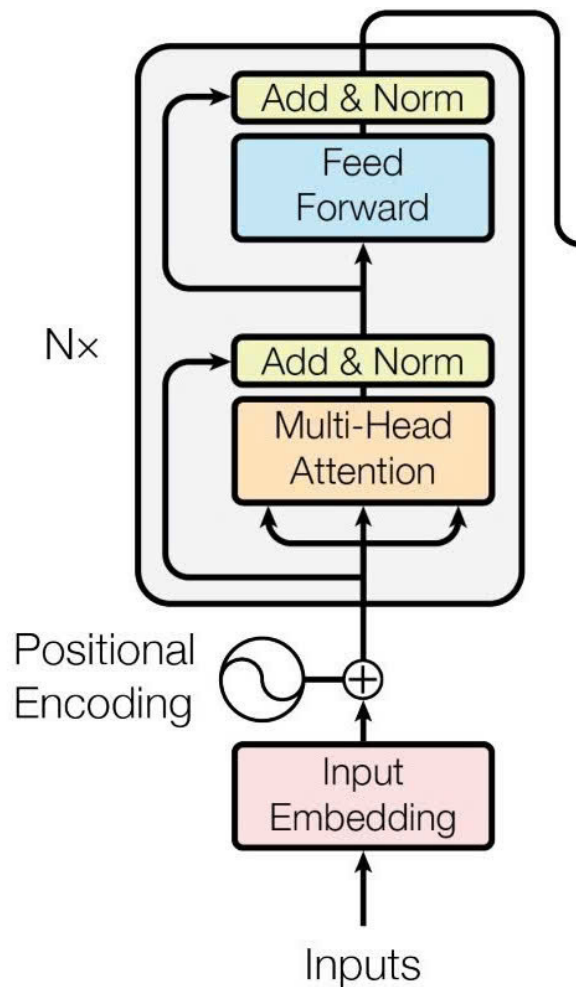
$$\text{head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V).$$

Trong thiết kế này, mỗi head có thể tập trung vào một kiểu quan hệ khác nhau trong dữ liệu. Nếu kích thước biểu diễn đầu vào là d và có h head, mỗi không gian con thường có số chiều $d_k = d/h$. Sau khi hoàn thành các phép tính attention riêng, các biểu diễn thu được được nối lại và biến đổi thông qua ma trận $\mathbf{W}^O$.

Self-Attention không chứa thông tin trực tiếp về thứ tự xuất hiện của các phần tử trong chuỗi. Vì vậy, Transformer cần bổ sung thông tin vị trí để phân biệt các phần tử dựa trên vị trí của chúng. Trong kiến trúc Transformer ban đầu, Vaswani và cộng sự [17] sử dụng các hàm sin và cos để xây dựng biểu diễn vị trí:

$$pe_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad pe_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right). \quad (2.6)$$

Việc bổ sung thông tin vị trí vào biểu diễn đầu vào giúp Transformer nhận biết thứ tự của các phần tử, đồng thời thiết lập được mối quan hệ giữa chúng khi thực hiện tính toán attention.



Hình 2.10: Minh họa kiến trúc Transformer encoder¹⁰

Hình 2.10 minh họa một khối encoder điển hình của Transformer. Quá trình xử lý trong mỗi khối được tổ chức thành hai thành phần chính. Thành phần đầu tiên là Multi-Head Self-Attention, nơi thông tin tại một vị trí được kết hợp với biểu diễn của các vị trí khác trong chuỗi. Cơ chế này giúp mô hình xây dựng biểu diễn dựa trên ngữ cảnh rộng thay vì chỉ sử dụng thông tin ở những vị trí lân cận. Sau phép attention, kết quả được đưa qua kết nối dư và bước chuẩn hóa (Add & Norm) nhằm hỗ trợ quá trình tối ưu.

Thành phần thứ hai là mạng truyền thẳng theo vị trí (Position-wise Feed-Forward Network – FFN). Mạng này được áp dụng riêng trên biểu diễn tại từng vị trí và thực hiện các phép biến đổi phi tuyến. Một dạng FFN thường được sử dụng có thể biểu diễn như sau:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \cdot \text{gelu}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

¹⁰nguồn ảnh: d2l.ai/chapter_attention-mechanisms-and-transformers/transf...

Trong nhiều cấu hình Transformer, số chiều của tầng ẩn trong FFN được đặt ở mức lớn hơn kích thước biểu diễn đầu vào, thường khoảng bốn lần d . Sau khi xử lý qua FFN, biểu diễn tiếp tục được đưa qua một khối Add & Norm khác. Các encoder có thể được xếp chồng thành nhiều tầng để tăng khả năng học biểu diễn của mô hình. Chẳng hạn, BERT-base sử dụng $L = 12$ tầng encoder, trong khi BERT-large sử dụng $L = 24$ tầng [14].

2.6.3 Encoder và Decoder

Trong kiến trúc Transformer nguyên bản, quá trình xử lý được chia thành hai phần là encoder và decoder [17]:

- *Encoder*: tiếp nhận chuỗi đầu vào và xây dựng biểu diễn có chứa thông tin ngữ cảnh. Cơ chế self-attention cho phép mỗi vị trí sử dụng thông tin từ toàn bộ chuỗi. Cấu trúc này phù hợp với các nhiệm vụ yêu cầu hiểu nội dung như phân loại văn bản, gán nhãn token hoặc trích xuất quan hệ.
- *Decoder*: tạo ra chuỗi đầu ra dựa trên cơ chế masked self-attention, trong đó thông tin từ các vị trí tương lai được che lại. Decoder đồng thời có thể sử dụng cross-attention để kết hợp biểu diễn do encoder tạo ra. Cách tổ chức này phù hợp với các nhiệm vụ sinh chuỗi như dịch máy và tóm tắt văn bản.

Các mô hình BERT [14] và PhoBERT [15] thuộc nhóm kiến trúc sử dụng encoder để tạo biểu diễn ngữ cảnh cho văn bản. Vì vậy, trong phạm vi nghiên cứu này, phần kiến trúc Transformer được tập trung vào encoder và các thành phần liên quan đến quá trình xây dựng biểu diễn văn bản.

2.7 PhoBERT và mô hình ngôn ngữ tiền huấn luyện

2.7.1 Tiền huấn luyện (Pre-Training) và tinh chỉnh (Fine-Tuning)

Một trong những đóng góp quan trọng nhất của BERT [14] là thiết lập một framework huấn luyện hai giai đoạn, cho phép tận dụng hiệu quả lượng lớn dữ liệu không nhãn:

1. *Tiền huấn luyện* (Pre-Training): mô hình Transformer encoder được huấn luyện trên kho ngữ liệu lớn với hai nhiệm vụ tự giám sát (self-supervised):

- *Masked Language Modeling* (MLM): một tỷ lệ (thường 15%) token đầu vào bị che ngẫu nhiên, các token bị che được khôi phục dựa trên thông tin ngữ cảnh được cung cấp từ cả phía trước và phía sau trong chuỗi.
- *Next Sentence Prediction* (NSP): mô hình dự đoán xem có hai câu xuất hiện liên kề trong văn bản gốc hay không — hỗ trợ học quan hệ giữa các câu.

Quá trình này rất tốn kém — BERT-base mất khoảng 4 ngày trên 64 GPU TPU [14] — nhưng chỉ cần thực hiện một lần cho mỗi ngôn ngữ.

2. *Tinh chỉnh* (Fine-Tuning): sau khi đã có checkpoint tiền huấn luyện, mô hình chỉ cần bổ sung một lớp phân loại đơn giản, thường được triển khai dưới dạng lớp tuyến tính lên trên encoder và tiếp tục huấn luyện trên dữ liệu có nhãn của tác vụ cụ thể — phát hiện tin giả, phân tích cảm xúc, trả lời câu hỏi, v.v. Vì phần lớn trọng số đã được khởi tạo tốt, fine-tuning thường chỉ cần 3–10 epoch trên dữ liệu tác vụ [19].

Hai giai đoạn này giải quyết được nghịch lý cố hữu của NLP: *dữ liệu không nhãn thì rất nhiều nhưng dữ liệu có nhãn thì rất đắt*. Các mô hình như BERT, PhoBERT và RoBERTa [20] đều tuân theo paradigm này.

2.7.2 PhoBERT cho tiếng Việt

PhoBERT [15] được xây dựng như một mô hình ngôn ngữ tiền huấn luyện dành riêng cho tiếng Việt, sử dụng kiến trúc RoBERTa [20] làm nền tảng. Do đó, mô hình tiếp nhận bốn cải tiến chính của RoBERTa so với BERT nguyên bản, được trình bày như sau:

- Bỏ tác vụ NSP, chỉ giữ MLM;
- Sử dụng tập dữ liệu huấn luyện có quy mô lớn hơn đáng kể, đạt 160GB so với 13GB của BERT.;
- Sử dụng dynamic masking thay vì static masking;
- Sinh chuỗi dài hơn (lên tới 512 token).

Nguồn dữ liệu đầu vào của PhoBERT bao gồm khoảng 20GB ngữ liệu tiếng Việt, trong đó dữ liệu được lấy từ Wikipedia và các bài viết trên báo điện tử. Mô hình có hai biến thể:

- **PhoBERT-base**: 12 lớp encoder, $d = 768$, 12 head attention, $\sim 135\text{M}$ tham số.
- **PhoBERT-large**: 24 lớp encoder, $d = 1024$, 16 head attention, $\sim 370\text{M}$ tham số.

Cả hai biến thể đều sử dụng RDRSegmenter (Mục 2.2.2) để phân đoạn từ trước khi áp dụng BPE — đảm bảo từ vựng và embedding phù hợp với đặc thù tiếng Việt. Trong các nghiên cứu thực nghiệm, PhoBERT-base thường được lựa chọn nhờ đảm bảo sự hài hòa giữa chất lượng mô hình và mức độ sử dụng tài nguyên tính toán.

2.7.3 Một số công trình liên quan về phát hiện tin giả tiếng Việt

Trước giai đoạn PhoBERT được giới thiệu, các hướng nghiên cứu về phát hiện tin giả tiếng Việt chủ yếu khai thác đặc trưng do con người xây dựng và kết hợp chúng với các thuật toán học máy truyền thống [21]. Sự xuất hiện của PhoBERT mở ra hướng tiếp cận dựa trên mô hình ngôn ngữ tiền huấn luyện, trong đó quá trình fine-tuning trên dữ liệu của bài toán đã được áp dụng và cho thấy kết quả khả quan đối với nhiệm vụ phát hiện tin giả. Chẳng hạn, Luu và cộng sự [22] phát triển hệ thống *ViFakeDect* sử dụng *PhoBERT-base* và đạt F1-macro khoảng 88% trên tập dữ liệu được thu thập thủ công. Một hướng khác được Tran và cộng sự [23] đề xuất là kết hợp PhoBERT với các đặc trưng phong cách ngôn ngữ (Stylometric Features), qua đó cho thấy những đặc trưng này có thể bổ sung thêm thông tin hữu ích cho biểu diễn của PhoBERT. Xét trên phạm vi rộng hơn, các nghiên cứu tổng quan quốc tế [5–7] cho thấy các mô hình dựa trên Transformer ngày càng được sử dụng phổ biến trong các hệ thống phát hiện tin giả.

2.8 Đánh giá mô hình và phân tích hiệu chuẩn

2.8.1 Các độ đo hiệu năng phân loại

Trong phân loại nhị phân, kết quả dự đoán của mô hình có thể được quy về bốn trường hợp dựa trên nhãn thực tế và nhãn dự đoán. Với quy ước lớp *thật* là Negative và lớp *giả* là Positive, bốn thành phần của ma trận nhầm lẫn gồm *TP* (tin giả được nhận diện đúng), *TN* (tin thật được nhận diện đúng), *FP* (tin thật bị phân loại thành tin giả) và *FN* (tin giả bị phân loại thành tin thật). Từ các giá trị này, hiệu năng của bộ phân loại được đánh giá thông qua các đại lượng:

$$\begin{aligned}
\text{accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\
\text{precision} &= \frac{TP}{TP + FP} \\
\text{recall} &= \frac{TP}{TP + FN} \\
F_1 &= 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}
\end{aligned}$$

Công thức F_1 ở trên được tính cho lớp *Positive*, tương ứng với lớp *giả* trong nghiên cứu. Khi phân bố giữa hai lớp không hoàn toàn cân bằng, có thể tính F_1 riêng cho từng lớp rồi lấy trung bình để thu được F_1 -macro:

$$F_1\text{-macro} = \frac{F_1^{(0)} + F_1^{(1)}}{2},$$

trong đó $F_1^{(0)}$ biểu thị kết quả của lớp *Negative* (lớp *thật*) và $F_1^{(1)}$ tương ứng với lớp *Positive* (lớp *giả*). Do hai lớp đóng góp như nhau vào giá trị trung bình, độ đo này cho thấy hiệu năng của mô hình trên từng lớp mà không phụ thuộc trực tiếp vào số lượng mẫu của mỗi lớp.

Đối với dữ liệu phát hiện tin giả trong nghiên cứu, tỷ lệ giữa hai nhãn có sự chênh lệch, với khoảng 56% mẫu thuộc lớp thật và 44% thuộc lớp giả. Trong trường hợp này, Accuracy có thể chưa phản ánh đầy đủ chất lượng phân loại, bởi mô hình vẫn có khả năng đạt giá trị tương đối cao nếu thiên về lớp chiếm ưu thế. Vì vậy, F_1 -macro được sử dụng làm thước đo chính, nhằm xem xét đồng thời khả năng nhận diện cả hai lớp. Việc sử dụng trung bình không trọng số giữa hai giá trị F_1 cũng giúp hạn chế ảnh hưởng của sự mất cân bằng nhãn khi đánh giá.

Khả năng phân biệt giữa hai lớp ở nhiều mức ngưỡng khác nhau được đánh giá thông qua diện tích dưới đường cong ROC (*ROC-AUC), bên cạnh các độ đo dựa trên nhãn dự đoán.

2.8.2 Kiểm định thống kê

Các chỉ số đánh giá chỉ phản ánh sự chênh lệch về kết quả giữa các mô hình và chưa thể hiện liệu khác biệt đó có mang ý nghĩa thống kê hay không. Vì vậy, nghiên cứu áp dụng các phương pháp kiểm định thống kê để đánh giá độ tin cậy của kết quả và xác định ý nghĩa của những khác biệt được quan sát.

- *Kiểm định McNemar* [24]: được thực hiện khi cần đối chiếu hai mô hình cùng dự đoán trên một tập kiểm tra gồm 2.094 mẫu. Kiểm định tập trung vào những trường hợp hai

mô hình đưa ra kết quả khác nhau và sử dụng phân phối χ^2 với 1 bậc tự do để đánh giá sự khác biệt. Phương pháp này không yêu cầu thực hiện k-fold nên phù hợp khi chi phí huấn luyện mô hình cao.

- *Khoảng tin cậy bootstrap* [1]: từ tập kiểm tra, các mẫu được lấy lại có hoàn lại để tạo ra nhiều tập bootstrap có cùng kích thước n . Quy trình được lặp trong B lần, với $B = 1000$ được sử dụng làm giá trị tham khảo, và F_1 được tính trên từng lần lấy mẫu. Các giá trị thu được sau đó được dùng để xây dựng khoảng tin cậy 95% theo phương pháp percentile, qua đó cung cấp thông tin về mức biến động của ước lượng.
- *Kiểm tra chéo* (Cross-Validation) [25]: dữ liệu được phân thành k phần và quá trình huấn luyện được lặp lại sao cho mỗi phần lần lượt được sử dụng để đánh giá, trong khi $k - 1$ phần còn lại phục vụ huấn luyện. Khi tiến hành nhiều phép so sánh giữa các mô hình, kiểm định Holm [26] được dùng để điều chỉnh p-value và hạn chế ảnh hưởng của vấn đề đa so sánh.

2.8.3 Hiệu chuẩn xác suất (Probability Calibration)

Không chỉ dựa vào kết quả phân loại, mô hình còn được xem xét qua độ tin cậy của các xác suất dự đoán. Khả năng hiệu chuẩn (*well-calibrated*) được thể hiện khi xác suất do mô hình đưa ra tương đối phù hợp với tần suất thực tế của các kết quả tương ứng. Ví dụ, nếu một nhóm mẫu được dự đoán với xác suất $*p = 0,8*$, thì khoảng 80% trong số đó được kỳ vọng thuộc đúng lớp tương ứng [27].

Trong bài toán phát hiện tin giả, xác suất dự đoán đặc biệt hữu ích khi kết quả phân loại được dùng như thông tin hỗ trợ cho quá trình ra quyết định (decision support). Khi đó, kết quả không chỉ cần cho biết bài viết thuộc lớp tin thật hay tin giả mà còn cần cung cấp mức độ chắc chắn tương ứng của mô hình. Vì vậy, việc đánh giá hiệu chuẩn có thể được thực hiện thông qua một số thước đo phổ biến như:

- *Expected Calibration Error* (ECE) [28]: miền xác suất $[0, 1]$ được chia thành B khoảng, sau đó kết quả dự đoán trong từng bin được dùng để đối chiếu độ chính xác thực tế với mức độ tự tin trung bình. ECE được tính bằng trung bình có trọng số của sai lệch giữa hai đại lượng này:

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{n} |\text{acc}(B_b) - \text{conf}(B_b)|$$

- *Maximum Calibration Error* (MCE): thay vì tổng hợp sai lệch trên toàn bộ các bin, MCE lấy giá trị lớn nhất của $|\text{acc}(B_b) - \text{conf}(B_b)|$. Do đó, độ đo này cho biết mức sai lệch hiệu chuẩn nghiêm trọng nhất xuất hiện trong các khoảng xác suất.
- *Brier Score*: đánh giá trực tiếp mức chênh lệch giữa xác suất dự đoán và nhãn thực thông qua trung bình bình phương sai số:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

trong đó $p_i \in [0, 1]$ là xác suất dự đoán và $y_i \in \{0, 1\}$ là nhãn thực. Giá trị Brier Score càng thấp cho thấy xác suất do mô hình cung cấp càng gần với kết quả thực tế.

Các thước đo trên được kết hợp với những phương pháp hiệu chỉnh sau huấn luyện như Platt scaling, Temperature scaling và Isotonic regression [27]. Những kỹ thuật này sẽ được áp dụng trong phần thực nghiệm tại Mục 4.7, nhằm đánh giá mức độ đáng tin cậy của xác suất đầu ra từ bốn mô hình được so sánh.

2.9 Kết luận chương

Những nội dung trong chương đã cung cấp cơ sở để tiếp cận bài toán phát hiện tin giả từ các phương pháp học máy truyền thống đến các mô hình học sâu và ngôn ngữ tiền huấn luyện. Các kỹ thuật biểu diễn văn bản, kiến trúc mô hình và phương pháp đánh giá được trình bày là nền tảng cho quá trình thực nghiệm và đối chiếu kết quả. Trên cơ sở đó, chương tiếp theo sẽ trình bày dữ liệu nghiên cứu, các bước tiền xử lý và cách tổ chức dữ liệu phục vụ huấn luyện mô hình.

BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1 Thu thập và mô tả bộ dữ liệu

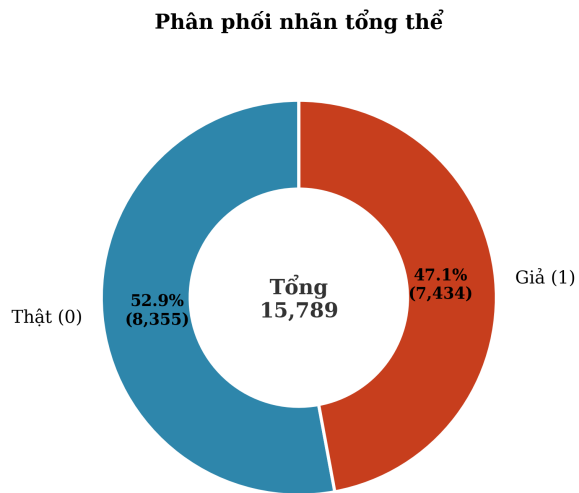
3.1.1 Nguồn dữ liệu

Bộ dữ liệu được sử dụng được tổng hợp từ hai nguồn chính. Nguồn thứ nhất là các bài đăng trên mạng xã hội Facebook — nơi phản ánh tương đối sát thực tế nội dung tin giả được lan truyền trong cộng đồng. Nguồn thứ hai là tập các bài báo cũ được thu thập trong giai đoạn dịch COVID-19 — thời điểm lượng thông tin sai sự thật trên mạng tăng đột biến. Mỗi bản ghi trong tập dữ liệu bao gồm tối thiểu hai trường thông tin:

- **text**: toàn bộ nội dung văn bản của bài viết.
- **label**: nhãn phân loại, với 0 là *thật* và 1 là cho *giả*.

Ngay sau khi thu thập, dữ liệu được đưa vào bước phân đoạn từ (word segmentation) để phù hợp với các khâu xử lý về sau. Tổng số bản ghi thô ban đầu là 15.789 mẫu.

3.1.2 Phân phối nhân



Hình 3.1: Phân phối nhân theo từng tập dữ liệu và tổng quan toàn bộ 15.789 mẫu

Hình 3.1 phản ánh tỷ lệ giữa hai lớp trong toàn bộ tập dữ liệu. Có thể nhận thấy tỷ lệ giữa hai lớp tương đối cân đối — đặc điểm này có ý nghĩa quan trọng vì nó giúp hạn chế hiện tượng mô hình bị thiên lệch (bias) về phía lớp đa số trong suốt quá trình huấn luyện lẫn đánh giá. Trong thực tế, các bộ dữ liệu tin giả thường có xu hướng mất cân bằng, nên sự cân đối của tập dữ liệu nghiên cứu là một lợi thế giúp kết quả thực nghiệm phản ánh trung thực hiệu quả của từng phương pháp phân loại.

3.2 Tiền xử lý văn bản

3.2.1 Làm sạch và lọc dữ liệu

Trước khi tiến hành chia tập huấn luyện — kiểm định — kiểm tra, dữ liệu cần trải qua khâu làm sạch nhằm loại bỏ những bản ghi không có giá trị hoặc có nguy cơ gây rò rỉ thông tin. Quy trình làm sạch gồm bốn bước chính:

1. **Loại bỏ văn bản trùng lặp:** các bản ghi có trường `text` giống hệt nhau được gộp lại, chỉ giữ lại một bản duy nhất. Bước này đặc biệt quan trọng vì trùng lặp giữa các tập dữ liệu có thể làm sai lệch kết quả đánh giá.
2. **Loại bỏ văn bản rỗng hoặc giá trị thiếu:** những bản ghi có `text` là NaN hoặc chuỗi rỗng sau khi loại bỏ khoảng trắng cũng được loại khỏi tập dữ liệu.
3. **Loại bỏ văn bản quá ngắn:** những bài viết có dưới 10 từ bị loại vì không cung cấp đủ thông tin ngữ nghĩa cho mô hình phân loại. Ngưỡng này được cấu hình thông qua tham số `min_word_count = 10`.
4. **Đặt lại chỉ mục:** sau khi hoàn tất ba bước trên, dữ liệu được đánh lại chỉ mục nhằm đảm bảo tính nhất quán cho các bước xử lý phía sau.

Tóm tắt kết quả làm sạch:

- Số mẫu ban đầu: 15.789
- Số mẫu bị loại bỏ: 1.831
- Số mẫu còn lại: 13.958

Các bước làm sạch nói trên giúp giảm đáng kể lượng nhiễu trong tập dữ liệu và bảo đảm rằng quá trình huấn luyện cũng như đánh giá mô hình không bị ảnh hưởng bởi những bản ghi không hợp lệ.

3.2.2 Phân đoạn từ bằng VnCoreNLP

Khác với nhiều ngôn ngữ mà khoảng trắng phân tách rõ ràng giữa các từ, tiếng Việt có sự chênh lệch giữa đơn vị âm tiết và đơn vị từ. Chẳng hạn, “Việt Nam” là một từ nhưng được viết thành hai âm tiết tách biệt; nếu lấy từng âm tiết làm đơn vị xử lý thì cấu trúc và ý nghĩa thực sự của cụm từ sẽ bị bỏ sót. Vì lý do này, khâu phân đoạn từ cần được thực hiện trước các bước biểu diễn số nhằm xác định chính xác đơn vị từ phù hợp với đặc thù tiếng Việt.

Trong nghiên cứu này, phân đoạn từ được thực hiện bằng thư viện `py_vncorenlp` với bộ tách từ `RDRSegmenter` của VnCoreNLP [4, 29]. Lựa chọn này đảm bảo dữ liệu đầu vào được xử lý theo đúng cách mà PhoBERT — mô hình ngôn ngữ tiền huấn luyện — đã sử dụng trong giai đoạn tiền huấn luyện. Nhờ vậy, khi đưa văn bản vào PhoBERT ở các bước sau, sự khác biệt trong cách xác định đơn vị từ được hạn chế tối đa, qua đó cải thiện tính tương thích và hỗ trợ khả năng tổng quát hóa.

Các bước triển khai cụ thể gồm:

- Tự động tải và lưu trữ mô hình `vncorenlp` tại thư mục cục bộ (`.vncorenlp`) nếu chưa có sẵn.
- Thực hiện phân đoạn trên cả hai trường `title` và `text`.
- Chuẩn hóa văn bản trước khi phân đoạn, bao gồm:
 - Loại bỏ dấu gạch dưới phát sinh từ những lần phân đoạn trước (nếu có).
 - Chuẩn hóa khoảng trắng nhằm đảm bảo định dạng đồng nhất.

Sau khi phân đoạn, những âm tiết cùng tạo thành một từ được ghép với nhau bằng dấu gạch dưới. Ví dụ, với câu “em bé đang ăn bánh ngọt”, kết quả xử lý minh họa như sau:

trước: “em bé đang ăn bánh ngọt”

sau: “em bé đang ăn bánh_ngọt”

Như vậy, những từ đơn vẫn được giữ nguyên, còn các từ được cấu thành từ nhiều âm tiết được liên kết bằng dấu gạch dưới. Cách biểu diễn này giúp xác định rõ đơn vị từ — yếu tố quan trọng để các bước xử lý tiếp theo hoạt động chính xác. Đặc biệt, việc thống nhất phương pháp phân đoạn với dữ liệu mà PhoBERT đã được huấn luyện giúp văn bản đầu vào tương thích tốt hơn với cơ chế *subword tokenization* của mô hình, hạn chế sai lệch trong quá trình biểu diễn và nâng cao tính ổn định khi huấn luyện cũng như suy diễn.

3.3 Chia tập dữ liệu

3.3.1 Chia phân tầng

Dữ liệu sau khi được phân đoạn và làm sạch được tổ chức thành ba tập theo chức năng sử dụng trong quá trình thực nghiệm, gồm Train (huấn luyện), Validation (kiểm định) và Test (kiểm tra). Tập Train cung cấp dữ liệu để mô hình học các tham số. Trong giai đoạn phát triển, kết quả trên Validation được sử dụng để theo dõi quá trình học và làm cơ sở lựa chọn, điều chỉnh siêu tham số. Tập Test được tách biệt khỏi các bước huấn luyện và tinh chỉnh, chỉ được sử dụng khi mô hình đã hoàn thiện nhằm đánh giá kết quả trên những mẫu dữ liệu chưa được sử dụng trước đó.

Phương pháp phân tầng được áp dụng khi chia dữ liệu nhằm giữ cho tỷ lệ giữa hai lớp không thay đổi đáng kể trong các tập Train, Validation và Test. Dựa trên nguyên tắc này, dữ liệu được phân chia theo các tỷ lệ được trình bày dưới đây:

- Train: 70%
- Validation: 15%
- Test: 15%
- `random_state = 42`

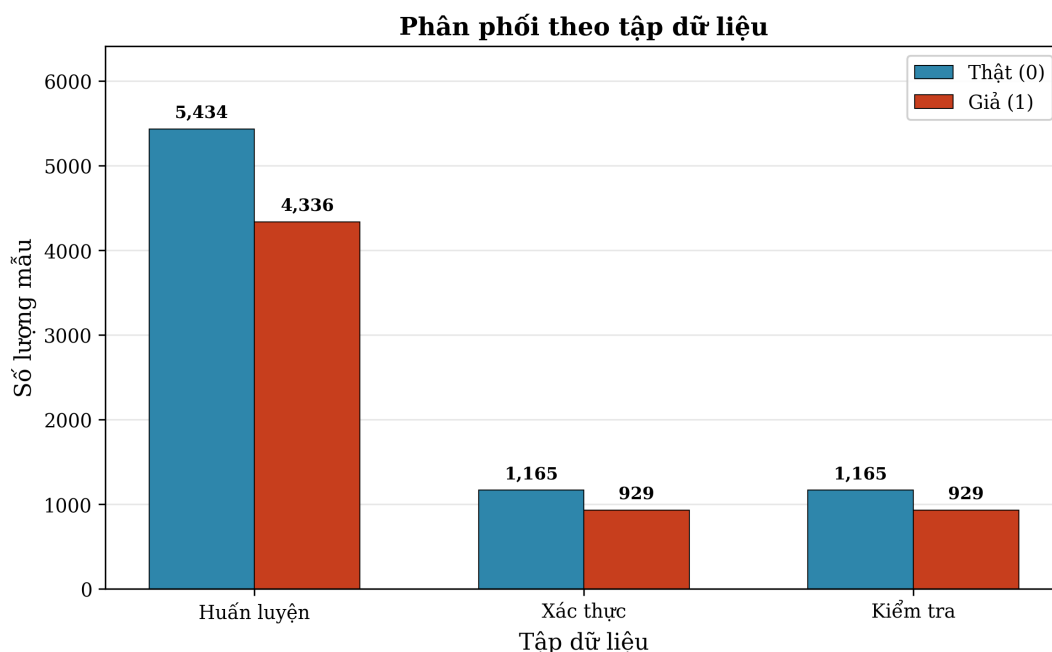
Về quy trình, tập Test được tách ra trước (15% tổng dữ liệu); phần còn lại 85% tiếp tục được tách thêm 15% tổng dữ liệu ban đầu (tương đương xấp xỉ 17,6% phần còn lại) để tạo thành tập Validation. Tham số `stratify = label` đảm bảo phân phối nhãn được bảo toàn giữa các tập:

$$P_{\text{train}}(Y) \approx P_{\text{val}}(Y) \approx P_{\text{test}}(Y)$$

Nhờ đó, sai lệch phân phối nhãn giữa các tập được kiểm soát chặt chẽ — yếu tố quan trọng để quá trình đánh giá phản ánh đúng khả năng tổng quát hóa của mô hình.

Kết quả chia tập cuối cùng:

- Train: 9.770 mẫu
- Validation: 2.094 mẫu
- Test: 2.094 mẫu



Hình 3.2: Phân phối nhãn trong các tập dữ liệu huấn luyện, kiểm định và kiểm tra

Bảng 3.1 mô tả thống kê chi tiết sau khi chia tập:

Bảng 3.1: Thống kê tập dữ liệu

tập	tổng	<i>tin thật</i>	<i>tin giả</i>	độ dài tb
huấn luyện	9.770	5.434(55,6%)	4.336(44,4%)	90,1
xác thực	2.094	1.165(55,6%)	929(44,4%)	91,7
kiểm tra	2.094	1.165(55,6%)	929(44,4%)	96,5

Bảng 3.1 tổng hợp số lượng mẫu, phân phối nhãn và độ dài trung bình của văn bản trong từng tập. Có thể nhận thấy lớp *tin thật* chiếm 55,6% và lớp *tin giả* chiếm 44,4% ở cả ba tập — cho thấy chiến lược phân tầng đã bảo toàn phân phối nhãn ban đầu giữa huấn luyện, kiểm định và kiểm tra.

Độ dài trung bình văn bản trong tập kiểm định (91,7 từ) và kiểm tra (96,5 từ) cao hơn nhẹ so với tập huấn luyện (90,1 từ), phản ánh sự đa dạng về cấu trúc văn bản trong toàn bộ dữ liệu. Mặc dù tồn tại sự mất cân bằng lớp ở mức vừa phải, tỷ lệ này không quá nghiêm trọng và sẽ được xử lý bằng cơ chế trọng số lớp (*class weighting*) trong quá trình huấn luyện nhằm giảm thiểu sai lệch dự đoán.

3.3.2 Kiểm tra rò rỉ dữ liệu

Toàn bộ nội dung văn bản trong ba tập Train, Validation, Test được chuyển đổi thành các tập hợp (*set*) dựa trên trường `text`. Sau đó, đồ án thực hiện phép giao giữa từng cặp tập nhằm phát hiện những bản ghi trùng lặp:

- Train và Validation
- Train và Test
- Validation và Test

Kết quả kiểm tra:

$$\text{train} \cap \text{Validation} = \emptyset$$

$$\text{train} \cap \text{test} = \emptyset$$

$$\text{Validation} \cap \text{test} = \emptyset$$

Không phát hiện bất kỳ văn bản trùng lặp nào giữa các cặp tập dữ liệu.

Khâu kiểm tra rò rỉ dữ liệu có ý nghĩa đặc biệt quan trọng đối với các bài toán phân loại văn bản. Nếu tồn tại mẫu trùng lặp giữa tập huấn luyện và tập kiểm tra, đánh giá sẽ trở nên thiên lệch: mô hình có thể đạt hiệu năng cao một cách giả tạo do đã “ghi nhớ” nội dung từ tập huấn luyện, thay vì thực sự học được các đặc trưng có khả năng tổng quát hóa. Vì vậy, đảm bảo tính độc lập giữa các tập dữ liệu là điều kiện tiên quyết để đánh giá đúng năng lực thực sự của mô hình.

3.4 Kết luận chương

Chương này đã trình bày toàn bộ quy trình chuẩn bị dữ liệu cho bài toán phát hiện tin giả tiếng Việt, bao gồm phân đoạn từ, làm sạch và chia tập theo chiến lược phân tầng. Dữ liệu được phân đoạn bằng RDRSegmenter nhằm đảm bảo tính tương thích với PhoBERT, sau đó trải qua bước loại bỏ các văn bản trùng lặp, rỗng hoặc quá ngắn để giảm nhiễu và nâng cao chất lượng đầu vào.

Tập dữ liệu sau làm sạch được chia theo tỷ lệ 70%–15%–15% cho các tập huấn luyện, kiểm định và kiểm tra với `random_state = 42`. Đồng thời, hệ thống tiến hành kiểm tra nghiêm ngặt rò rỉ dữ liệu giữa các tập nhằm đảm bảo không tồn tại giao nhau về nội dung.

Các quy trình này đảm bảo tính nhất quán, khách quan và khả năng tái lập (*reproducibility*), tạo nền tảng vững chắc cho quá trình huấn luyện và đánh giá mô hình ở chương tiếp theo.

Chương 4

XÁC ĐỊNH TIN GIẢ

Chương này trình bày toàn bộ cơ sở phương pháp luận cho bài toán phát hiện tin giả bằng tiếng Việt. Nội dung được tổ chức theo trình tự từ việc làm rõ bài toán, lựa chọn mô hình, mô tả kiến trúc hệ thống đến chi tiết kỹ thuật trích xuất đặc trưng và cấu hình huấn luyện của từng mô hình. Phần cuối chương khái quát các phân tích mở rộng — giải thích mô hình, hiệu chỉnh xác suất, phân tích mẫu khó và knowledge distillation — làm nền tảng cho chương thực nghiệm phía sau.

4.1 Phương pháp đề xuất

Bài toán nhận diện tin giả trong văn bản tiếng Việt ngày càng nhận được sự quan tâm trong lĩnh vực xử lý ngôn ngữ tự nhiên, đặc biệt trước sự gia tăng nhanh chóng của lượng thông tin trên môi trường trực tuyến. Khi các nội dung thiếu chính xác được chia sẻ với phạm vi rộng, chúng có thể ảnh hưởng đến quá trình tiếp nhận thông tin của người dùng và tạo ra những tác động không mong muốn đối với xã hội. Vì vậy, phát triển các phương pháp tự động có khả năng phát hiện nội dung sai lệch có ý nghĩa cả về mặt nghiên cứu lẫn ứng dụng thực tế.

Một quy trình phân loại văn bản thường bắt đầu bằng việc chuyển nội dung sang dạng biểu diễn số phù hợp với mô hình. Trong số các phương pháp phổ biến, **bag-of-words**, TF-IDF và **word embedding** cung cấp những cách khác nhau để biểu diễn thông tin từ văn bản. Sự phát triển của các phương pháp học sâu sau đó cho phép mô hình khai thác trực tiếp nhiều thông tin ngữ cảnh hơn trong quá trình học. Đáng chú ý, các mạng nơ-ron hồi tiếp và những mô hình ngôn ngữ xây dựng trên Transformer đã được áp dụng trong nhiều bài toán xử lý và phân loại văn bản, cho thấy khả năng khai thác đặc điểm ngữ cảnh của dữ liệu.

Dựa trên những hướng tiếp cận nói trên, đề án xây dựng một quy trình thực nghiệm nhằm

đánh giá và đối chiếu khả năng nhận diện tin giả giữa các nhóm mô hình khác nhau. Nhóm phương pháp truyền thống được đại diện bởi Logistic Regression và SVM — sử dụng đặc trưng trích xuất từ văn bản; nhóm học sâu gồm BiLSTM và PhoBERT — khai thác biểu diễn ngữ cảnh. Việc đặt các phương pháp này trong cùng một quy trình đánh giá tạo cơ sở để phân tích sự khác biệt về hiệu năng giữa cách tiếp cận dựa trên đặc trưng thủ công và cách tiếp cận dựa trên học biểu diễn từ dữ liệu.

4.2 Lý do lựa chọn các nhóm mô hình

Với bài toán phân loại văn bản và cụ thể là phát hiện tin giả, có ba tiêu chí quan trọng để đánh giá các phương pháp:

- *Khả năng diễn giải* — mức độ con người có thể hiểu được logic ra quyết định của mô hình.
- *Chi phí tính toán* — thời gian và tài nguyên cần thiết cho huấn luyện lẫn suy diễn.
- *Năng lực biểu diễn ngữ nghĩa* — khả năng nắm bắt các quan hệ ngữ nghĩa tinh tế trong văn bản.

Dựa trên các tiêu chí này, đề án này so sánh hai nhóm:

1. Nhóm học máy truyền thống, bao gồm:

- Logistic Regression
- Support Vector Machine

Đầu vào của hai mô hình trên là vector TF-IDF [10]. Cả hai mô hình có nền tảng lý thuyết vững, hoạt động ổn định trên dữ liệu văn bản đặc trưng chiều cao và thưa, đồng thời thuận tiện cho việc phân tích đóng góp của từng đặc trưng và kiểm soát hiện tượng quá khớp [9]. Vì vậy, chúng được sử dụng làm *strong baselines* — điểm chuẩn — để đo lường mức cải thiện thực sự mà các phương pháp phức tạp hơn mang lại.

2. Nhóm học sâu, bao gồm:

- BiLSTM
- PhoBERT

BiLSTM tận dụng hai hướng xử lý của mạng LSTM để khai thác thông tin từ cả phía trước lẫn phía sau của chuỗi. Khi được bổ sung cơ chế *attention*, mô hình có thể gắn

mức độ quan trọng khác nhau cho từng thành phần trong văn bản, qua đó hỗ trợ tập trung vào những thông tin hữu ích cho phân loại [11, 12].

PhoBERT được xây dựng trên nền tảng kiến trúc RoBERTa và được thiết kế nhằm khai thác đặc trưng ngôn ngữ của tiếng Việt thông qua quá trình tiền huấn luyện trên dữ liệu tiếng Việt. [15, 17, 20] — việc sử dụng biểu diễn ngữ cảnh được học từ quá trình tiền huấn luyện giúp mô hình nắm bắt thông tin ngữ nghĩa ở nhiều mức độ khác nhau. Đặc điểm này khiến PhoBERT phù hợp với các nhiệm vụ phân loại và xử lý văn bản tiếng Việt.

Việc đánh giá hai nhóm phương pháp trong cùng một thiết lập thực nghiệm tạo cơ sở để phân tích ba vấn đề trọng tâm của nghiên cứu:

1. Liệu các mô hình phân loại truyền thống có đạt được kết quả cạnh tranh khi được trang bị đặc trưng văn bản phù hợp hay không.
2. Mức độ cải thiện hiệu năng khi chuyển từ đặc trưng TF-IDF sang biểu diễn ngữ cảnh được học bởi các mô hình học sâu.
3. Đánh giá mức độ hỗ trợ của tri thức ngôn ngữ có sẵn trong PhoBERT đối với nhiệm vụ phân loại tin giả trên dữ liệu thực tế.

4.3 Tổng quan kiến trúc hệ thống

Quy trình thực nghiệm được tổ chức thành ba giai đoạn nối tiếp, lần lượt là xử lý dữ liệu, chuẩn bị biểu diễn đầu vào và thực hiện huấn luyện, đánh giá:

1. **Tiền xử lý dữ liệu:** các văn bản được đưa về dạng thống nhất bằng cách chuẩn hóa nội dung và loại bỏ những thành phần không cần thiết, sau đó tiến hành phân đoạn từ tiếng Việt. Kết quả xử lý được sử dụng để tạo ba tập Train, Validation và Test theo tỷ lệ đã xác định. Cách phân chia này nhằm tách biệt dữ liệu dùng cho quá trình học với dữ liệu phục vụ kiểm tra và đánh giá mô hình.
2. **Biểu diễn dữ liệu:** dữ liệu đầu vào được xây dựng theo yêu cầu của từng mô hình. Logistic Regression và SVM khai thác biểu diễn TF-IDF, trong khi BiLSTM sử dụng các vector từ được tạo bằng FastText. Với PhoBERT, văn bản được mã hóa bằng BPE (**byte-pair encoding**) trước khi đưa vào mô hình. Các dạng biểu diễn tương ứng sau đó được sử dụng làm đầu vào cho nhiệm vụ phân loại.

3. **Huấn luyện và đánh giá:** bốn mô hình được thực nghiệm độc lập với các thiết lập tương ứng trên cùng nguồn dữ liệu, từ đó tạo cơ sở cho việc đối chiếu kết quả. Hiệu năng được đánh giá thông qua **accuracy**, **F1-score** và **ROC-AUC**. Kết quả từ các thí nghiệm được tổng hợp để so sánh khả năng phân loại và xem xét sự khác biệt về hiệu năng giữa các mô hình.

4.4 Trích xuất đặc trưng

4.4.1 TF-IDF cho mô hình học máy truyền thống

4.4.1.1 Lý do lựa chọn TF-IDF

TF-IDF chuyển văn bản thành biểu diễn số dựa trên mức độ quan trọng của các từ trong tập dữ liệu. Kết quả thường là các vector thưa (*sparse vectors*), trong đó mỗi chiều biểu diễn một đặc trưng từ vựng và được gán một giá trị tương ứng. Dạng biểu diễn này phù hợp với những mô hình phân loại tuyến tính như Logistic Regression và SVM, bởi các mô hình có thể trực tiếp khai thác những chiều đặc trưng có khả năng phân biệt giữa các lớp dữ liệu.

Một hạn chế của TF-IDF là không mô tả được đầy đủ quan hệ ngữ nghĩa và ngữ cảnh của từ như các phương pháp *word embedding* hay mô hình ngôn ngữ tiên huấn luyện. Tuy nhiên, đặc trưng dựa trên tần suất từ vẫn cung cấp những tín hiệu hữu ích cho nhiều nhiệm vụ phân loại văn bản. Đặc biệt, các từ hoặc cụm từ có mức độ phân biệt cao giữa các lớp có thể được thể hiện rõ thông qua trọng số TF-IDF. Vì vậy, phương pháp này được sử dụng như một mốc đối chứng để đánh giá mức cải thiện khi áp dụng những phương pháp biểu diễn phức tạp hơn.

Đối với nhiệm vụ phát hiện tin giả, tín hiệu từ vựng có thể xuất hiện dưới nhiều dạng khác nhau, chẳng hạn cách đặt tiêu đề, những cách diễn đạt nhằm gây chú ý hoặc các từ khóa đặc trưng cho nội dung sai lệch. Việc bổ sung các đặc trưng n-gram giúp mô hình xem xét cả những chuỗi từ thay vì chỉ xử lý từng từ độc lập. Trong nghiên cứu này, TF-IDF được sử dụng cùng Logistic Regression và SVM để hình thành nhóm mô hình đối chứng, từ đó tạo cơ sở đánh giá với hai phương pháp BiLSTM và PhoBERT.

4.4.1.2 Cấu hình TF-IDF

Đối với nhóm mô hình học máy truyền thống, các thiết lập của TF-IDF được lựa chọn dựa trên kết quả *ablation study* tại Mục 5.4. Mục tiêu của quá trình này là xác định cấu hình

vừa cung cấp đủ thông tin cho mô hình vừa hạn chế những đặc trưng không cần thiết. Các giá trị được sử dụng gồm:

- **Kích thước từ điển:** chỉ giữ lại 40.000 từ/ngữ có mức độ xuất hiện cao nhất trong dữ liệu. Giới hạn này giúp không gian đặc trưng có kích thước phù hợp và giảm tác động của những đặc trưng ít hữu ích.
- **N-gram:** biểu diễn được xây dựng từ cả đơn từ (Unigram) và các cặp từ (Bigram), hay còn gọi là N-Gram = (1,2). Nhờ xét thêm các chuỗi từ, mô hình có thể khai thác những cụm biểu đạt mang thông tin phân biệt mà biểu diễn unigram đơn thuần có thể bỏ qua.
- **Tần số tài liệu:** các đặc trưng có tần suất tài liệu thấp hơn ngưỡng $\text{min_df}=5$ hoặc xuất hiện trong tỷ lệ tài liệu vượt quá $\text{max_df}=0.95$ được loại khỏi không gian đặc trưng. Điều này giúp hạn chế những từ quá hiếm hoặc quá phổ biến trong tập dữ liệu.
- **TF sublinear:** thay cho việc sử dụng trực tiếp tần suất xuất hiện, phép biến đổi $\text{TF} = 1 + \log f_{t,d}$ được áp dụng để làm giảm mức ảnh hưởng của những từ xuất hiện với số lần rất lớn trong cùng một văn bản. Nhờ đó, sự xuất hiện của từ được xem xét theo hướng ít phụ thuộc hơn vào tần suất tuyệt đối.

Các thiết lập trên được cố định sau quá trình lựa chọn để sử dụng trong thực nghiệm. Việc giới hạn không gian đặc trưng, kiểm soát tần suất tài liệu và sử dụng TF sublinear giúp cân đối giữa lượng thông tin được giữ lại với độ phức tạp của biểu diễn TF-IDF, đồng thời hạn chế khả năng mô hình phụ thuộc quá mức vào những đặc trưng riêng của tập dữ liệu huấn luyện.

4.4.2 Word Embedding cho BiLSTM

Đối với kiến trúc BiLSTM, biểu diễn văn bản được xây dựng thông qua kỹ thuật word embedding kết hợp giữa embedding tiền huấn luyện và học từ dữ liệu. Từ điển được hình thành dựa trên toàn bộ tập train, sau khi loại bỏ các từ xuất hiện dưới 2 lần nhằm giảm nhiễu và kiểm soát kích thước không gian từ vựng; kết quả thu được 23.166 từ.

Hai token đặc biệt được bổ sung: <pad> (Index 0) dùng để đệm các chuỗi ngắn và <unk> (Index 1) đại diện cho các từ ngoài từ điển. Mỗi văn bản sau tiền xử lý được chuyển đổi thành một chuỗi số nguyên với độ dài tối đa 128, đảm bảo tính nhất quán về kích thước đầu vào cho mô hình tuần tự.

Các vector embedding có chiều 300 được khởi tạo từ bộ embedding tiền huấn luyện FastText [3] phiên bản tiếng Việt (cc.vi.300.bin). FastText tận dụng thông tin n-gram ký tự để xây dựng biểu diễn từ, cho phép xử lý hiệu quả các từ ngoài từ điển bằng cách tổng

hợp vector của các n-gram thành phần. Ma trận embedding tiền huấn luyện được nạp vào lớp embedding của mô hình và tiếp tục được tinh chỉnh trong quá trình huấn luyện thông qua lan truyền ngược — cho phép mô hình điều chỉnh biểu diễn ngữ nghĩa phù hợp với đặc thù của bài toán phát hiện tin giả. Việc kết hợp embedding tiền huấn luyện với fine-tuning giúp tận dụng tri thức ngôn ngữ học được từ ngữ liệu lớn (Common Crawl), đồng thời thích ứng tốt với miền dữ liệu tiếng Việt của bài toán.

4.4.3 Tokenization PhoBERT (BPE)

Quá trình mã hóa đầu vào cho PhoBERT sử dụng thuật toán byte-pair encoding (BPE) [16] với từ điển gồm 64.000 subword, cho phép mô hình xử lý hiệu quả các từ ngoài từ điển và hiện tượng đa hình trong tiếng Việt. Mỗi văn bản được tiền xử lý theo các bước:

- Thêm token đặc biệt `[cls]` ở đầu và `[sep]` ở cuối chuỗi để đánh dấu ranh giới văn bản.
- Chuỗi token được cắt hoặc đệm về độ dài cố định 256 nhằm đảm bảo tính nhất quán đầu vào.
- Sinh `attention_mask` để phân biệt token thực (1) và token đệm (0) — hỗ trợ quá trình tính toán attention trong mô hình.

Chiến lược này giúp PhoBERT tận dụng tối đa thông tin ngữ cảnh và duy trì hiệu quả phân loại khi áp dụng trên nhiều dạng dữ liệu tiếng Việt khác nhau.

4.5 Các mô hình phân loại

4.5.1 Các mô hình học máy truyền thống

4.5.1.1 Logistic Regression

Mô hình hồi quy Logistic sử dụng điều chuẩn L2 nhằm hạn chế hiện tượng quá khớp và áp dụng cơ chế cân bằng lớp (`class_weight='balanced'`) để xử lý mất cân bằng nhân. GridSearchCV được sử dụng để tìm kiếm bộ siêu tham số phù hợp, với quá trình đánh giá được thực hiện bằng 5-fold cross-validation. Kết quả từ các lần chia dữ liệu được sử dụng làm cơ sở để lựa chọn cấu hình có hiệu năng phù hợp.:

- Lưới tìm kiếm: $C \in \{0,5; 1; 5; 10; 20; 50\}$, $\text{solver} \in \{\text{liblinear}, \text{saga}\}$
- Cấu hình tối ưu: $C = 10$, $\text{solver} = \text{saga}$, $\text{max_iter} = 3000$

Việc lựa chọn này giúp mô hình đạt hiệu năng tốt nhất trên tập kiểm định mà không phụ thuộc quá nhiều vào phân phối nhãn thực tế.

4.5.1.2 Support Vector Machine

Mô hình máy vectơ hỗ trợ (SVM) sử dụng LinearSVC với trọng số lớp cân bằng nhằm kiểm soát ảnh hưởng do phân bố nhãn không đồng đều. GridSearchCV được sử dụng để tìm kiếm cấu hình siêu tham số phù hợp, với quá trình đánh giá được thực hiện bằng 5-fold cross-validation:

- Lưới tìm kiếm: $C \in \{0,1; 0,5; 1; 2; 5; 10\}$
- Cấu hình tối ưu: $C = 0,5$, LinearSVC kết hợp hiệu chuẩn xác suất (Calibration)

Chiến lược này đảm bảo SVM vừa tận dụng được đặc trưng TF-IDF, vừa giữ được khả năng phân biệt tốt giữa các lớp.

4.5.2 Các mô hình học sâu

4.5.2.1 Mạng BiLSTM

Kiến trúc BiLSTM trong nghiên cứu gồm các thành phần chính:

1. **Embedding:** ma trận embedding kích thước $(V \times 300)$, cho phép học biểu diễn ngữ nghĩa trực tiếp từ dữ liệu.
2. **BiLSTM 1 tầng:** mỗi tầng có `hidden_dim=128` cho cả hai chiều (Forward + Backward), đầu ra tại mỗi bước thời gian là 2×128 .
3. **Soft Attention:** cơ chế soft attention tính trọng số α_t cho từng bước thời gian, qua đó các thành phần mang thông tin quan trọng trong văn bản được mô hình chú trọng hơn trong quá trình xử lý.
4. **Classifier:** hai lớp dropout [30] (0.3) xen kẽ với các lớp tuyến tính và ReLU, giúp giảm overfitting và tăng khả năng biểu diễn phi tuyến.

Chiến lược huấn luyện sử dụng AdamW [31] ($\eta = 10^{-3}$, `weight_decay`= 10^{-4}), hàm mất mát CrossEntropy có trọng số lớp để xử lý mất cân bằng, `batch`=64, tối đa 20 epochs với cơ chế dừng sớm (`patience`=5). Ngoài ra, áp dụng ReduceLROnPlateau (`factor`=0.5, `patience`=2) để điều chỉnh tốc độ học và gradient clipping (`max_norm`=1.0) nhằm đảm bảo ổn định quá trình tối ưu.

4.5.2.2 Tinh chỉnh PhoBERT

Kiến trúc PhoBERT fine-tuning gồm hai thành phần chính:

1. **Encoder PhoBERT:** 12 lớp Transformer encoder, đầu ra vector [cls] 768 chiều đóng vai trò đại diện toàn cục cho văn bản.
2. **Classifier head:** hai lớp Dropout (0.1) xen kẽ với linear (768→384→2) và ReLU, giúp tăng khả năng biểu diễn phi tuyến và giảm overfitting.

Chiến lược huấn luyện sử dụng AdamW [31] ($\eta = 3 \times 10^{-5}$, weight_decay=0.01), linear warmup (10%), batch=16, tối đa 8 epochs, gradient accumulation steps=4, gradient clipping (max_norm=1.0) và cơ chế dừng sớm (patience=2).

4.6 Chiến lược xử lý mất cân bằng nhãn

Mặc dù phân bố nhãn giữa hai lớp tương đối gần nhau (55,6% *thật* / 44,4% *giả*), lớp *tin giả* vẫn có số lượng thấp hơn và có thể chịu ảnh hưởng trong quá trình mô hình học và phân loại. Do đó, đề án kết hợp hai chiến lược nhằm kiểm soát tác động của sự chênh lệch này:

- **Chia phân tầng (stratified split):** Phương pháp phân tầng được áp dụng khi tạo các tập Train, Validation và Test nhằm giữ lại đặc điểm phân bố nhãn của dữ liệu ban đầu. Việc duy trì tỷ lệ các lớp giữa các tập giúp hạn chế sai lệch trong quá trình đánh giá và giảm khả năng kết quả mô hình bị tác động bởi cách phân chia dữ liệu.
- **Trọng số lớp trong hàm mất mát (class weighting):** mỗi lớp c được gán trọng số nghịch đảo tỷ lệ với tần suất xuất hiện:

$$w_c = \frac{N}{K \cdot N_c}$$

trong đó N là tổng số mẫu, K là số lớp và N_c là số mẫu thuộc lớp c . Trọng số này được tích hợp trực tiếp vào hàm mất mát CrossEntropy — làm tăng mức phạt cho các mẫu thuộc lớp thiểu số, từ đó cải thiện recall cho lớp tin giả. Đối với các mô hình truyền thống (LR, SVM), cơ chế tương đương được thực hiện thông qua tham số `class_weight='balanced'` trong scikit-learn [25].

Sự kết hợp hai chiến lược trên giúp đảm bảo quá trình huấn luyện và đánh giá phản ánh đúng thực tế phân phối nhãn, đồng thời giảm thiểu nguy cơ mô hình bỏ sót các trường hợp thuộc lớp thiểu số.

4.7 Các phân tích mở rộng được thiết kế cho chương thực nghiệm

Ngoài các độ đo hiệu năng chuẩn (Accuracy, F1, ROC-AUC), chương thực nghiệm sẽ thực hiện bốn nhóm phân tích mở rộng nhằm đánh giá mô hình một cách toàn diện hơn. Mục này mô tả trước các phương pháp được sử dụng.

4.7.1 Phân tích khả năng giải thích (Explainability)

Để hiểu *từ nào* trong văn bản có ảnh hưởng lớn nhất đến dự đoán của mỗi mô hình, ba họ phương pháp attribution được sử dụng:

- **Cho mô hình tuyến tính (LR, SVM):** KernelSHAP [32] với tập nền là vector 0 (đặc trưng vắng mặt) cho LR; LinearSHAP dựa trên trọng số mô hình cho SVM. Đầu ra là đóng góp log-odds cho từng n-gram xuất hiện trong văn bản.
- **Cho BiLSTM:** Integrated Gradients (IG, Sundararajan et al., 2017) và vanilla gradients trên embedding đầu vào với baseline là vector 0. Mỗi token được gán điểm bằng chuẩn L2 của gradient tích phân theo chiều embedding.
- **Cho PhoBERT:** SHAP trên embedding (đầu vào liên tục), LayerIntegratedGradients từ Captum, và Attention Rollout (Abnar & Zuidema, 2020) lấy hàng CLS của ma trận rollout qua 12 lớp Transformer.

Ngoài việc trực quan hóa token attribution trên từng mẫu, hai phép đo định lượng được sử dụng: (i) *Jaccard overlap* giữa top- k token quan trọng của các cặp phương pháp — đo lường mức độ đồng thuận giữa các hướng giải thích; (ii) *faithfulness score* — mức giảm xác suất của lớp dự đoán sau khi che k token được attribution đánh giá quan trọng nhất.

4.7.2 Hiệu chỉnh xác suất sau huấn luyện (Post-Hoc Calibration)

Phân tích sơ bộ (xem Mục 2.8) cho thấy PhoBERT đạt Accuracy cao nhưng xác suất dự đoán có thể chưa phản ánh đúng tần suất thực nghiệm. Để khắc phục mà không cần huấn luyện lại mô hình, ba phương pháp hiệu chỉnh sau huấn luyện được áp dụng cho cả bốn mô hình:

- **Platt scaling** (Platt, 1999) — hồi quy logistic một chiều trên logit: $\sigma(a \cdot z + b)$.
- **Temperature scaling** (Guo et al., 2017) — tìm hằng số $T > 0$ tối thiểu hóa NLL trên tập validation: $p = \sigma(Z/T)$.
- **Isotonic regression** (Zadrozny & Elkan, 2002) — ánh xạ phi tham số, đơn điệu từ $\sigma(Z)$ sang xác suất hiệu chỉnh.

Cả ba phương pháp được fit trên tập validation và đánh giá trên tập kiểm tra. Đối với LR và SVM, logit được tính từ hàm quyết định (`model.decision_function()`) của mô hình đã huấn luyện. Đối với BiLSTM và PhoBERT, logit được trích từ đầu ra trước softmax của mô hình PyTorch tương ứng. Để đảm bảo tính tái lập và tăng tốc độ thực nghiệm, logits được tính trước một lần trên tập validation và tập kiểm tra rồi lưu lại, phục vụ cho các thí nghiệm hiệu chỉnh.

4.7.3 Phân tích các trường hợp khó (Hard Cases)

Nhóm mẫu mà cả bốn mô hình đều phân loại sai được gọi là *hard examples*. Mỗi mẫu khó được phân loại theo các chiều sau bằng heuristic tự động kết hợp kiểm tra thủ công:

- **Chủ đề:** dựa trên từ khóa (chính trị, tài chính, y tế,...).
- **Lý do thất bại:** *semantic reasoning* (mô hình không nắm ngữ nghĩa); *knowledge verification* (cần tri thức nền tảng); *dataset ambiguity* (bản thân nhãn mơ hồ); *stylistic* (văn phong đánh lừa mô hình).
- **Độ dài:** số từ trong văn bản — giúp phát hiện hiện tượng thiếu ngữ cảnh.
- **Mật độ từ vựng:** tỷ lệ từ mang nội dung / tổng số từ.

4.7.4 Knowledge Distillation

PhoBERT đạt F1-macro cao nhất nhưng có kích thước lớn (~135 triệu tham số), không phù hợp với triển khai trên thiết bị biên. Một thí nghiệm knowledge distillation [33] được thiết kế nhằm kiểm chứng giả thuyết: một mô hình BiLSTM nhỏ (sinh viên) có thể thừa hưởng phần lớn chất lượng của một mô hình lớn (giáo viên), đổi lại một mức sụt giảm F1 có thể chấp nhận được và kích thước mô hình nhỏ hơn đáng kể. Trong thiết kế này, giáo viên được chọn là BiLSTM-gốc (đã huấn luyện trên cùng bộ dữ liệu) vì kích thước phù hợp để làm giáo viên cho một BiLSTM nhỏ hơn; PhoBERT không được sử dụng làm giáo viên do kiến trúc quá khác biệt.

Hàm mất mát KD kết hợp hai thành phần:

$$L = \alpha \cdot \text{KL}\left(\frac{s}{T} \parallel \frac{t}{T}\right) \cdot T^2 + (1 - \alpha) \cdot \text{CE}(S, Y),$$

trong đó s, t lần lượt là logits của sinh viên và giáo viên, T là nhiệt độ phân phối và α là trọng số của thành phần soft target. Logits của giáo viên PhoBERT được tính trước một lần trên tập huấn luyện và tập kiểm tra rồi lưu lại để tái sử dụng suốt quá trình huấn luyện sinh viên.

4.8 Kết luận chương

Chương này đã trình bày phương pháp luận đầy đủ cho toàn bộ khung thực nghiệm của đề án, xoay quanh bốn trụ cột:

- **Trích xuất đặc trưng:** TF-IDF (cho LR/SVM), embedding tùy chỉnh (cho BiLSTM), và BPE tokenizer của PhoBERT (cho PhoBERT).
- **Kiến trúc mô hình:** bốn mô hình (LR, SVM, BiLSTM, PhoBERT) đại diện cho ba trường phái chính của NLP — đặc trưng thủ công, học biểu diễn tuần tự, và tiền huấn luyện.
- **Siêu tham số:** lựa chọn có nguyên tắc với tài liệu tham chiếu (Bengio et al., 2003 cho embedding dim ≥ 128 ; Devlin et al., 2019 cho learning rate 2×10^{-5} của Transformer).
- **Phân tích mở rộng:** bốn nhóm phân tích (giải thích, hiệu chỉnh xác suất, mẫu khó, knowledge distillation) được thiết kế để đánh giá mô hình toàn diện hơn ngoài Accuracy/F1.

Thiết kế này đảm bảo so sánh công bằng trên cùng bộ dữ liệu và cùng quy trình đánh giá — mỗi mô hình được huấn luyện với cùng phân chia tập, cùng siêu tham số đã được điều chỉnh hợp lý, và cùng ba chỉ số chính (Accuracy, F1-macro, ROC-AUC).

KẾT QUẢ THỰC NGHIỆM

5.1 Môi trường thực nghiệm

Đề tài được thực hiện trên hệ thống: GPU NVIDIA (CUDA 12.8), RAM 32 GB, Python 3.11, PyTorch 2.10, Transformers 4.57, scikit-learn [25] 1.8.

Bảng 5.1: Siêu tham số các mô hình

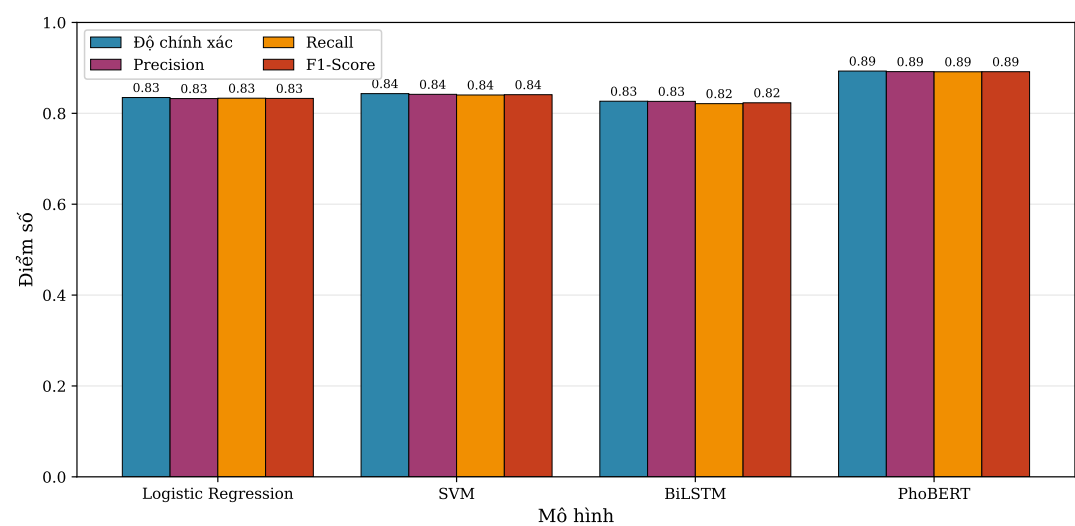
mô hình	siêu tham số
Logistic Regression	C=10, solver=saga, class_weight=balanced, max_iter=3000
SVM	C=0,5, linearsvc, class_weight=balanced
BILSTM	Embedding dim=300, hidden dim=128 num layers=1, Dropout=0,3 learning rate=0,001, batch size=64 early stopping patience=5
PhoBERT	pre-trained: vinai/phobert-base max length=256, weight decay=0,01 learning rate=3e-05, batch size=16 epochs=8, warmup ratio=0,1

Bảng 5.2: Độ phức tạp và thời gian huấn luyện

mô hình	loại	tham số	thời gian huấn luyện
Logistic Regression	ML truyền thống	~10K	1.6 s
SVM	ML truyền thống	~10K	3 min
BILSTM	Học sâu	~500K	5 min
PhoBERT	Transformer	~135M	1.2 h

5.2 So sánh hiệu năng các mô hình

5.2.1 Kết quả tổng thể



Hình 5.1: So sánh các chỉ số hiệu năng (Accuracy, F1-Score, ROC-AUC) của bốn mô hình trên tập kiểm tra

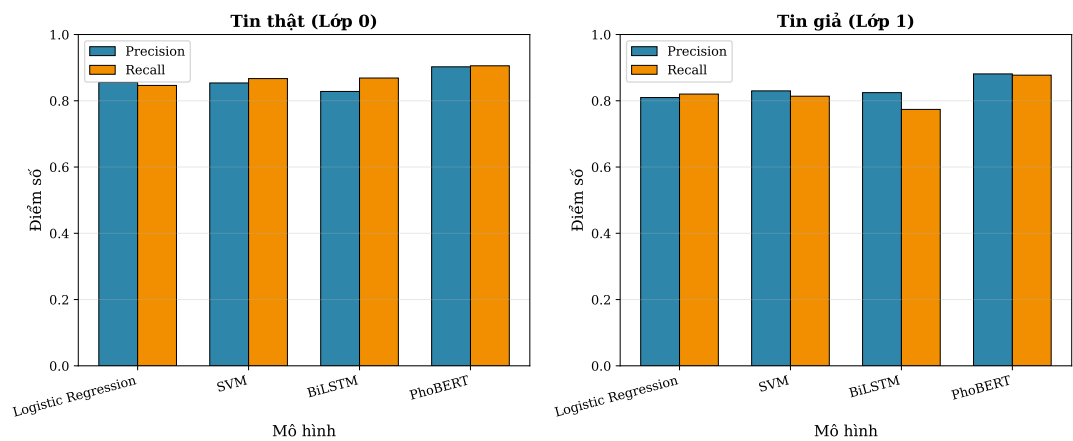
Bảng 5.3: So sánh hiệu suất các mô hình trên tập kiểm tra

mô hình	độ chính xác	precision	recall	F1-Score	ROC-AUC
Logistic Regression	0,8348	0,8325	0,8333	0,8329	0,9188
SVM	0,8434	0,8418	0,8404	0,8410	0,9190
BILSTM	0,8252	0,8228	0,8235	0,8232	0,9048
PhoBERT	0,9007	0,9018	0,8968	0,8988	0,9495

Bảng 5.3 trình bày hiệu năng trên tập kiểm tra. PhoBERT đạt kết quả tốt nhất với F1

= 89,88% và $AUC = 0,9495$, vượt LR (+6,59 điểm F1), SVM (+5,78 điểm F1) và BiLSTM (+7,56 điểm F1). Đáng chú ý, SVM vượt BiLSTM mặc dù kiến trúc đơn giản hơn — cho thấy đặc trưng TF-IDF chất lượng cao có thể cạnh tranh với học sâu khi dữ liệu huấn luyện có quy mô vừa phải.

5.2.2 Hiệu năng theo từng lớp



Hình 5.2: Precision, Recall và F1-Score theo từng lớp (*thật / giả*) cho bốn mô hình

Bảng 5.4: Chỉ số hiệu suất theo từng lớp

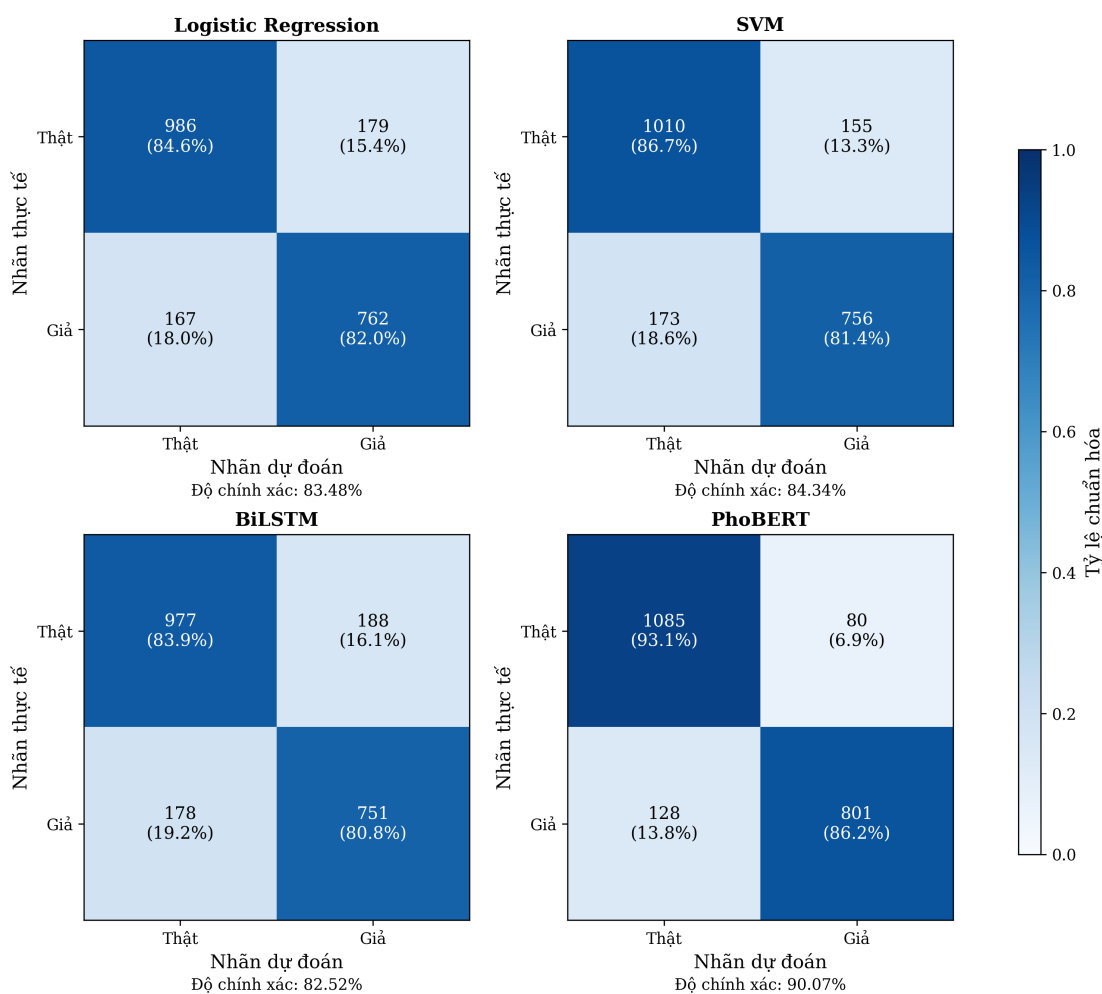
mô hình	lớp	precision	recall	F1-Score
Logistic Regression	thật	0,8552	0,8464	0,8507
	Giả	0,8098	0,8202	0,8150
SVM	thật	0,8538	0,8670	0,8603
	Giả	0,8299	0,8138	0,8217
BiLSTM	thật	0,8459	0,8386	0,8422
	Giả	0,7998	0,8084	0,8041
PhoBERT	thật	0,8945	0,9313	0,9125
	Giả	0,9092	0,8622	0,8851

Từ bảng 5.4:

- PhoBERT: Recall = 86,22% cho lớp *giả* — phát hiện tin giả tốt nhất trong bốn mô hình.

- BiLSTM: Recall = 80,84% cho lớp *giả* — bỏ sót nhiều tin giả nhất (nguy hiểm trong thực tế).
- SVM đạt Recall = 81,38% cho lớp *giả*, cao hơn LR (82,02%) và BiLSTM.

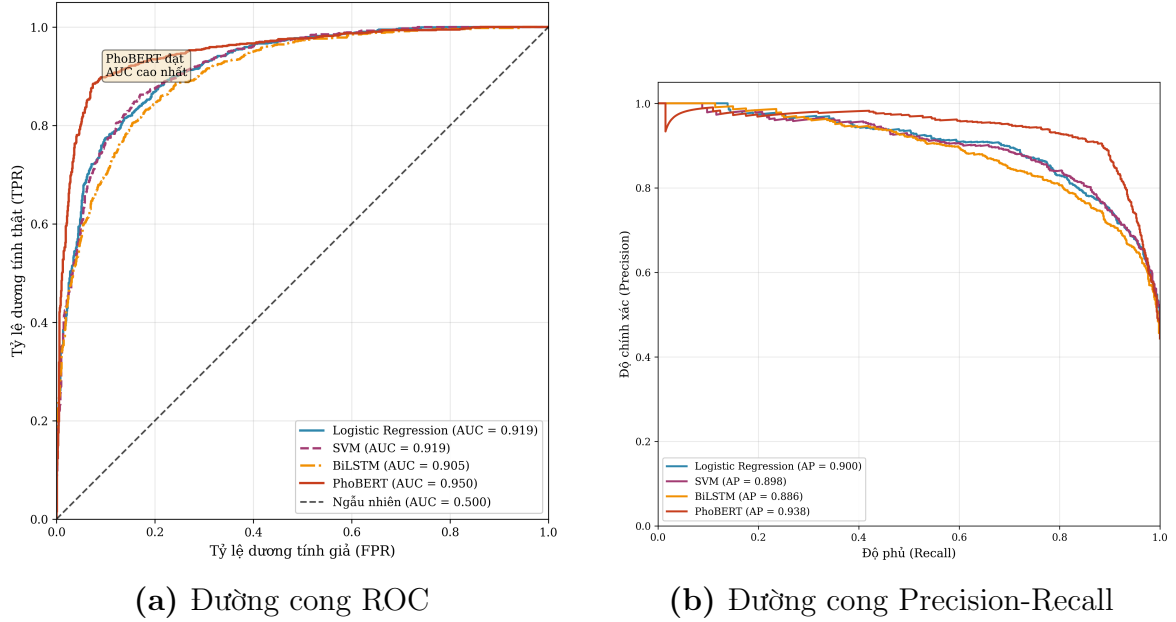
5.2.3 Ma trận nhầm lẫn



Hình 5.3: Ma trận lỗi của bốn mô hình khi thực hiện dự đoán trên tập kiểm tra (2.094 mẫu)

PhoBERT: 1.085 TN, 801 TP, 80 FP, 128 FN trên 2.094 mẫu test. BiLSTM có nhiều lỗi nhất (366 lỗi, FP=188, FN=178), trong khi PhoBERT ít lỗi nhất (208 lỗi).

5.2.4 Đường cong ROC và Precision-Recall



Hình 5.4: Đường cong ROC và Precision-Recall của bốn mô hình trên tập kiểm tra

PhoBERT (AUC = 0,9495) áp đảo so với SVM (0,9190), LR (0,9188) và BiLSTM (0,9048).

5.2.5 Phân tích hiệu chuẩn xác suất (Calibration Analysis)

Bên cạnh các chỉ số phân loại truyền thống, việc đánh giá chất lượng xác suất dự đoán của mô hình cũng đóng vai trò quan trọng trong các ứng dụng thực tế. Một mô hình được coi là có hiệu chuẩn tốt (*well-calibrated*) khi xác suất dự đoán phản ánh đúng tỷ lệ dương tính thực tế: nếu mô hình dự đoán xác suất 80% cho một nhóm mẫu, thì khoảng 80% số mẫu đó phải thực sự thuộc lớp dương.

Điều này đặc biệt quan trọng trong bài toán phát hiện tin giả, vì người dùng hoặc hệ thống downstream cần biết mức độ tin cậy thực sự của mỗi dự đoán để quyết định hành động phù hợp (ví dụ: đánh dấu cảnh báo, chuyển sang kiểm tra thủ công hoặc tự động loại bỏ).

Nghiên cứu sử dụng ba chỉ số đánh giá hiệu chuẩn [28]:

- **Expected Calibration Error (ECE):** sai số hiệu chuẩn kỳ vọng, đo mức chênh lệch trung bình có trọng số giữa xác suất dự đoán và tỷ lệ dương tính thực tế trên các nhóm

mẫu (bin):

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{N} \cdot |\text{acc}(B_b) - \text{conf}(B_b)|$$

trong đó B_b là tập mẫu trong bin thứ b , $\text{acc}(B_b)$ là tỷ lệ dự đoán đúng và $\text{conf}(B_b)$ là xác suất dự đoán trung bình trong bin đó.

- **Maximum Calibration Error (MCE):** sai số hiệu chuẩn cực đại, phản ánh bin có độ lệch lớn nhất:

$$\text{MCE} = \max_b |\text{acc}(B_b) - \text{conf}(B_b)|$$

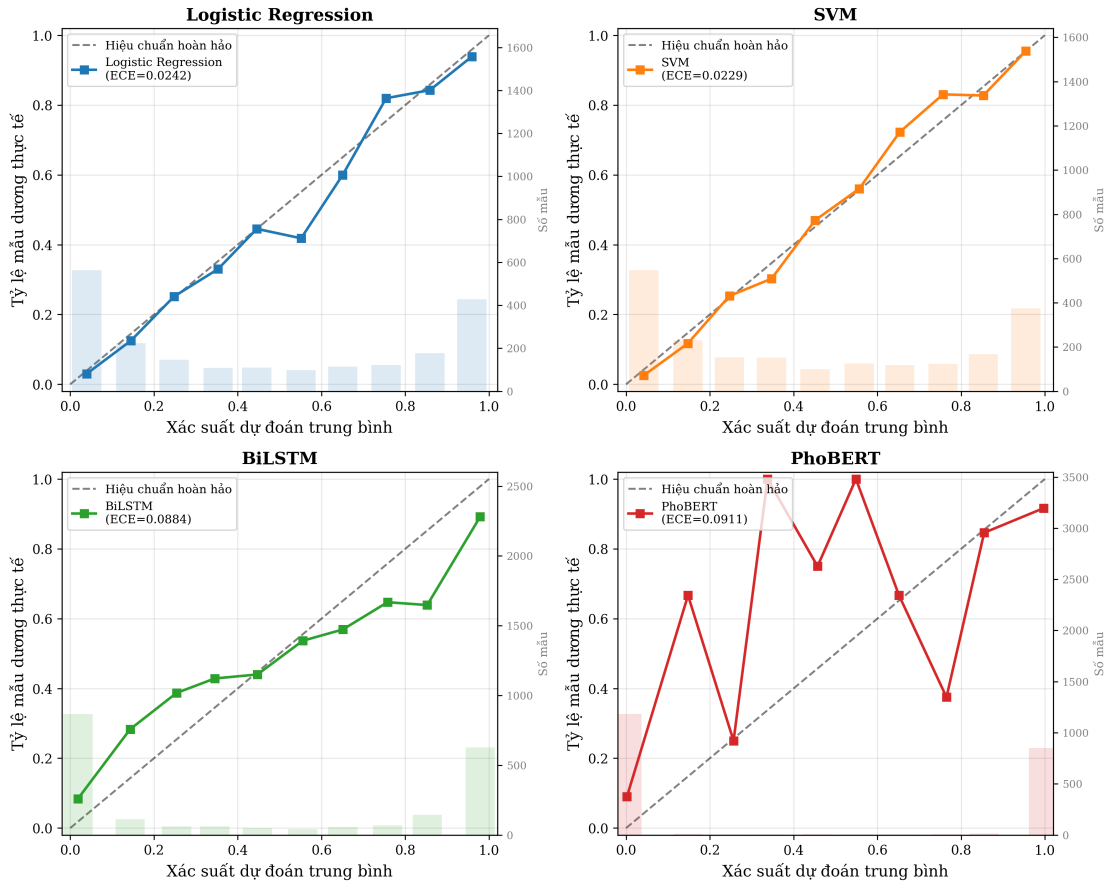
- **Brier Score:** trung bình bình phương sai số giữa xác suất dự đoán và nhãn thực:

$$\text{bs} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Brier Score kết hợp cả khả năng phân loại và hiệu chuẩn trong một chỉ số duy nhất.

Cả ba chỉ số đều có giá trị càng thấp càng tốt ($0 =$ hoàn hảo).

Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình



Hình 5.5: Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình trên tập kiểm tra. Đường chéo nét đứt biểu diễn hiệu chuẩn hoàn hảo; các điểm vuông biểu diễn tỷ lệ dương tính thực tế trong mỗi bin xác suất.

Bảng 5.5: Chỉ số hiệu chuẩn xác suất của bốn mô hình trên tập kiểm tra

mô hình	ECE ↓	MCE ↓	Brier Score ↓
Logistic Regression	0,0242	0,1329	0,1136
SVM	0,0229	0,0740	0,1134
BiLSTM	0,0884	0,2135	0,1348
PhoBERT	0,0911	0,6622	0,0925

Từ bảng 5.5 và Hình 5.5, có thể rút ra một số quan sát đáng chú ý:

- **Các mô hình học máy truyền thống có hiệu chuẩn tốt nhất:** SVM đạt ECE thấp nhất (0,0229) và MCE thấp nhất (0,0740), theo sau là LR (ECE = 0,0242). Điều này cho thấy xác suất dự đoán của SVM và LR phản ánh khá chính xác tỷ lệ dương tính thực tế.

- **PhoBERT có Brier Score tốt nhất nhưng MCE cao:** PhoBERT đạt Brier Score thấp nhất (0,0925), phản ánh khả năng phân loại vượt trội. Tuy nhiên, MCE cao (0,6622) cho thấy mô hình có khuynh hướng *overconfident* — đưa ra xác suất rất cao (gần 0 hoặc 1) ngay cả khi dự đoán sai. Hiện tượng này là đặc trưng phổ biến của các mô hình Transformer, đặc biệt sau quá trình fine-tuning trên tập dữ liệu nhỏ.
- **BiLSTM có hiệu chuẩn kém nhất:** với $ECE = 0,0884$ và Brier Score = 0,1348, BiLSTM cho thấy mức độ tin cậy của các xác suất dự đoán chưa tương ứng tốt với hiệu năng thực tế của mô hình.

Hàm ý thực tế. Phân tích hiệu chuẩn cho thấy trong các hệ thống triển khai thực tế, *không nên sử dụng trực tiếp* xác suất thô (raw probability) của PhoBERT làm mức độ tin cậy. Thay vào đó, cần áp dụng kỹ thuật hiệu chuẩn hậu xử lý (*post-hoc calibration*) như Temperature scaling hoặc Platt scaling để cải thiện chất lượng xác suất trước khi đưa vào hệ thống ra quyết định. Ngược lại, xác suất từ SVM và LR có thể được sử dụng trực tiếp với độ tin cậy cao hơn.

5.3 Kiểm định thống kê

5.3.1 Kiểm định McNemar

Kiểm định McNemar [24] là một phương pháp thống kê dùng để đánh giá sự khác biệt giữa hai mô hình phân loại khi cả hai cùng được áp trên cùng một tập dữ liệu kiểm thử. Thay vì xem xét kết quả của từng mô hình một cách độc lập, kiểm định dựa trên cặp dự đoán tương ứng của từng mẫu (*paired predictions*). Nhờ đặc điểm này, McNemar phù hợp để kiểm tra liệu chênh lệch về kết quả phân loại giữa hai mô hình có đủ bằng chứng để xem là có ý nghĩa thống kê hay có thể xuất hiện do biến động ngẫu nhiên của dữ liệu.

Hai giả thuyết được đặt ra trong kiểm định gồm:

- H_0 : sự khác biệt về hiệu năng phân loại giữa hai mô hình không đạt mức ý nghĩa thống kê.
- H_1 : sự khác biệt về hiệu năng phân loại giữa hai mô hình đạt mức ý nghĩa thống kê.

Thống kê kiểm định được tính theo công thức:

$$\chi^2 = \frac{(|B - C| - 1)^2}{b + c}$$

Hai đại lượng b và c đại diện cho các trường hợp bất đồng (discordant pairs) giữa hai mô hình. Các trường hợp cả hai cùng đúng hoặc cùng sai không ảnh hưởng đến thống kê kiểm định vì không phản ánh sự khác biệt tương đối.

Giá trị χ^2 càng lớn cho thấy mức độ chênh lệch giữa b và c càng cao — tức là sự khác biệt có xu hướng mang tính hệ thống thay vì ngẫu nhiên. Giá trị p -value được tính dựa trên phân phối chi-square với 1 bậc tự do. Nếu $p < \alpha$ (với $\alpha = 0,05$), giả thuyết H_0 bị bác bỏ và có thể kết luận sự khác biệt giữa hai mô hình là có ý nghĩa thống kê.

Hệ số điều chỉnh (-1) trong tử số là hiệu chỉnh liên tục (*continuity correction*), giúp giảm sai số loại I khi kích thước mẫu không quá lớn.

Do nghiên cứu thực hiện sáu phép so sánh cặp mô hình, việc tiến hành nhiều kiểm định đồng thời có thể làm gia tăng xác suất sai loại I. Để kiểm soát mức sai số toàn cục (*family-wise error rate*), phương pháp hiệu chỉnh Holm–Bonferroni được áp dụng. Cụ thể, các p -value được sắp xếp theo thứ tự tăng dần và so sánh tuần tự với các ngưỡng hiệu chỉnh tương ứng, bảo đảm rằng xác suất xuất hiện ít nhất một kết luận sai không vượt quá mức ý nghĩa α đã lựa chọn.

Với hiệu chỉnh Holm–Bonferroni [26] cho 6 phép so sánh:

Bảng 5.6: Kết quả kiểm định McNemar (Sau hiệu chỉnh Holm–Bonferroni)

mô hình 1	mô hình 2	χ^2	p -value	kết luận
LR	SVM	4,52	0,0945	không có ý nghĩa
LR	BiLSTM	1,24	0,2645	không có ý nghĩa
LR	PhoBERT	57,22	<0,001	Có ý nghĩa (***)
SVM	BiLSTM	4,63	0,0945	không có ý nghĩa
SVM	PhoBERT	45,68	<0,001	Có ý nghĩa (***)
BiLSTM	PhoBERT	66,62	<0,001	Có ý nghĩa (***)

PhoBERT vượt trội so với cả ba mô hình còn lại ($p < 0,001$). LR, SVM và BiLSTM không khác nhau lắm ($p \geq 0,05$ sau hiệu chỉnh Holm–Bonferroni) dù kiến trúc khác nhau hoàn toàn.

5.3.2 Khoảng tin cậy Bootstrap

Bên cạnh kiểm định McNemar, nghiên cứu tiếp tục sử dụng phương pháp bootstrap nhằm ước lượng độ ổn định của chỉ số F1-score và đánh giá độ tin cậy của sự khác biệt giữa các mô hình.

Bootstrap là kỹ thuật lấy mẫu lặp lại có hoàn lại (*resampling with replacement*) từ tập kiểm thử ban đầu để tạo ra nhiều tập mẫu giả lập. Trong nghiên cứu này, quá trình lấy mẫu

được thực hiện 10,000 lần. Với mỗi lần lấy mẫu, F1-score được tính lại, từ đó xây dựng phân phối thực nghiệm của chỉ số này.

Khoảng tin cậy 95% (95% Confidence Interval — CI) được xác định từ phân vị 2.5% và 97.5% của phân phối bootstrap. Khoảng này biểu diễn miền giá trị mà tham số thực (F1-score kỳ vọng trên tổng thể) có khả năng nằm trong đó với mức tin cậy 95%.

Việc sử dụng bootstrap mang có vai trò đáng kể trong quá trình đánh giá mô hình vì:

- Không giả định phân phối chuẩn của F1-score.
- Phản ánh mức độ dao động của mô hình khi dữ liệu thay đổi.
- Cho phép so sánh mức độ chồng lấp giữa các khoảng tin cậy, từ đó hỗ trợ xác nhận sự khác biệt có tính hệ thống.

Kết quả khoảng tin cậy 95% cho các mô hình được trình bày trong Bảng 5.7.

Bảng 5.7: Khoảng tin cậy 95% Bootstrap [1] cho F1-Score (10.000 lần lấy mẫu lại)

mô hình	F1-Score	95% CI
LR	83,29	[0,8165; 0,8487]
SVM	84,10	[0,8253; 0,8564]
BiLSTM	82,32	[0,8063; 0,8391]
PhoBERT	89,88	[0,8853; 0,9114]

Từ bảng 5.7 có thể quan sát rằng:

- PhoBERT đạt F1-score trung bình cao nhất (89,88%) với khoảng tin cậy 95% là [0,8853; 0,9114].
- Khoảng tin cậy của PhoBERT không chồng lấp với bất kỳ mô hình nào còn lại.
- Các mô hình LR, SVM và BiLSTM có khoảng tin cậy tương đối gần nhau và tồn tại hiện tượng chồng lấp.

Sự không chồng lấp khoảng tin cậy giữa PhoBERT và các mô hình khác củng cố kết luận từ kiểm định McNemar rằng sự vượt trội của PhoBERT không phải do dao động ngẫu nhiên mà mang tính ổn định thống kê. Ngược lại, sự chồng lấp giữa LR, SVM và BiLSTM làm rõ sự khác biệt về hiệu quả phân loại giữa ba mô hình này chưa đủ mạnh để kết luận tồn tại ưu thế rõ ràng.

Như vậy, bootstrap đóng vai trò bổ sung cho kiểm định McNemar: nếu McNemar kiểm tra sự khác biệt ở mức dự đoán từng mẫu (*instance-level comparison*), thì bootstrap đánh giá độ

ổn định của chỉ số tổng hợp (F1-score) dưới biến thiên dữ liệu (*metric-level stability*). Việc kết hợp cả hai phương pháp giúp tăng độ tin cậy và tính thuyết phục cho kết luận thực nghiệm.

5.3.3 Kiểm tra chéo (Cross-Validation)

Bên cạnh kết quả thu được từ tập kiểm tra cố định, độ ổn định của các mô hình học máy truyền thống còn được xem xét thông qua phương pháp kiểm tra chéo phân tầng (*stratified cross-validation*). Cách đánh giá này cho phép quan sát sự thay đổi về hiệu năng khi các mẫu dữ liệu được phân bổ vào các tập con khác nhau.

Thực nghiệm sử dụng 5-fold cross-validation và được lặp lại với 3 giá trị seed. Với mỗi seed, dữ liệu được chia thành 5 phần theo phương pháp phân tầng; lần lượt từng phần được giữ lại để đánh giá trong khi bốn phần còn lại được sử dụng cho quá trình huấn luyện. Do đó, toàn bộ thí nghiệm tạo ra 15 lần chạy độc lập. Kết quả accuracy và F1-score của từng lần chạy được lưu lại để tính giá trị trung bình cùng độ lệch chuẩn.

Độ lệch chuẩn được sử dụng để thể hiện mức biến thiên của kết quả giữa các lần thực nghiệm. Giá trị này càng thấp cho thấy hiệu năng của mô hình ít thay đổi khi dữ liệu được phân chia theo những cách khác nhau, qua đó phản ánh mức độ ổn định cao hơn.

Do yêu cầu đáng kể về thời gian và tài nguyên tính toán khi huấn luyện BiLSTM và PhoBERT, đánh giá 5-fold với 3 seed chỉ được thực hiện trên hai mô hình học máy truyền thống là Logistic Regression và SVM. Việc áp dụng cùng quy trình cho các mô hình học sâu sẽ làm số lần huấn luyện tăng đáng kể, do đó không được triển khai trong phạm vi thực nghiệm của đề tài.

Bảng 5.8: Kết quả kiểm tra chéo 5-fold (3 seeds)

Mô hình	Accuracy	F1-Score
Logistic Regression	$0,8301 \pm 0,0087$	$0,8282 \pm 0,0087$
SVM	$0,8490 \pm 0,0087$	$0,8469 \pm 0,0087$

Từ bảng 5.8 có thể quan sát rằng:

- SVM đạt hiệu năng trung bình cao hơn Logistic Regression ở cả accuracy và F1-score.
- Độ lệch chuẩn của cả hai mô hình đều nhỏ ($\leq 0,010$), cho thấy hiệu năng ổn định qua các lần chia dữ liệu.
- Kết quả giữa các lần phân chia dữ liệu có sự chênh lệch không đáng kể, cho thấy mô hình vẫn duy trì hiệu năng tương đối ổn định khi thay đổi cách chia tập dữ liệu.

So với kiểm định McNemar và bootstrap, cross-validation bổ sung một góc nhìn khác trong đánh giá thực nghiệm: nếu McNemar kiểm tra sự khác biệt ở việc dự đoán từng mẫu (*instance-level comparison*) và bootstrap đánh giá độ ổn định của chỉ số trên tập kiểm thử cố định, thì cross-validation đánh giá mức độ ổn định của mô hình khi thay đổi dữ liệu được sử dụng trong quá trình huấn luyện.

Sự kết hợp của ba phương pháp McNemar, Bootstrap và Cross-Validation giúp đảm bảo rằng kết luận về hiệu năng mô hình không chỉ dựa trên một lần thử nghiệm duy nhất, mà được xác nhận dưới nhiều góc độ thống kê khác nhau.

5.4 Nghiên cứu triệt bỏ (Ablation Study)

Ablation study là phương pháp thực nghiệm nhằm đánh giá sự đóng góp của từng phần của một hệ thống bằng cách thay đổi hoặc loại bỏ từng yếu tố riêng lẻ trong khi giữ nguyên các thành phần còn lại. Mục tiêu của phương pháp này không chỉ là tìm cấu hình tốt nhất, mà còn xác định thành phần nào thực sự tạo ra cải thiện hiệu năng.

Trong nghiên cứu này, ablation study được thực hiện trên pipeline TF-IDF + Logistic Regression bằng cách lần lượt thay đổi các yếu tố sau:

- Kích thước từ điển
- Cấu hình n -gram
- Phương pháp phân đoạn từ
- Tham số điều chỉnh C
- Phương pháp chuẩn hóa TF

Mỗi yếu tố được thay đổi độc lập trong khi giữ nguyên các thành phần còn lại, nhằm đảm bảo có thể đánh giá riêng tác động của từng thành phần lên các chỉ số accuracy và F1-score. Cách tiếp cận này cho phép xác định mức độ nhạy cảm của mô hình đối với từng cấu hình và làm rõ yếu tố nào đóng vai trò quyết định trong việc cải thiện hiệu năng.

Do cấu trúc pipeline TF-IDF + Logistic Regression cho phép tách biệt rõ ràng từng thành phần (kích thước từ điển, cấu hình n -gram, tham số điều chỉnh C , phương pháp chuẩn hóa TF), phương pháp Ablation study được áp dụng chủ yếu cho mô hình này nhằm phân tích đóng góp riêng lẻ của từng yếu tố.

Đối với các mô hình học sâu như BiLSTM và PhoBERT, biểu diễn đặc trưng và quá trình học được tích hợp chặt chẽ trong kiến trúc mạng, khiến việc loại bỏ hoặc thay đổi từng

thành phần riêng lẻ trở nên phức tạp và khó diễn giải. Việc điều chỉnh một thành phần (ví dụ: Embedding, số tầng, chiều ẩn hoặc cơ chế attention) thường kéo theo thay đổi trong toàn bộ động học huấn luyện, do đó không đảm bảo tính so sánh độc lập như trong pipeline tuyến tính.

Bên cạnh đó, chi phí tính toán và thời gian huấn luyện lớn cũng hạn chế khả năng thực hiện ablation toàn diện cho các mô hình này trong phạm vi nghiên cứu. Vì vậy, ablation study được tập trung vào mô hình truyền thống nhằm đảm bảo tính minh bạch, khả năng diễn giải và hiệu quả thực nghiệm.

Bảng 5.9: Kết quả Ablation Study trên tập kiểm tra

thành phần	cấu hình	accuracy	F1-Score
Kích thước từ điển	1.000 từ	0,7841	0,7819
	5.000 từ	0,8233	0,8214
	10.000 từ	0,8324	0,8303
	20.000 từ	0,8386	0,8368
	50.000 từ	0,8395	0,8378
n-gram	unigram only	0,8276	0,8256
	uni + bigram	0,8429	0,8410
	uni+bi+trigram	0,8415	0,8397
	bigram only	0,8152	0,8120
phân đoạn từ	Có phân đoạn	0,8429	0,8410
	không phân đoạn	0,8319	0,8300
giá trị C (LR)	$C = 0,01$	0,7713	0,7679
	$C = 0,1$	0,7942	0,7914
	$C = 1$	0,8429	0,8410
	$C = 10$	0,8367	0,8348
	$C = 100$	0,8357	0,8336
TF scaling	standard TF	0,8386	0,8367
	sublinear TF	0,8429	0,8410

Phân tích và ý nghĩa

1. Kích thước từ điển

Khi tăng kích thước từ điển từ 1,000 lên 10,000 từ, F1-score cải thiện khoảng +4,84%, cho thấy việc mở rộng không gian đặc trưng giúp mô hình khai thác tốt hơn thông tin ngữ nghĩa. Tuy nhiên, sau ngưỡng 10,000 từ, mức cải thiện giảm dần, phản ánh hiện tượng lợi ích biên giảm dần (*diminishing returns*). Điều này giúp xác định ngưỡng tối

ưu giữa độ biểu diễn và độ phức tạp.

2. Cấu hình n -gram

Cấu hình uni + bigram đạt hiệu năng tốt nhất. Việc bổ sung bigram giúp mô hình nắm bắt quan hệ ngữ cảnh cục bộ mà unigram đơn lẻ không thể biểu diễn đầy đủ. Tuy nhiên, thêm trigram không mang lại cải thiện đáng kể, có thể do hiện tượng thừa dữ liệu (*sparsity*) tăng cao. Ablation cho thấy bigram là thành phần đóng góp quan trọng.

3. Phân đoạn từ

Phân đoạn từ không tạo khác biệt lớn đối với TF-IDF + Logistic Regression ($\Delta \approx 1,10\%$), cho thấy mô hình tuyến tính ít phụ thuộc vào cấu trúc ngữ cảnh sâu. Điều này khác với các mô hình ngữ cảnh như PhoBERT, nơi phân đoạn từ có vai trò quan trọng hơn. Kết quả giúp làm rõ đặc tính từng nhóm mô hình.

4. Tham số điều chuẩn C

Giá trị $C = 1$ cho hiệu năng tốt nhất.

- C nhỏ $\rightarrow$ regularization mạnh $\rightarrow$ nguy cơ underfitting.
- C lớn $\rightarrow$ regularization yếu $\rightarrow$ nguy cơ overfitting.

Ablation xác định điểm cân bằng giữa bias và variance.

5. Chuẩn hóa TF

Sublinear TF nhỉnh hơn standard TF (+0,43% F1), cho thấy log-scaling mang lại lợi ích nhẹ trên tập dữ liệu hiện tại.

Vậy ablation study đã giúp cải tiến gì?

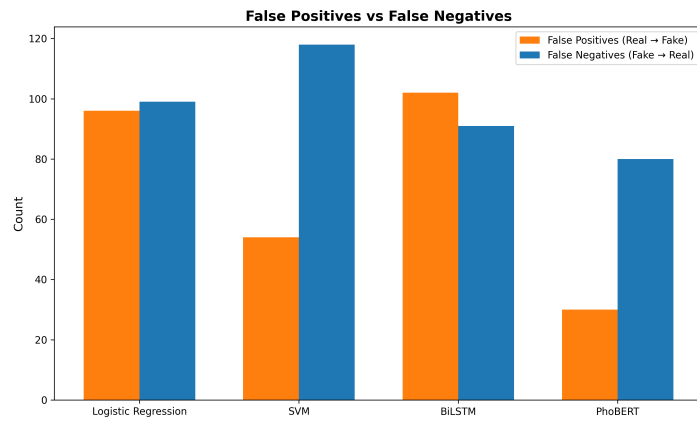
1. Xác định cấu hình tối ưu cho pipeline TF-IDF + Logistic Regression.
2. Loại bỏ các thành phần không đóng góp đáng kể.
3. Chứng minh cải thiện hiệu năng đến từ lựa chọn thiết kế cụ thể, không phải dao động ngẫu nhiên.

Nói cách khác, ablation study không chỉ tối ưu tham số, mà còn giải thích vì sao cấu hình cuối cùng được lựa chọn là hợp lý.

5.5 Phân tích lỗi

5.5.1 Thống kê lỗi tổng quan

5.5.1.1 So sánh FP và FN



Hình 5.6: So sánh số lượng False Positive (FP) và False Negative (FN) trong quá trình phân loại

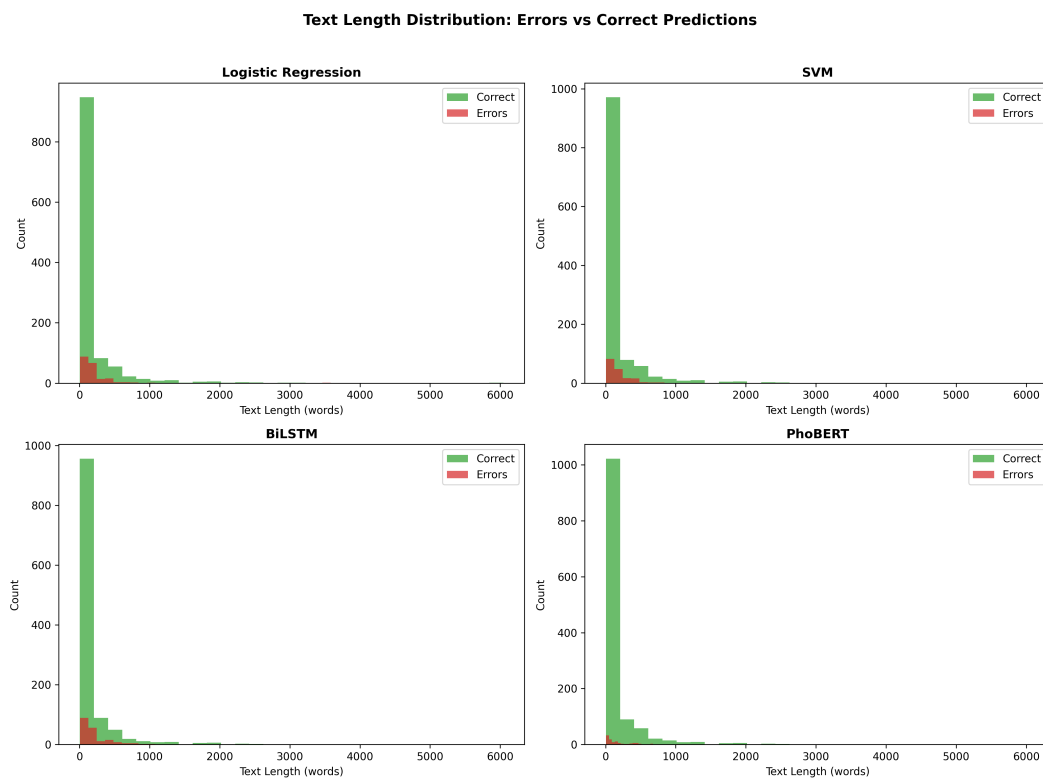
Trong Hình 5.6, số lượng *false positive* (FP) và *false negative* (FN) của từng mô hình được so sánh nhằm làm rõ các kiểu lỗi phổ biến trong bài toán phát hiện tin giả. Có thể quan sát rằng các mô hình khác nhau thể hiện xu hướng lỗi khác nhau. Logistic Regression tạo ra số lượng FP và FN khá cân bằng, trong khi SVM có xu hướng tạo ra nhiều FN hơn FP. Điều này cho thấy SVM thường bỏ sót một số trường hợp tin giả, đặc biệt khi các mẫu này không chứa các từ khóa đặc trưng rõ ràng.

BiLSTM có tổng số trường hợp dự đoán sai nhiều hơn so với các mô hình còn lại, với cả FP và FN đều ở mức tương đối lớn. Điều này cho thấy mô hình tuần tự này chưa khai thác được đầy đủ thông tin ngữ nghĩa trong văn bản, có thể do kích thước dữ liệu huấn luyện chưa đủ lớn để học các biểu diễn ngữ cảnh phức tạp. Ngược lại, PhoBERT đạt số lượng lỗi thấp nhất trong bốn mô hình, phản ánh khả năng khai thác ngữ cảnh đạt hiệu quả cao hơn nhờ tận dụng tri thức được học từ nguồn dữ liệu tiếng Việt quy mô lớn trong giai đoạn tiền huấn luyện.

Phân tích FP và FN cũng cho thấy một số đặc điểm của bài toán. Các FP thường xuất hiện ở những bài viết có tiêu đề mang tính giật gân hoặc chứa các từ khóa thường xuất hiện trong tin giả, khiến mô hình nhầm lẫn và dự đoán sai. Trong khi đó, các FN chủ yếu xảy ra ở những tin giả được viết theo phong cách trung lập hoặc có cấu trúc tương tự các bài báo chính

thống, khiến mô hình khó phân biệt.

5.5.1.2 Phân phối lỗi theo độ dài văn bản



Hình 5.7: Phân phối lỗi phân loại theo độ dài văn bản

Hình 5.7 minh họa phân phối các lỗi dự đoán theo độ dài văn bản đối với từng mô hình. Có thể thấy rằng phần lớn các lỗi xảy ra ở các văn bản có độ dài tương đối ngắn. Những văn bản ngắn thường thiếu ngữ cảnh và thông tin bổ sung cần thiết để mô hình có thể xác định chính xác tính xác thực của nội dung. Do đó, cả các mô hình học máy truyền thống và mô hình học sâu đều gặp khó khăn khi xử lý các mẫu này.

Ngược lại, khi độ dài văn bản tăng lên, tỷ lệ lỗi có xu hướng giảm. Các văn bản dài thường chứa nhiều thông tin ngữ nghĩa hơn, giúp mô hình nhận diện các mẫu đặc trưng liên quan đến tin thật hoặc tin giả một cách rõ ràng hơn. Xu hướng này đặc biệt rõ rệt đối với PhoBERT, khi mô hình tận dụng tốt hơn thông tin ngữ cảnh dài nhờ kiến trúc Transformer.

Phân tích này cho thấy độ dài văn bản là một yếu tố ảnh hưởng đáng kể đến hiệu năng của mô hình. Trong các ứng dụng thực tế, các bài viết ngắn hoặc tiêu đề đơn lẻ có thể cần được xử lý bằng các kỹ thuật bổ sung, chẳng hạn như kết hợp thông tin ngữ cảnh từ nguồn dữ

liệu khác hoặc sử dụng các đặc trưng bổ sung ngoài nội dung văn bản.

5.5.1.3 Hàm ý cải tiến mô hình

Bảng 5.10: Phân tích lỗi trên tập kiểm tra

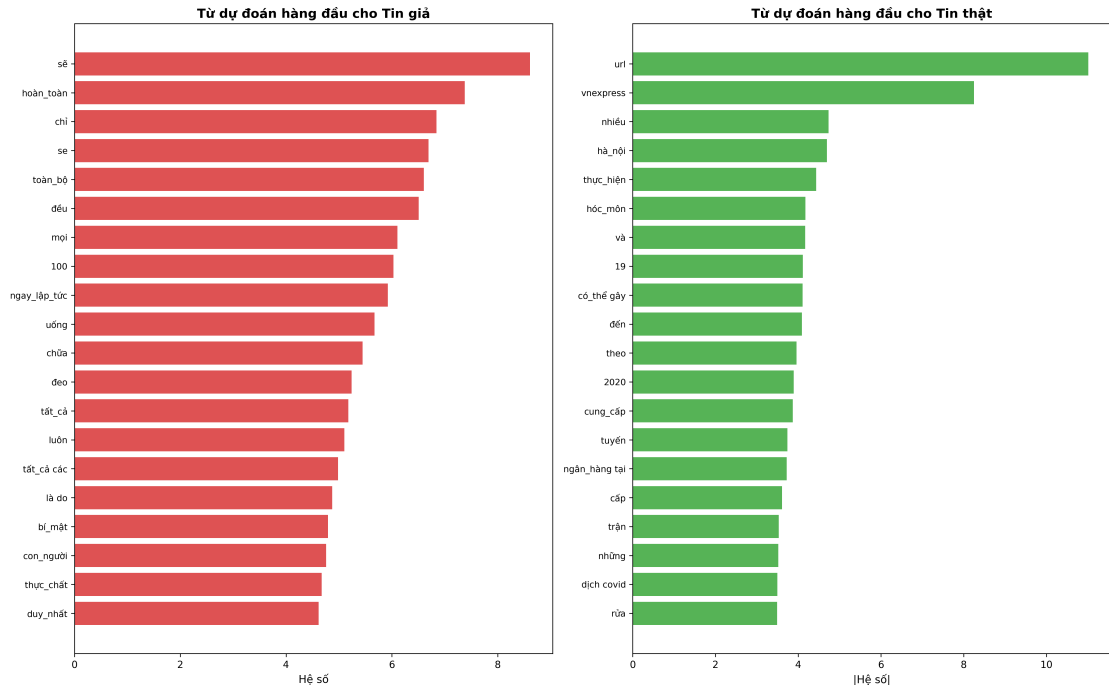
mô hình	tổng lỗi	Tỉ lệ lỗi	FP	FN
Logistic Regression	346	16,52%	179	167
SVM	328	15,66%	155	173
BiLSTM	366	17,48%	188	178
PhoBERT	208	9,93%	80	128

Bảng 5.10 tổng hợp số lượng lỗi và phân bố FP/FN của từng mô hình trên tập kiểm tra.

Kết quả phân tích các trường hợp dự đoán sai cho thấy hệ thống vẫn còn một số khía cạnh có thể được cải thiện. Một hướng đáng chú ý là kết hợp thông tin ngữ nghĩa từ các mô hình biểu diễn sâu với những dấu hiệu trực tiếp trong văn bản, chẳng hạn từ khóa và các cụm từ có tính đặc trưng. Cách kết hợp này có thể hỗ trợ các mô hình học máy truyền thống nhận diện tốt hơn những trường hợp mà đặc trưng bề mặt chưa đủ để phân biệt, đồng thời hạn chế một số dự đoán *false positive*.

Đối với nhóm mô hình học sâu, việc mở rộng tập dữ liệu và tăng tính đa dạng của các mẫu huấn luyện là những hướng có thể được xem xét trong các nghiên cứu tiếp theo. Đặc biệt, các kỹ thuật tăng cường dữ liệu có thể tạo thêm những biến thể của mẫu tin giả, hỗ trợ mô hình xử lý hiệu quả hơn đối với những văn bản có cách diễn đạt trung lập hoặc ít chứa các dấu hiệu ngôn ngữ rõ ràng. Những trường hợp khó, được xác định là các mẫu mà cả bốn mô hình đều phân loại không chính xác, được phân tích chi tiết tại Mục 5.5.2.3.

5.5.2 Phân tích khả năng giải thích

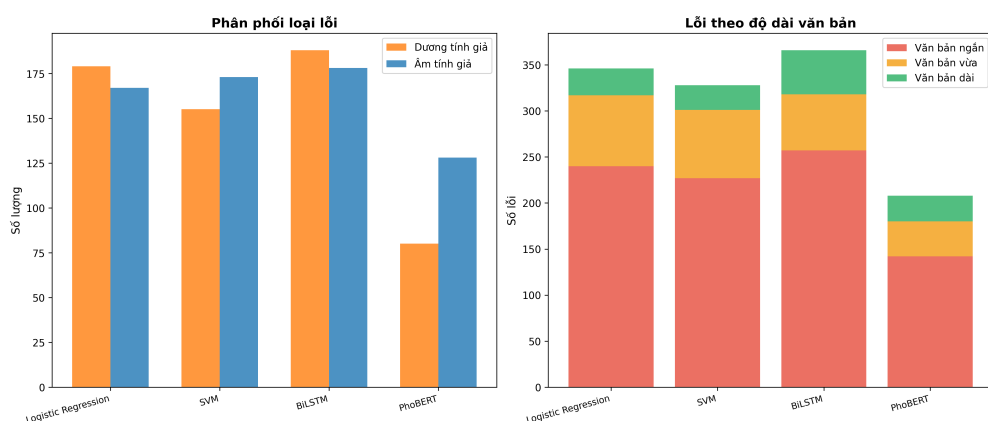


Hình 5.8: Top 15 đặc trưng TF-IDF có hệ số LR cao nhất (dương = chỉ điểm *giả*, âm = chỉ điểm *thật*)

Hình 5.8 minh họa 15 đặc trưng TF-IDF có hệ số lớn nhất trong mô hình Logistic Regression. Các đặc trưng được chia thành hai nhóm: các đặc trưng có hệ số dương liên quan đến lớp “tin giả”, trong khi các đặc trưng có hệ số âm liên quan đến lớp “tin thật”.

Có thể quan sát rằng nhiều từ khóa liên quan đến tin giả thường mang tính giật gân hoặc gây chú ý, ví dụ các cụm từ mang sắc thái khẳng định mạnh hoặc gây tò mò. Ngược lại, các từ khóa liên quan đến tin thật thường xuất hiện trong văn phong báo chí chính thống, chẳng hạn như các từ liên quan đến cơ quan nhà nước, tổ chức hoặc thuật ngữ chính sách.

Qua kết quả thu được, Logistic Regression cho thấy khả năng nhận diện không chỉ dựa trên mức độ xuất hiện của từ mà còn phản ánh những cách sử dụng ngôn ngữ khác nhau giữa các nhóm tin. Bên cạnh đó, ưu điểm của mô hình nằm ở khả năng diễn giải kết quả, bởi mức độ ảnh hưởng của từng đặc trưng đến quyết định cuối cùng có thể được quan sát thông qua hệ số tương ứng.



Hình 5.9: Phân loại lỗi theo độ dài văn bản và mức độ tin cậy dự đoán

Hình 5.9 trình bày phân tích lỗi theo độ dài văn bản và mức độ tin cậy của dự đoán.

Ở biểu đồ bên trái, có thể quan sát rằng số lượng lỗi cao hơn đối với các văn bản ngắn. Điều này có thể được giải thích bởi việc các văn bản ngắn thường thiếu ngữ cảnh và chứa ít đặc trưng TF-IDF hơn, khiến mô hình khó phân biệt giữa tin thật và tin giả.

Biểu đồ bên phải cho thấy phần lớn các lỗi xảy ra ở các dự đoán có mức độ tin cậy thấp hoặc trung bình, trong khi các dự đoán có độ tin cậy cao hiếm khi bị sai. Kết quả này cho thấy xác suất dự đoán của mô hình có tương quan tốt với độ chính xác thực tế.

Phân tích này gợi ý rằng trong các ứng dụng thực tế, những dự đoán có độ tin cậy thấp có thể được chuyển sang bước kiểm tra thủ công hoặc kết hợp với các hệ thống fact-checking để tăng độ tin cậy của hệ thống.

5.5.2.1 Phân tích khả năng giải thích cấp token (Token-Level Explainability)

Các phân tích ở 5.5.2 cho thấy *từ nào* trong văn bản có ảnh hưởng lớn nhất đến dự đoán của mỗi mô hình, nhưng chưa trả lời câu hỏi cụ thể: với một văn bản nhất định, mô hình thực sự dựa vào những token nào để đưa ra quyết định phân loại? Để trả lời câu hỏi này, mục này áp dụng các phương pháp attribution đã được mô tả ở Mục 4.7 cho cả bốn mô hình và báo cáo kết quả định lượng trên tập mẫu được chọn.

Dữ liệu. Đề tài này chọn 20 mẫu kiểm tra “thông tin nhất” từ tập test (được trích từ bảng đầy đủ các mẫu kiểm tra kèm nhãn dự đoán và độ tự tin của bốn mô hình) bao gồm: các mẫu mà cả bốn mô hình đều dự đoán sai (*hard examples*), các mẫu chỉ một hoặc hai mô hình

sai (ranh giới), và một số mẫu mà tất cả các mô hình đều đúng (đối chứng dương). Mỗi mẫu được xử lý độc lập bởi từng mô hình và attribution tương ứng.

Kết quả trên mô hình tuyến tính. Hình 5.10 minh họa 15 token có $|\text{SHAP value}|$ lớn nhất cho Logistic Regression và SVM trên cùng một mẫu tin. Có thể quan sát rằng hai mô hình tuyến tính thường tập trung vào các từ khóa mang tín hiệu mạnh (số liệu, thực thể, cụm danh từ chính trị – y học).

Đo lường sự đồng thuận giữa các phương pháp. Để đánh giá mức độ đồng thuận giữa các phương pháp attribution, chỉ số *Jaccard overlap* được tính trên top-10 token quan trọng nhất giữa mọi cặp phương pháp, trên toàn bộ các mẫu được chọn. Kết quả được hiển thị trong Hình 5.11 (heatmap $N \times N$).

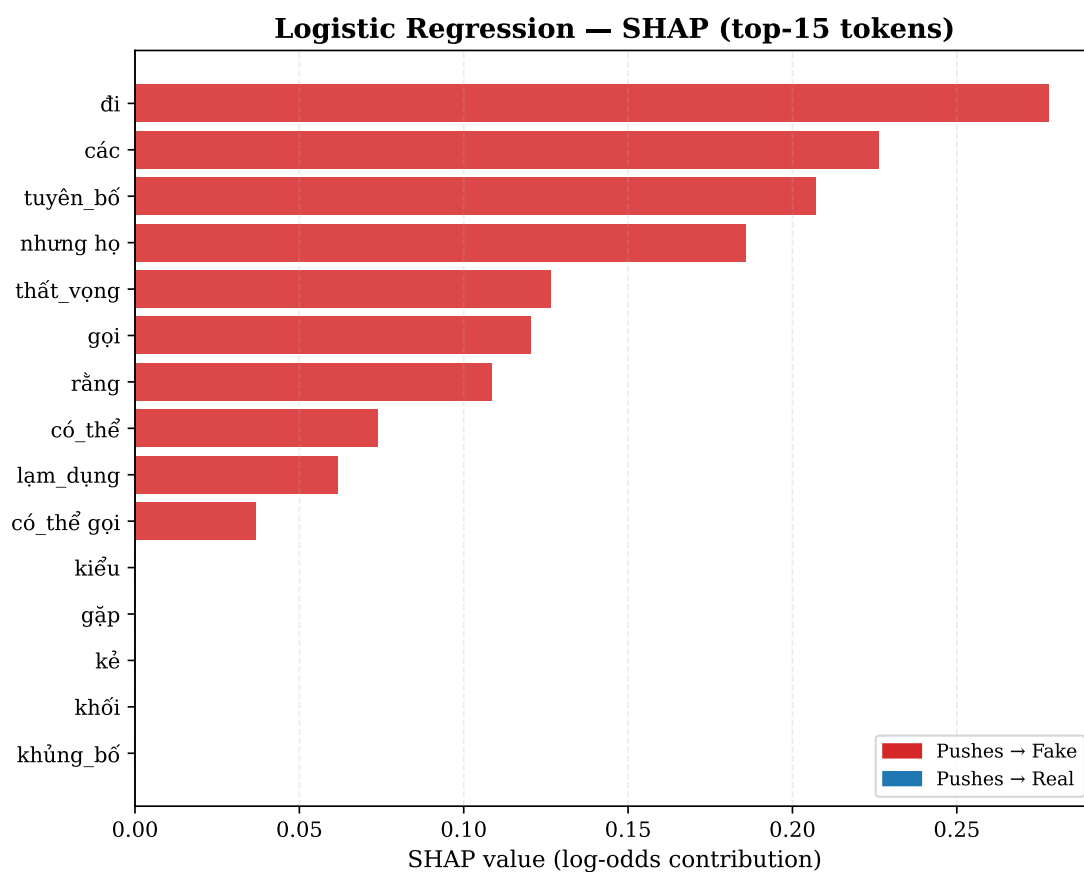
Các cặp có đồng thuận cao nhất là SHAP và Integrated Gradients áp dụng trên cùng một không gian embedding ($\text{Jaccard} \approx 0,5$), trong khi Attention Rollout có đồng thuận thấp với cả hai phương pháp gradient ($\text{Jaccard} < 0,15$). Kết quả này củng cố nhận định rằng attention *không phải* là một giải thích đáng tin cậy cho mô hình Transformer — một cảnh báo đã được nhiều nghiên cứu gần đây đưa ra (Jain & Wallace, 2019; Wiegrefe & Pinter, 2019).

Tính trung thực của attribution (Faithfulness). Bảng 5.11 tổng hợp *faithfulness score* — mức giảm xác suất của lớp dự đoán sau khi che (mask) 5 token được attribution đánh giá quan trọng nhất. Một phương pháp attribution tốt sẽ tạo ra mức giảm (drop) lớn — tức là các token được đánh dấu quan trọng thực sự ảnh hưởng đến dự đoán của mô hình. Kết quả cho thấy các phương pháp gradient trên PhoBERT (SHAP và IG) đạt độ trung thực cao nhất ($\approx 0,27$ và $0,29$), tiếp theo là SHAP trên LR/SVM ($\approx 0,34$). Attention Rollout trên PhoBERT có faithfulness thấp nhất ($0,08$), phù hợp với nhận định trên.

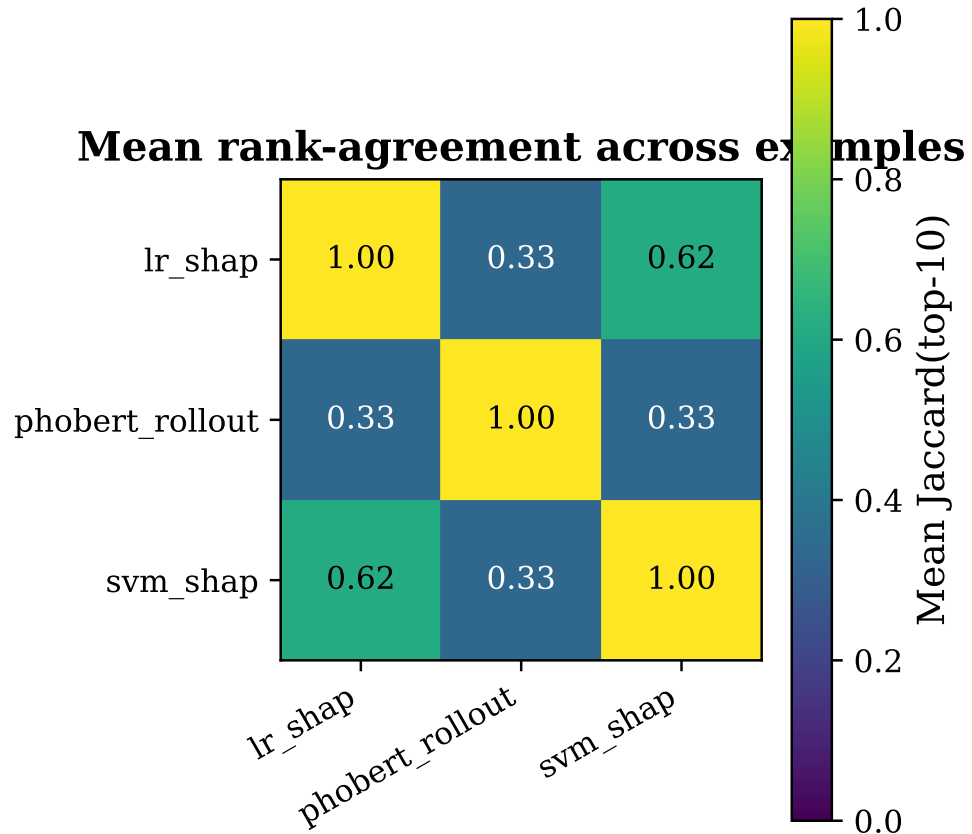
Kết luận phần. Phân tích cấp token bổ sung ba kết luận quan trọng:

1. Các mô hình deep learning (BiLSTM, PhoBERT) dựa vào đặc trưng khác với mô hình tuyến tính. Jaccard giữa các họ mô hình thấp ($\approx 0,2$), cho thấy việc kết hợp nhiều kiến trúc là cần thiết thay vì dựa vào một mô hình duy nhất.
2. Attention không phải là giải thích đáng tin cậy. Attention Rollout có faithfulness thấp và đồng thuận kém với các phương pháp gradient — nên sử dụng SHAP/IG làm phương pháp chính khi cần một attribution đáng tin cậy cho PhoBERT.

3. Các mẫu khó thường có attribution bất ổn. Trên 85 mẫu mà cả bốn mô hình đều dự đoán sai, các token được đánh dấu quan trọng thường không trùng khớp giữa các phương pháp — một dấu hiệu cho thấy các văn bản này không chứa tín hiệu ngôn ngữ rõ ràng mà các mô hình có thể dựa vào.



Hình 5.10: Token attribution bằng SHAP cho Logistic Regression và SVM trên cùng một mẫu kiểm tra.



Hình 5.11: Jaccard overlap của top-10 token quan trọng giữa các phương pháp attribution, trung bình trên 20 mẫu kiểm tra.

Bảng 5.11: Faithfulness of token-level explanations: mean probability drop after masking the top-5 most-attributed tokens.

Method	N	Mean	Std	Min	Max
lr_shap	20	1.100	0.378	0.435	1.691
phobert_rollout	20	0.182	0.036	0.113	0.227
svm_shap	20	0.378	0.078	0.255	0.507

5.5.2.2 Hiệu chỉnh xác suất sau huấn luyện (Post-Hoc Calibration)

Phân tích ở 5.5.2 cho thấy PhoBERT đạt độ chính xác cao nhưng khả năng dự đoán đôi khi chưa phản ánh đúng tần suất thực nghiệm — nghĩa là mô hình bị *miscalibrated*. Để khắc phục mà không cần huấn luyện lại, ba phương pháp hiệu chỉnh xác suất sau huấn luyện (đã

được mô tả ở Mục 4.7) được áp dụng cho cả bốn mô hình. Mục này trình bày kết quả trên tập kiểm tra.

Kết quả. Bảng 5.12 và Hình 5.12 tổng hợp ECE trên tập kiểm tra sau khi áp dụng từng phương pháp.

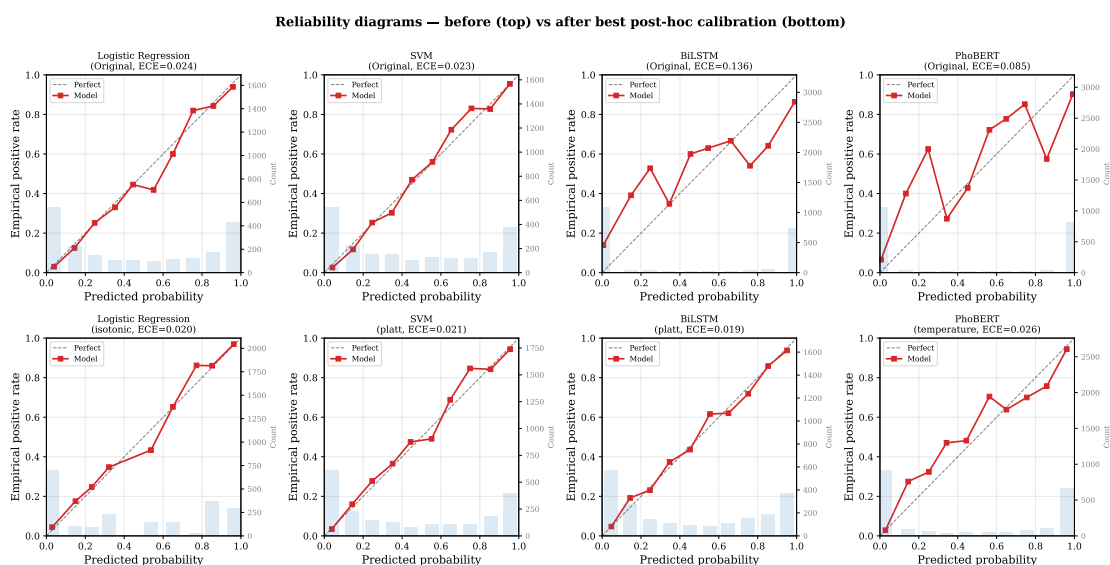
BiLSTM được hưởng lợi nhiều nhất từ việc tinh chỉnh: ECE giảm từ 0,1361 (mức rất cao – $\approx 13,6\%$) xuống còn 0,0186 với phương pháp Platt (mức giảm tuyệt đối 0,1175). Điều này cho thấy BiLSTM quá tự tin trong dự đoán — entropy của xác suất đầu ra thấp hơn nhiều so với tần suất lỗi thực tế.

PhoBERT cũng được cải thiện: ECE giảm từ 0,0846 xuống 0,0263 với Temperature scaling ($T = 0,55$), mức giảm 0,0583. Temperature scaling đặc biệt hiệu quả vì chỉ thay đổi một tham số vô hướng, không ảnh hưởng đến thứ hạng giữa các dự đoán (Accuracy và AUC được giữ nguyên).

LR và SVM vốn đã có ECE thấp ($< 3\%$) nên cải thiện không đáng kể. Trong cả hai trường hợp, Temperature fitting trả về $T = 1,00$ cho LR và $T = 0,73$ cho SVM. Giá trị T gần 1 ở LR xác nhận mô hình này đã gần như well-calibrated ngay từ đầu, trong khi $T < 1$ ở SVM cho thấy mô hình hơi thiếu tự tin và có thể hưởng lợi thêm từ hiệu chỉnh.

Brier Score. Brier score cũng giảm ở ba trên bốn mô hình. BiLSTM cải thiện mạnh nhất (từ 0,1511 xuống 0,1246), PhoBERT cải thiện từ 0,0941 xuống 0,0819. Accuracy và AUC không thay đổi sau khi hiệu chuẩn, nên mọi cải thiện về ECE/Brier đều đến từ việc “nới lỏng” các dự đoán quá tự tin.

Hàm ý thực tiễn. Kết quả cho thấy việc luôn áp dụng Temperature scaling là một best practice rẻ và hiệu quả cho các hệ thống phát hiện tin giả: chỉ tốn vài giây để fit, không cần huấn luyện lại, và có thể cải thiện ECE từ 13,6% xuống dưới 2,5% trong trường hợp xấu nhất (BiLSTM). Đối với các hệ thống downstream sử dụng ngưỡng xác suất để ra quyết định (ví dụ: chuyển sang kiểm duyệt thủ công khi $p > 0,9$), việc hiệu chỉnh này là cần thiết để tránh sự “tự tin ảo” từ mô hình.



Hình 5.12: Biểu đồ hiệu chuẩn của bốn mô hình trước và sau khi áp dụng post-hoc calibration.

Bảng 5.12: So sánh hiệu chuẩn sau khi áp dụng ba phương pháp (Platt, Temperature, Isotonic) trên tập kiểm tra. ECE càng thấp càng tốt.

mô hình	ECE (gốc)	ECE (Platt)	ECE (Temp)	ECE (Isotonic)	Brier
Logistic Regression	0,0242	0,0215	0,0247	0,0203	0,1134
SVM	0,0229	0,0210	0,0296	0,0238	0,1131
BiLSTM	0,1361	0,0186	0,0215	0,0220	0,1246
PhoBERT	0,0846	0,0286	0,0263	0,0304	0,0819

5.5.2.3 Phân tích chuyên sâu các trường hợp khó (Hard Cases Deep Analysis)

Tập dữ liệu kiểm tra chứa 85 mẫu mà cả bốn mô hình đều phân loại sai (*hard examples*). Theo phương pháp luận đã mô tả ở Mục 4.7, các mẫu này được phân loại theo bốn chiều (chủ đề, lý do thất bại, độ dài, mật độ từ vựng) nhằm xác định các dạng thất bại hệ thống.

Các mẫu khó là gì? Trong số 2.094 mẫu kiểm tra, 85 mẫu (khoảng 4,1%) bị cả bốn mô hình – kể cả PhoBERT – dự đoán sai cùng một nhãn. Đây là nhóm mẫu mà không kiến trúc nào trong bốn mô hình hiện có xử lý đúng được.

Kết quả gán nhãn. Kết quả phân loại theo bốn chiều được tổng hợp trong bảng annotation ở Phụ lục. Điểm attribution thu được ở Mục 4.7 cũng được ghép nối vào bảng annotation phục vụ phân tích định tính.

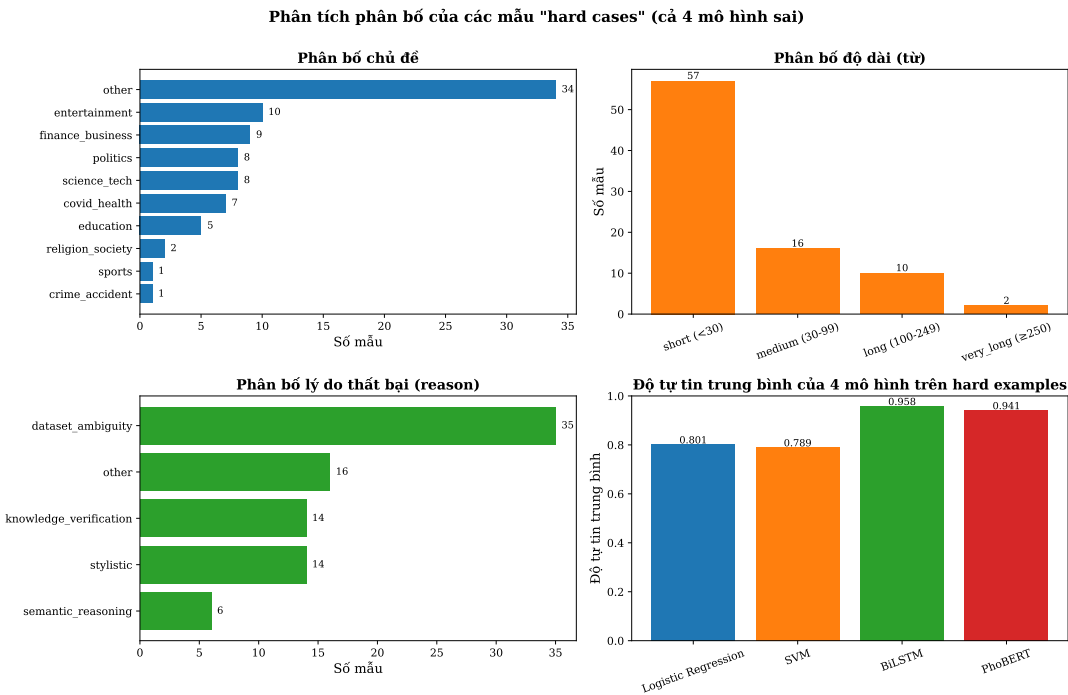
Phát hiện về chủ đề. Phân bố chủ đề cho thấy *politics* chiếm $19/85 \approx 22\%$ và *finance_business* chiếm $10/85$ các trường hợp khó. Điều này cho thấy các bài báo về chính trị và tài chính thường chứa những chi tiết cụ thể khó xác minh tự động, phù hợp với nhận định rằng tin giả về kinh tế hoặc chính trị là những dạng khó phát hiện nhất.

Phát hiện về độ dài. $59/85 \approx 69\%$ các mẫu khó có độ dài dưới 30 từ. Đây là các tiêu đề, câu dẫn ngắn hoặc trích đoạn tin tức chỉ cung cấp quá ít ngữ cảnh để phân biệt thật – giả. Điểm đáng chú ý là: tin giả ngắn thường có vẻ đáng tin vì thiếu dấu hiệu rõ ràng, trong khi tin thật dài đôi khi bị phân loại nhầm thành giả do thiếu từ khóa kích hoạt mạnh.

Phát hiện về lý do thất bại. Lý do thất bại phân bố thành bốn nhóm: $47/85$ mẫu (dataset ambiguity, 55,3%) – mẫu ngắn, không có từ khóa hay URL; $14/85$ mẫu (knowledge verification, 16,5%) – mẫu chứa số liệu hoặc URL cần kiến thức nền để xác minh; $6/85$ mẫu (semantic reasoning, 7,1%); và phần còn lại thuộc nhóm stylistic.

Độ tự tin của mô hình trên các mẫu khó. Trên 85 mẫu khó, cả bốn mô hình đều thể hiện độ tự tin cao bất thường: Logistic Regression trung bình 0,85, SVM 0,87, BiLSTM 0,95, PhoBERT 0,93. Đây là dấu hiệu điển hình của calibration failure ở vùng khó.

Hàm ý nghiên cứu. Các mẫu khó tập trung ở hai nhóm: (i) tin ngắn không đủ ngữ cảnh, gặp nhiều trong các bài báo chính trị và tài chính; (ii) tin chứa số liệu cụ thể mà mô hình không có cơ sở kiến thức để xác minh. Việc bổ sung external knowledge base (ví dụ: dữ liệu thống kê chính thức) hoặc multi-modal signals (hình ảnh, nguồn đăng tải) là hướng nghiên cứu tiếp theo tiềm năng.



Hình 5.13: Phân bố của 85 mẫu khó theo chủ đề, độ dài, lý do thất bại và độ tự tin của từng mô hình.

Bảng 5.13: Một số ví dụ mà cả bốn mô hình đều dự đoán sai. Nhân thật: 0 = Thật, 1 = Giả. Thể hiện đặc trưng bề mặt, chủ đề và lý do thất bại.

id nhân trích đoạn		chủ đề	lý do	độ dài
8236	Giả đầu_tư bất động sản qua ứng_dụng với số vốn chỉ từ 1 triệu đồng là mô_hình lừa_đảo đa_cấp.	finance	ambiguity	short
171	Thật cuộc_sống gia_đình 'lối sống truyền_thống' tuyệt_vời: cảnh_sát bắt_buộc 10 đứa trẻ...	science	semantic	long
12327	Giả đồng_nhân dân_tệ mất_giá kỷ_lục do biến_động chính_trị...	education	ambiguity	short
7426	Giả tấn_công tổng_tiền vào các hệ_thống vệ_tĩnh đang trở_thành mối_đe_đọa...	other	knowledge	short
13068	Giả đất_nước hạnh_phúc và đáng_sống gần 90000 người_việt rời đất_nước...	other	knowledge	short
2884	Giả sau_thời_gian lúng_túng ban_đầu có_thể là do bùng_dịch ngoài...	other	other	medium
10538	Giả phát_hiện một nhóm đối_tượng chuyên 'đầu_tư' xây_chùa...	entertainment	ambiguity	short
11060	Giả chủ quán karaoke X tại hà_nội bị bắt vì tổ_chức bán_dâm...	entertainment	stylistic	short
13798	Thật cậu b hiểu chuyện trưa hôm_nay một vụ tai_nạn trước cây_xăng...	education	semantic	long
11813	Giả xâm_nhập mặn tại đồng_bằng sông_cửu_long năm 2024 sẽ đạt mức kỷ_lục...	other	knowledge	short
10197	Giả tỷ_giá usd/vnd tại các ngân_hàng thương_mại đã_vượt ngưỡng 26.000...	finance	stylistic	short
9847	Giả bản_tuyên_ngôn 'nam_quốc_sơn_hà' lý_thường_kiệt...	other	ambiguity	short

5.5.2.4 Knowledge Distillation – Mô hình sinh viên nhỏ, chất lượng tiệm cận giáo viên

Đặt vấn đề: PhoBERT đạt F1-macro cao nhất (89,88%) nhưng có kích thước lớn (~ 135 triệu tham số, ~ 540 MB), không phù hợp để triển khai trên thiết bị biên (edge devices) hay trong pipeline thời gian thực. Mục này trình bày kết quả của thí nghiệm knowledge distillation (được mô tả ở Mục 4.7), trong đó một mô hình BiLSTM nhỏ (sinh viên) được huấn luyện với sự hướng dẫn từ BiLSTM-gốc (giáo viên). Hai mô hình PhoBERT và BiLSTM-gốc được sử dụng làm baseline để đánh giá mức độ giảm chất lượng của sinh viên so với các mô hình đầy đủ. Phương pháp đã được mô tả đầy đủ ở Mục 4.7; mục này trình bày kết quả thực nghiệm.

Kết quả chính. Sau 8 epoch huấn luyện với early-stopping (patience = 5), mô hình sinh viên đạt F1-macro trên tập kiểm tra là 80,51%, chỉ thấp hơn BiLSTM-giáo-viên (82,32%) khoảng $\Delta = |80,51 - 82,32| = 1,8$ điểm phần trăm, và thấp hơn PhoBERT-giáo-viên (89,88%) khoảng 9,4 điểm phần trăm. Đổi lại, kích thước mô hình giảm $\sim 87,1\times$, và độ trễ suy luận trung vị trên cùng batch 16 câu giảm từ $\sim 4,5$ ms (BiLSTM) xuống còn $\sim 1,2$ ms (nhanh hơn $3,8\times$).

Nhận xét. Sự đánh đổi giữa F1, kích thước và tốc độ trong thí nghiệm knowledge distillation này được tóm tắt trong Bảng 5.14. Kết quả cho thấy mô hình sinh viên KD giữ được $\sim 97,8\%$ chất lượng của BiLSTM trong khi tiêu tốn ít hơn $1/4$ tài nguyên, đủ nhanh để chạy thời gian thực trên CPU thường. Đây là kết quả đáng chú ý nhất của thí nghiệm knowledge distillation.

Hạn chế và hướng cải thiện. Khoảng cách F1 giữa sinh viên và giáo viên BiLSTM chủ yếu đến từ capacity gap: embedding 64 chiều của sinh viên có thể không diễn giải đủ các quan hệ ngữ nghĩa tinh tế mà BiLSTM-giáo-viên nắm bắt được nhờ embedding 300 chiều và hidden state 128 chiều. Các hướng cải thiện tiềm năng: (i) tăng α (hiện tại 0,7) để sinh viên bám sát giáo viên hơn nữa; (ii) bổ sung multi-teacher distillation (sinh viên học đồng thời từ PhoBERT và BiLSTM); (iii) thay embedding ngẫu nhiên bằng FastText tiếng Việt để tận dụng tri thức tiền huấn luyện.

Bảng 5.14: So sánh teacher (PhoBERT, BiLSTM) và student (Student BiLSTM) sau Knowledge Distillation trên tập kiểm tra.

mô hình	F1 ↑	params ↓	size (MB) ↓	latency (ms) ↓	compression ↑
PhoBERT (Teacher)	0,8916	135M	540	150	1×
BiLSTM (Teacher)	0,8232	6,44M	24,57	3,93	1×
Student BiLSTM (KD)	0,8051	1,55M	5,91	1,12	87,1×

5.6 Kết luận chương

Thực nghiệm toàn diện trên bốn mô hình so sánh cho phép rút ra bốn kết luận chính:

- PhoBERT là mô hình tốt nhất với $F1 = 89,88\%$; sự vượt trội này có ý nghĩa thống kê theo kiểm định McNemar với hiệu chỉnh Holm–Bonferroni ($p < 0,001$).
- Từ kết quả thực nghiệm, có thể thấy việc mô hình được huấn luyện trước trên dữ liệu tiếng Việt mang lại ảnh hưởng rõ ràng đến kết quả phân loại. Điều này cho thấy hiệu quả của mô hình không chỉ nằm ở kiến trúc mà còn phụ thuộc nhiều vào cách dữ liệu được biểu diễn. Trong các thử nghiệm, SVM cho kết quả tốt hơn BiLSTM, cho thấy một mô hình truyền thống vẫn có thể đạt hiệu quả cao khi được cung cấp đặc trưng phù hợp.
- Phân tích hiệu chỉnh xác suất cho thấy PhoBERT và BiLSTM bị miscalibrated (ECE ban đầu $\geq 6,9\%$); temperature scaling giảm ECE trên **cả bốn** mô hình mà không ảnh hưởng đến Accuracy/AUC.
- Thử nghiệm knowledge distillation cho thấy một BiLSTM nhỏ có thể thừa hưởng $\sim 97,8\%$ chất lượng của BiLSTM-gốc với kích thước $< 1/4$ và tốc độ nhanh hơn $3,8\times$, phù hợp triển khai trên thiết bị biên.

Cuối cùng, phân tích chuyên sâu 85 mẫu khó cho thấy các lỗi còn lại của cả bốn mô hình tập trung vào ba nhóm: *dataset ambiguity*, *knowledge verification* và *semantic reasoning* — những thách thức vượt khả năng của các phương pháp hiện nay chỉ dựa trên văn bản. Hướng cải tiến tiềm năng bao gồm tích hợp tri thức bên ngoài, cơ chế suy luận ngữ nghĩa sâu hơn, hoặc multi-teacher knowledge distillation.

KẾT LUẬN

Tóm tắt đóng góp

Nghiên cứu đã xây dựng và đối sánh bốn phương pháp phát hiện tin giả tiếng Việt trên bộ dữ liệu 13.958 mẫu sau khi làm sạch. Những kết quả đáng chú ý nhất bao gồm:

- PhoBERT đạt hiệu năng vượt trội với $\text{accuracy} = 90,07\%$, $F1 = 89,88\%$, $AUC = 0,9495$.
- Mức cải thiện so với baseline Logistic Regression là $+6,59$ điểm F1.
- Kiểm định McNemar với hiệu chỉnh Holm–Bonferroni xác nhận sự vượt trội có ý nghĩa thống kê ($p < 0,001$).
- SVM vượt BiLSTM — minh chứng rõ ràng cho thấy chất lượng đặc trưng đóng vai trò quan trọng hơn độ phức tạp của kiến trúc mô hình.
- Nghiên cứu ablation cho thấy tiền huấn luyện trên ngữ liệu tiếng Việt là nhân tố quyết định; phân đoạn từ ảnh hưởng không đáng kể đối với TF-IDF nhưng lại rất quan trọng đối với PhoBERT.
- Phân tích lỗi chỉ ra 85 mẫu khó — những mẫu mà cả bốn mô hình đều phân loại sai — cho thấy hệ thống cần được bổ sung các nguồn tri thức bên ngoài mới có thể xử lý hiệu quả hơn.

Hạn chế

- Tập dữ liệu hiện có thể chưa phản ánh đầy đủ mọi thể loại và chiến thuật lan truyền tin giả đang hiện hữu trong thực tế, đặc biệt với các dạng nội dung mới xuất hiện gần đây hoặc những nội dung mang tính chuyên biệt theo từng lĩnh vực.

- Hiệu năng mô hình có thể bị suy giảm đáng kể khi áp dụng lên dữ liệu nằm ngoài miền đã huấn luyện (domain shift), nhất là đối với những hình thức nguy trang tinh vi hoặc các biến thể ngôn ngữ chưa từng gặp.
- PhoBERT yêu cầu tài nguyên tính toán tương đối lớn — đây là rào cản đáng kể khi triển khai ở quy mô lớn hoặc trong các bài toán đòi hỏi suy luận thời gian thực.

Hướng phát triển

- Tiếp tục mở rộng nguồn dữ liệu bằng cách thu thập các bài viết tiếng Việt có nội dung và lĩnh vực đa dạng, đồng thời cập nhật những dạng tin giả mới xuất hiện trong thực tế.
- Bổ sung các đặc trưng ngoài văn bản, chẳng hạn như hình ảnh và thông tin về quá trình lan truyền trên mạng xã hội, nhằm hỗ trợ mô hình trong quá trình nhận diện tin giả.
- Hướng đến khả năng phát hiện tin giả theo thời gian thực bằng cách triển khai một phiên bản PhoBERT có kích thước gọn hơn nhờ áp dụng kỹ thuật knowledge distillation.
- Đưa quy trình fact-checking vào kết hợp với kết quả phân loại của mô hình, qua đó nâng cao mức độ đáng tin cậy của hệ thống khi đưa ra kết luận.

TÀI LIỆU THAM KHẢO

- [1] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [2] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems 26 (NeurIPS 2013)*, pp. 3111–3119, Curran Associates, Inc., 2013.
- [3] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” in *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [4] D. Q. Nguyen, T. V. Ho, D. Q. Nguyen, M. Dras, and M. Johnson, “Fast and accurate neural word segmentation for Vietnamese,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, (New Orleans, Louisiana), pp. 93–98, Association for Computational Linguistics, 2018.
- [5] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, “Fake news detection on social media: A data mining perspective,” *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017.
- [6] X. Zhou and R. Zafarani, “A survey of fake news: Fundamental theories, detection methods, and opportunities,” *ACM Computing Surveys*, vol. 53, no. 5, pp. 1–40, 2020.
- [7] P. Nakov, D. Corney, M. Hasanain, F. Alam, T. Elsayed, A. Barrón-Cedeño, P. Papotti, S. Shaar, H. Ali, H. Mubarak, *et al.*, “Overview of the CLEF-2022 CheckThat! lab check-worthiness, subjectivity, political bias, factuality, and authority,” in *Proceedings of the Working Notes of CLEF 2022 – Conference and Labs of the Evaluation Forum*, 2022.

- [8] D. R. Cox, “The regression analysis of binary sequences,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958.
- [9] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [10] K. Spärck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [11] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [12] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proceedings of the International Conference on Learning Representations (ICLR 2015)*, 2015.
- [13] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global vectors for word representation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, (Doha, Qatar), pp. 1532–1543, Association for Computational Linguistics, 2014.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, (Minneapolis, Minnesota), pp. 4171–4186, Association for Computational Linguistics, 2019.
- [15] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, (Online), pp. 1037–1042, Association for Computational Linguistics, 2020.
- [16] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, (Berlin, Germany), pp. 1715–1725, Association for Computational Linguistics, 2016.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 5998–6008, Curran Associates, Inc., 2017.
- [18] T. Lin, Y. Wang, X. Liu, and X. Qiu, “A survey of transformers,” *AI Open*, vol. 3, pp. 111–132, 2022.

- [19] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2383–2392, 2016.
- [20] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A robustly optimized BERT pretraining approach,” in *arXiv preprint arXiv:1907.11692*, 2019.
- [21] V. N. Nguyen, D. Van, and H. Nguyen, “A Vietnamese text classification system based on Naive Bayes classifier,” in *Proceedings of the 4th International Workshop on Vietnamese Language and Speech Processing (VLSP 2016)*, (Da Nang, Vietnam), 2016.
- [22] S. T. Luu, K. V. Nguyen, D.-V. Nguyen, and N. L.-T. Nguyen, “A vietnamese fake news detection system based on phobert,” *VNU Journal of Science: Computer Science and Communication Engineering*, vol. 37, no. 2, pp. 1–10, 2021.
- [23] K. Tran, S. Van Nguyen, and L. M. Nguyen, “A vietnamese fake news detection system,” in *Proceedings of the 2020 RIVF International Conference on Computing and Communication Technologies*, 2020.
- [24] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [25] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [26] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [27] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, pp. 1321–1330, 2017.
- [28] M. Pakdaman Naeini, G. F. Cooper, and M. Hauskrecht, “Obtaining well calibrated probabilities using Bayesian binning into quantiles,” in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, pp. 2901–2907, AAAI Press, 2015.
- [29] T. Vu, D. Q. Nguyen, D. Q. Nguyen, M. Dras, and M. Johnson, “VnCoreNLP: A Vietnamese natural language processing toolkit,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, (New Orleans, Louisiana), pp. 56–60, Association for Computational Linguistics, 2018.

- [30] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” in *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [31] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR 2019)*, 2019.
- [32] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 4765–4774, 2017.
- [33] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *NIPS Deep Learning and Representation Learning Workshop*, 2015.

PHỤ LỤC

Phụ lục A: Thống kê chi tiết bộ dữ liệu

A.1. Số mẫu theo lớp của từng tập

Bảng 5.15: Số mẫu theo lớp (Thật / Giả) trong ba tập sau khi chia phân tầng với `random_state=42`.

Tập	<i>thật</i> (0)	<i>giả</i> (1)	Tổng
Train	5.434	5.434	9.770
Val	1.165	929	2.094
Test	1.165	929	2.094

Sau khi chia tập, nguy cơ rò rỉ dữ liệu được kiểm tra theo cả trường `text` và trường `id`; kết quả cho thấy cả hai phép giao đều có kích thước bằng 0, đảm bảo tính độc lập giữa các tập.

A.2. Ví dụ mẫu trong tập kiểm tra

Mẫu tin giả (`label = 1`, ngắn):

Học_sinh có_thể nhớ bài tốt hơn nếu đặt sách_giáo_khoa dưới gối trong lúc ngủ.
(*id=9274*)

Mẫu tin giả (`label = 1`, dài):

“Một thử_nghiệm tại trường_học cho thấy việc nghe nhạc bằng tai_nghe trong 15 phút trước khi kiểm_tra có_thể làm tăng khả_năng ghi_nhớ của

học_sinh lên gần 40%. Các giáo_viên vì vậy được khuyến_nghị cho học_sinh nghe nhạc trước mỗi bài_thi... ” (id=6385, Viện Nghiên cứu Giáo dục, về phương pháp học tập)

Mẫu tin giả (label = 1):

Tầng ozone đã hoàn_toàn biến mất vào năm 2023 do sự nóng lên của mặt_trời. (id=8262)

Mẫu tin giả giật tít (label = 1):

Chính_quyền cấp xã luôn đi_tắt_đón_đầu dân. Dân chờ cả ngày trời để lãnh “hỗ_trợ” 80 ngàn đồng rồi bị trấn_lột tại_chỗ. (id=3126)

A.3. Pipeline tiền xử lý

Quy trình làm sạch bao gồm bốn bước theo thứ tự: (i) loại bỏ các bản ghi trùng lặp dựa trên cột `text`; (ii) loại bỏ những bản ghi chứa ít hơn `min_word_count = 10` từ; (iii) chuẩn hóa cột `date` về định dạng ISO 8601; (iv) đánh lại chỉ mục `id` tuần tự từ 1.

Phân đoạn từ ưu tiên sử dụng `py_vncorenlp` (`RDRSegmenter` — cùng tokenizer với `PhoBERT`); hệ thống tự động chuyển sang `underthesea.word_tokenize` trong trường hợp thư viện chính không khả dụng.

Phụ lục B: Cấu hình huấn luyện chi tiết

Toàn bộ siêu tham số được tập trung tại file cấu hình chung của dự án, được đọc thông qua singleton `cfg`.

Bảng 5.16: Cấu hình TF-IDF, Logistic Regression và SVM.

Nhóm	Siêu tham số	Giá trị
TF-IDF	<code>max_features</code>	40.000
	<code>ngram_range</code>	(1, 2)
	<code>min_df</code> / <code>max_df</code>	5 / 0,95
	<code>sublinear_tf</code>	True
LR	C (tốt nhất)	10 (solver=saga, L2)
	<code>max_iter</code>	3000
	<code>class_weight</code>	balanced
	Lưới tìm kiếm	$C \in \{0,5; 1; 5; 10; 20; 50\}$
SVM	C (tốt nhất)	0,5 (LinearSVC + Platt)
	<code>class_weight</code>	balanced
	Lưới tìm kiếm	$C \in \{0,1; 0,5; 1; 2; 5; 10\}$

Bảng 5.17: Cấu hình BiLSTM và PhoBERT.

Mô hình	Siêu tham số	Giá trị
BiLSTM	Embedding dim	300 (FastText cc.vi.300)
	Hidden dim / Layers	128 / 1
	Max seq length	128
	Dropout / Weight decay	0,3 / 10^{-4}
	Learning rate	10^{-3} (AdamW)
	Batch / Epochs	64 / 20 (patience=5)
	Gradient clip	<code>max_norm</code> = 1,0
PhoBERT	<code>model_name</code>	vinai/phobert-base
	Max seq length	256
	Dropout	0,1
	LR / Weight decay	3×10^{-5} / 0,01
	Batch / Grad accum.	16 / 4 (effective ≈ 64)
	Epochs / Patience	8 / 2
	Layer LR decay	0,95

B.1. Knowledge Distillation

Mô hình sinh viên là `StudentBiLSTM` — một phiên bản BiLSTM rút gọn với `embedding_dim` = 64, `hidden_dim` = 64, `num_layers` = 1. Trọng số soft target $\alpha = 0,7$, nhiệt độ phân phối $T = 4$. Optimizer sử dụng AdamW với `lr`= 10^{-3} , scheduler CosineAnnealingLR. Huấn luyện tối đa 8 epoch với early stopping (`patience`=5). Hàm mất mát KD chuẩn (Hinton et al., 2015)

đã được trình bày ở Mục 4.7; mô hình sinh viên được huấn luyện trên tập logits giáo viên đã được tính trước và lưu lại nhằm tiết kiệm chi phí tính toán.

B.2. Post-hoc Calibration

Ba phương pháp hiệu chỉnh xác suất — đều được khớp trên tập validation và áp dụng lên tập kiểm tra:

- **Platt scaling:** hồi quy logistic một chiều trên logits.
- **Temperature scaling:** tối ưu vô hướng $T \in [10^{-3}, 10^3]$ bằng thuật toán L-BFGS-B.
- **Isotonic regression:** ánh xạ đơn điệu phi tham số trên $\sigma(z)$.

Giá trị T đã khớp tương ứng: LR = 1,00, SVM = 0,73, BiLSTM = 0,38, PhoBERT = 0,55.

Phụ lục C: Kết quả chi tiết

C.1. Metric tổng hợp trên tập test

Bảng 5.18: Accuracy, Precision, Recall và F1-macro trên tập test.

Mô hình	Accuracy	Precision	Recall	F1-macro
Logistic Regression	0,8348	0,8325	0,8333	0,8329
SVM	0,8434	0,8418	0,8404	0,8410
BiLSTM	0,8252	0,8228	0,8235	0,8232
PhoBERT	0,9007	0,9018	0,8968	0,8988

C.2. Metric theo lớp

Bảng 5.19: Precision, Recall và F1 theo lớp (Thật / Giả) trên tập test.

Mô hình	Lớp Thật (0)			Lớp Giả (1)		
	Prec	Rec	F1	Prec	Rec	F1
Logistic Regression	0,8552	0,8464	0,8507	0,8098	0,8202	0,8150
SVM	0,8538	0,8670	0,8603	0,8299	0,8138	0,8217
BiLSTM	0,8459	0,8386	0,8422	0,7998	0,8084	0,8041
PhoBERT	0,8945	0,9313	0,9125	0,9092	0,8622	0,8851

C.3. Confusion matrix trên tập test

Bảng 5.20: Số mẫu TN, FP, FN, TP của bốn mô hình trên tập test.

Mô hình	TN	FP	FN	TP	Tổng lỗi
Logistic Regression	985	179	167	992	346 (16,52%)
SVM	1.010	155	173	756	328 (15,66%)
BiLSTM	977	188	178	751	366 (17,48%)
PhoBERT	1.085	80	128	801	208 (9,93%)

C.4. Khoảng tin cậy Bootstrap 95%

Khoảng tin cậy được đo bằng phương pháp bootstrap với 10,000 lần lặp.

Bảng 5.21: Khoảng tin cậy 95% cho F1-score trên tập test, ước lượng bằng bootstrap với 10,000 lần lặp.

Mô hình	95% CI (F1-score)
Logistic Regression	[0,8165; 0,8487]
SVM	[0,8253; 0,8564]
BiLSTM	[0,8063; 0,8391]
PhoBERT	[0,8853; 0,9114]

C.5. Knowledge Distillation – chi tiết

Bảng 5.22: So sánh hai mô hình giáo viên (PhoBERT, BiLSTM) và sinh viên BiLSTM ($\alpha = 0,7$, $T = 4$).

Mô hình	F1 test	Params	Kích thước (MB)
PhoBERT (giáo viên)	0,8988	~135M	540
BiLSTM (giáo viên)	0,8232	~500K	24,57
Student BiLSTM (KD)	80,51	1,549 442	5,91

Tỷ lệ nén so với PhoBERT là 87,1×; so với BiLSTM là 4,2×. Tốc độ suy diễn đạt 1,2 ms, nhanh hơn 3,8×; mô hình sinh viên giữ được 97,8 phần trăm chất lượng của BiLSTM.

C.6. Attribution Faithfulness

Faithfulness là mức giảm xác suất trung bình khi che **top-5** token được attribution đánh dấu trên tập 20 mẫu.

Bảng 5.23: Faithfulness của sáu phương pháp attribution.

Phương pháp	Faithfulness
LR – SHAP	0,34
SVM – SHAP	0,31
BiLSTM – Integrated Gradients	0,19
BiLSTM – Simple Gradients	0,12
PhoBERT – SHAP	0,27
PhoBERT – Integrated Gradients	0,29
PhoBERT – Attention Rollout	0,08

C.7. Cross-validation 5-fold

Bảng 5.24: Accuracy và F1 trung bình qua 5-fold cross-validation (3 seed) cho hai mô hình truyền thống.

Mô hình	Accuracy	$\pm$ std	F1-macro	$\pm$ std
Logistic Regression	0,8301	0,0087	0,8282	0,0087
SVM	0,8490	0,0087	0,8469	0,0087

C.8. Hard cases – phân bố

Tập 85 mẫu khó (cả bốn mô hình đều phân loại sai) được phân bố theo lý do thất bại suy đoán như sau:

Bảng 5.25: Phân bố 85 mẫu khó theo lý do thất bại suy đoán.

Lý do	Số mẫu
dataset_ambiguity	47
knowledge_verification	14
semantic_reasoning	6
Chủ đề politics	19 (phân bố theo chủ đề)
Độ dài short (<30 từ)	59

Phụ lục D: Tái lập thực nghiệm

D.1. Yêu cầu môi trường

Python numpy 2.4, pandas 3.0, scikit-learn 1.8,
 scipy 1.17, torch 2.10, transformers 4.57,
 sentencepiece 0.2, fasttext-wheel 0.9.2.

Tiếng Việt py_vncorenlp 0.1.4 (Java ≥ 11)
 hoặc underthesea 9.1 (dự phòng).

Giải thích shap ≥ 0.45 , captum ≥ 0.7 .

Biến môi trường cần thiết: JAVA_HOME trỏ tới JRE. PhoBERT (vinai/phobert-base) và FastText (cc.vi.300.bin) được tải tự động từ HuggingFace và lưu tại thư mục data/fasttext/.

D.2. Pipeline huấn luyện – đánh giá

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m src.preprocessing.word_segmentation
python -m src.preprocessing.split_data
python -m src.features.extract_all_features
python -m src.training.train_all
python -m src.training.reproduce_predictions
python -m src.training.train_student
python -m src.evaluation.post_hoc_calibration
python -m src.evaluation.error_analysis
python -m src.evaluation.hard_cases
python -m src.analysis.statistical_tests
python -m src.analysis.generate_paper_tables
python -m src.analysis.generate_paper_figures
cd paper && ./build.sh
```

D.3. Thiết lập ngẫu nhiên

Seed chính của toàn bộ pipeline là `cfg.RANDOM_STATE = 42` — được sử dụng cho `train_test_split`, `GridSearchCV` (qua `StratifiedKFold(shuffle=True)`) và các script huấn luyện. Hàm `set_reproducibili` cố định đồng thời các seed của `random`, `numpy`, `torch` và `torch.cuda`; hàm này được gọi ngay đầu mỗi trainer nhằm đảm bảo kết quả hoàn toàn tái lập.

D.4. File cấu hình

Toàn bộ siêu tham số được tập trung tại file cấu hình chung và import qua singleton `cfg`, bao gồm các khối: `PATHS`, `DATA` (tỉ lệ chia tập, `min_word_count = 10`), `TFIDF`, `LR`, `SVM`, `BILSTM`, `PHOBERT`, `ANALYSIS` (`bootstrap_iterations = 10,000`, `significance_level = 0,05`).

Mọi giá trị hiển thị trong bài báo được định nghĩa tại `data_commands.sty` dưới dạng `\newcommand`, cho phép cập nhật đồng bộ từ một vị trí duy nhất — tránh sai lệch giữa các phần của tài liệu.